VANIER
CÉGEP/COLLEGE

Project Planning & Prototyping

J. F. N. Salik,

247-519-VA

Department of Computer Engineering Technology
Vanier College
821 Sainte-Croix Avenue
Saint-Laurent, Québec H4L 3X9

This document was last modified on
Saturday 30th May, 2026 at 11:42am.

Title	**Project Planning & Prototyping**
Date	May 30, 2026
Author	J. F. N. Salik, (salikj@vaniercollege.qc.ca)
Identification	247-519-VA
Template	Sparkle Version 1.1 for LaTeX 2ε

VANIER
CÉGEP/COLLEGE

Contents

CHAPTER 1

OVERVIEW: PROTOTYPES, PRODUCTS & THE BUSINESS OF ENGINEERING

Every project begins with a claim: *this idea is worth building.* Your job in Project I is to find out whether that claim is true—cheaply, quickly, and with enough documentation that the answer is defensible. By the end of this course you will have built a working proof-of-concept, assembled the record that shows how you got there, and produced the business case that connects your hands-on work to the real world of engineering firms and markets. This chapter gives you the map. You will see what you are building, why it matters, what separates a prototype from a finished product, and what turns a validated prototype into revenue.

Learning Objectives

1. State what a prototype is for and describe the two deliverables you will produce in Project I.
2. Describe how a proven prototype becomes a product and identify the link between Project I and Project II.
3. Explain your role as a technician working alongside engineers.
4. Explain why industry work is justified by the value it creates, and interpret a negative economic finding correctly.
5. Identify the principal business models that turn a validated prototype into revenue.

1.1 What You Will Build

You will build two things: a **proof-of-concept prototype** and **the record of how you got there.**

The prototype is a working artifact that answers a specific technical and business question. It does not need to be pretty, manufacturable, or certifiable. It needs to

demonstrate that the core idea functions under realistic conditions.

The record is a portfolio of documents—specification, risk register, architecture diagrams, build logs, test data, and cost analysis—that accumulate as you work. You do not write the portfolio at the end. You assemble it from artifacts you produce at each stage. If you work that way, the portfolio is nearly complete before you realize you are making one.

Together, the prototype and its record constitute your output from Project I. The record is as important as the hardware: an engineer can reproduce, evaluate, and extend documented work; undocumented work is disposable.

1.2 The Course Map: Two Tracks Over Twelve Weeks

Figure 1.1 shows the full engineering journey from concept to market. Project I occupies the second stage—the prototype—and Project II the third, where the proven concept is developed into a manufacturable product. The arrow between them is not automatic: what crosses it is the documentation you create in Project I.

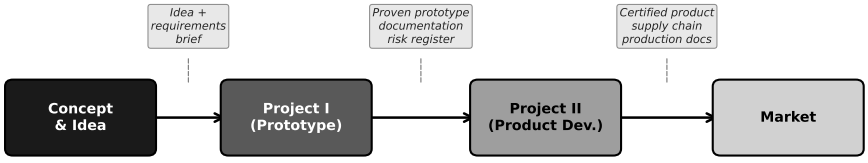

Figure 1.1: The engineering pipeline from concept to market. Project I delivers the proven prototype and its documentation; Project II turns that foundation into a manufacturable product.

The course runs two **parallel tracks**: a theory track and a lab (practice) track. Figure 1.2 shows how they align across the twelve weeks.

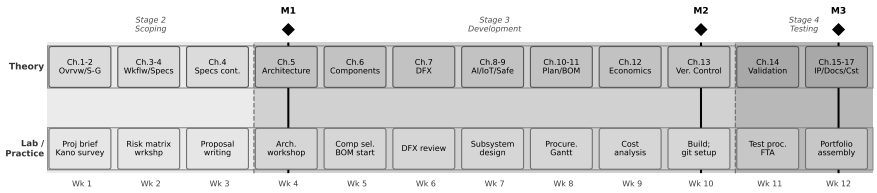

Figure 1.2: The two-track course structure over twelve weeks. The theory track introduces each topic just before the lab requires it; the seven workflow stages unfold across the practice track, with three graded milestones (M1–M3).

The theory track moves through the seventeen chapters of this book at roughly one to two chapters per week, with Chapter 1 and Chapter 2 paired in Week 1. The lab track runs the **seven-stage Prototype Development Cycle**, which you will study in depth in Chapter 3. At the overview level, the seven stages are:

1. **Specification & Risk** — translate the idea into measurable requirements and a

risk ranking.

2. **Scoping & Planning** — decide what you will build, set a schedule, and identify long-lead procurement.
3. **Architecture** — decompose the system into subsystems and define their interfaces.
4. **Subsystem Design** — make detailed design decisions for each subsystem.
5. **Build & Integration** — fabricate and assemble; bring subsystems together.
6. **Validate & Measure** — test against the specification; collect data; decide whether to iterate.
7. **Compile & Economics** — assemble the portfolio and analyse the cost and business case.

Three milestones punctuate the semester. **Milestone 1** (Proposal, end of Week 3) delivers your specification and risk register. **Milestone 2** (Design Review, end of Week 9) delivers your architecture, build records, and preliminary cost analysis. **Milestone 3** (Final Portfolio, end of Week 12) delivers the complete project record.

The theory topics are sequenced so that each week's reading lands just as the lab needs it: you study specifications in Week 3, then immediately write your own in the lab. This is not a coincidence—it is the design of the course.

1.3 From Prototype to Product: The Gap

A prototype and a product solve different problems. A **prototype** answers a question about technical feasibility and business viability. A **product** is an artifact designed to be made reliably, in volume, at a cost that leaves margin, by people who were not involved in the original design, and supported over a service life that may span years.

The gap between them is real and wide. A successful prototype does not automatically become a product. The following properties must be designed in—not bolted on afterward:

Manufacturability. Can the design be made in volume using available processes, tooling, and supply chains? Hand-soldered one-of-a-kind assemblies are not manufacturable at scale.

Reliability. Does the design meet a minimum operating life under realistic field conditions? A prototype that runs for an hour in a lab may fail within a week in the field due to thermal cycling, vibration, or humidity.

Cost at volume. Is the per-unit cost, at the production volumes the business requires, low enough to leave an acceptable margin? Chapter 10 develops the cost model in full.

Certification. Does the design meet applicable electrical safety, radio emissions, and (for IoT and robotics) functional safety standards? Certification is a legal prerequisite in most markets, not an optional finishing step.

Supportability. Can the product be diagnosed and repaired by field technicians? Are replacement parts available over the product lifetime? Is the documentation sufficient for a technician who was not present at the original build?

None of these properties is addressed in Project I. That is not a failure; it is appropriate scope management. Your job in Project I is to answer the central technical and business question as quickly and cheaply as possible. Project II then takes that answer as its starting point and builds the full product around it. What crosses the boundary is your documentation: the specification, the architecture, the risk register, the validated design decisions. The formal process by which a prototype earns the right to become a product — Stage-Gate — is introduced in Chapter 2.

1.3.0.0.1 In Practice. A team building a wearable health monitor completes a functional prototype in twelve weeks: it reads heart rate, displays data on a small screen, and transmits to a phone app. The team is pleased—and surprised, six months later, to learn that the product redesign for manufacturing took an additional eighteen months. The enclosure needed injection-moulding tooling. The radio module required FCC and IC certification. The battery management circuit had to be redesigned for the thermal envelope of a wearable. The prototype proved the concept; the product required a separate, larger effort. Neither effort was wasted.

1.4 Working with Engineers: What You Own

In industry you will work on a team that includes engineers. Understanding who owns what prevents frustration and makes you more effective.

Engineers own:

- Requirements—what the system must do, expressed as measurable criteria.
- Design accountability—the engineer signs off on the architecture and carries legal responsibility for the design.
- Risk decisions—the call to proceed, stop, or redesign rests with the engineer of record.

You own:

- Building—breadboarding, PCB assembly, mechanical fabrication, 3D printing, cable harnesses.
- Testing—executing the test procedure, collecting and recording data honestly, flagging anomalies.
- Documenting—keeping the build log current, maintaining the as-built BOM, photographing assembly steps.
- Procuring—sourcing components, getting quotes, placing purchase orders, receiving and inspecting parts.

- Troubleshooting—diagnosing faults at the bench level.

This boundary is not about status; it reflects training and accountability. A technician who takes over design decisions without the engineer's sign-off creates liability for everyone. A technician who executes, documents, and communicates problems clearly makes the engineer's job tractable. Your hands-on judgment—knowing that a solder joint looks wrong, that a component is getting too hot to touch, that the measured current does not match the schematic—is irreplaceable. Engineers who have lost touch with the bench depend on you for that judgment.

1.5 Engineering Businesses by Size

The same prototype, handed to three firms of different sizes, will be handled very differently. Figure 1.3 illustrates how process formality, documentation weight, and approval layers scale with firm size.

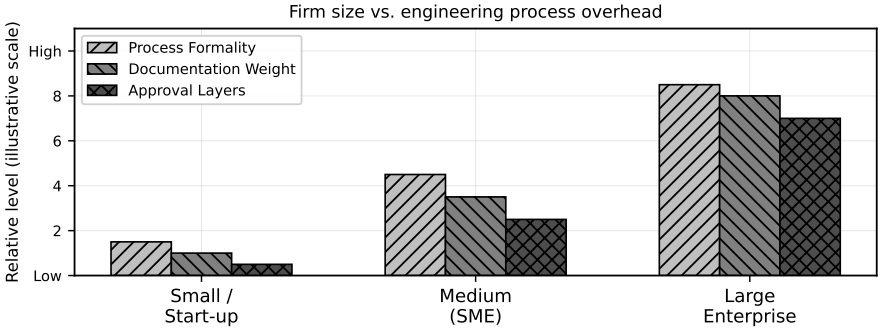

Figure 1.3: Firm-size spectrum: process formality, documentation weight, and approval layers all increase substantially from small start-ups to large enterprises. The prototype development approach must match the firm context.

Small firms and start-ups (under ≈20 people) move fast and tolerate ambiguity. A single engineer may own the entire design. Documentation is minimal—often just enough to reproduce the build. The prototype is frequently made by the same people who conceived it. Speed matters more than process. The risk is that institutional knowledge lives in people's heads and disappears when they leave.

Medium firms (SMEs) (20–500 people) have begun to institutionalise. There are defined roles, some formal review gates, and more documentation requirements. A design change needs sign-off from more than one person. The prototype team is likely distinct from the product team, so documentation quality directly affects the handoff.

Large enterprises (500+ people) operate under heavy process—ISO standards, stage-gate reviews, change control boards, formal test plans, and full configuration management. A prototype at a large firm may require months of internal approval before the first component is ordered. Documentation must be complete, traceable, and auditable.

None of these contexts is inherently better. A start-up that adopts enterprise process collapses under bureaucratic overhead. A large firm that abandons process

loses traceability and produces unrepeatable results. Your job is to recognise the context you are in and calibrate your documentation effort accordingly.

1.5.0.0.1 In Practice. A three-person IoT start-up builds a temperature monitoring module in four weeks: breadboard, firmware, cloud logging, mobile app. The "documentation" is a shared folder with photos and a text file. Two years later, the same company has grown to forty people. A new hire tries to reproduce the original circuit for a customer request. The photos are there; the design rationale is not. It takes three weeks to reverse-engineer the component selection. The original four weeks of work left a four-week debt that compounded with every new hire.

1.6 Science for Profit: Cost, Time, and Volume

Engineering work is not science for its own sake—it is science directed at creating value. This does not diminish the intellectual content; it sharpens the questions. In a commercial context, every technical decision carries an economic consequence, and the three most important constraints are **cost**, **time**, and **volume**.

Cost constrains what you can build and at what margin. A solution that is technically elegant but costs twice the target variable cost is not a viable product.

Time constrains competitive advantage. A prototype finished six months late may miss a market window entirely. Time is the one resource you cannot recover.

Volume determines the economic structure. At low volumes, fixed costs (tooling, engineering, certification) dominate the per-unit cost. At high volumes, variable costs dominate. Chapter 10 develops this model formally; here, it is enough to recognise that the prototype's per-unit cost—often assembled by hand from individual components—tells you nothing about what the product will cost at 1,000 units.

A critical professional skill is the ability to interpret a negative economic finding correctly. Suppose your prototype proves that the design works but the cost analysis shows that no viable margin exists at any realistic production volume. This is not failure—it is the most valuable result the prototype could have produced. You have saved the firm the cost of a full product development effort on an unviable design. Document the finding clearly, state the assumptions, and let the engineering team decide whether to pivot or stop. A prototype that says "no" is doing exactly what a prototype is for.

1.7 How a Prototype Becomes Revenue

Validating a prototype is not the end of the story. The business question that drives the prototype is always, ultimately: how does this create value that someone will pay for? Figure 1.4 maps the major revenue models available once a prototype has been validated.

Product sale (direct). The most familiar model: manufacture a unit, sell it for

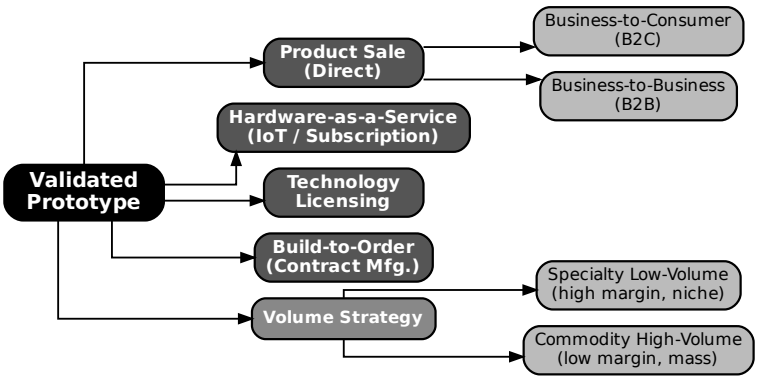

Figure 1.4: Revenue models reachable from a validated prototype. The choice of model shapes what the prototype must prove and what the product development effort must deliver.

more than it costs to make and sell. The two dominant sub-types differ in who the customer is. In a **business-to-consumer (B2C)** model, you sell to individuals; volume is potentially high but margins are typically lower and customer acquisition is expensive. In a **business-to-business (B2B)** model, you sell to companies; volumes are lower but unit prices and margins are generally higher, and relationships carry more weight than brand.

Hardware-as-a-Service (HaaS / IoT subscription). The customer pays an upfront hardware fee and a recurring subscription for connectivity, data processing, and support. The prototype must prove both the hardware function *and* the cloud-connectivity architecture. This model creates recurring revenue and customer lock-in but requires ongoing operational cost for the cloud backend. It is common for IoT devices that produce data the customer values continuously.

Technology licensing. The firm does not manufacture; it licenses the design, firmware, or algorithm to other manufacturers. The prototype proves that the intellectual property is real and valuable. Licensing requires strong IP documentation and often patents. Beyond licensing, intellectual property ownership is a business-model consideration for every revenue model — Chapter 15 covers the practical IP obligations a technician must understand.

Build-to-order (contract manufacturing). The firm builds custom units against specific customer purchase orders. Volume is low per customer; the customer drives the specification. Margins can be high because the customer is paying for customisation. The prototype is typically a demonstration to win the contract.

Volume strategy: specialty vs. commodity. Within product sale, the firm chooses where on the volume spectrum to compete. **Specialty low-volume** production targets niche markets with high unit prices and high margins; low volume means less pressure on tooling costs but fewer units over which to amortise fixed costs. **Commodity high-volume** production targets large markets with low unit prices and thin margins; every cent of variable cost matters, and design-for-manufacture is mandatory from the first design decision.

The choice of revenue model is made before the prototype is designed, not after. It shapes what the prototype must prove, what fidelity is needed, and what documentation must be produced. A HaaS prototype that lacks a working cloud connection has not answered the central business question. A licensing prototype with no IP documentation has nothing to license.

Worked Example: The Sorting Subsystem Business Case

Throughout this book, every chapter works through the same running example: a **connected IoT robotic sorting subsystem**—a sensor-guided actuator that classifies items on a bench conveyor, diverts them to the correct output lane, reports status over a network, and is designed as a module within a larger automation product.

Firm context. The team is a twelve-person engineering start-up that has identified a market in light-industrial sorting for food processing and pharmaceutical packaging. The firm is too small for enterprise process but large enough to have separate engineering and operations functions. Documentation weight is medium: enough to enable reproduction by another team member, not enough to require formal configuration management.

Business model. The chosen model is **Hardware-as-a-Service**: customers purchase the sorter module at a hardware price and pay an annual subscription for cloud-hosted analytics, remote monitoring, and firmware updates. This model requires the prototype to prove two things: (1) the mechanical and electronic sorter works reliably, and (2) the cloud connectivity architecture is sound and can sustain the required data throughput.

What the prototype must answer. The central question is: *Can a sub-$500 CAD bill-of-materials sorter module achieve $\geq$95% sort accuracy at 6 items/min while maintaining a reliable MQTT connection over a standard Wi-Fi network?* If yes, the hardware is viable and the HaaS model can proceed. If no, the firm needs to know whether the failure is in the mechanical design, the sensing, the firmware, or the network layer—so the portfolio must be structured to show that.

Margin calculation. Suppose the firm targets a hardware unit price of $p = \$1{,}200$ CAD and estimates the variable cost of a production unit (materials + direct labour at volume) at $c_{\text{var}} = \$480$ CAD. (These are early estimates; Chapter 10 will refine them using the full cost model.)

The unit margin is:

$$m = p - c_{\text{var}} = \$1{,}200 - \$480 = \$720 \text{ CAD/unit.} \tag{1.1}$$

The gross margin fraction is:

$$\text{GM} = \frac{p - c_{\text{var}}}{p} = \frac{\$720}{\$1{,}200} = 0.60 \quad (60\%). \tag{1.2}$$

A 60% hardware gross margin is strong for an electromechanical module. It must, however, sustain both the hardware warranty cost and the ongoing cost of the cloud subscription infrastructure. Chapter 10 will introduce the full unit-cost model,

$C_{\text{unit}}(N) = C_{\text{fixed}}/N + c_{\text{var}}$, which shows how the fixed non-recurring costs (tooling, firmware development, certification) dilute this margin at low production volumes.

Project context brief (your first portfolio artifact).

- *Project:* IoT robotic sorting subsystem module.
- *Business model:* Hardware-as-a-Service; hardware sale + annual cloud subscription.
- *Firm context:* 12-person engineering start-up; medium documentation weight.
- *Business question the prototype must answer:* Can the module achieve $\geq$95% sort accuracy at 6 items/min with a reliable cloud connection, at a BOM cost below $500 CAD?

Try It Yourself

A micro-robotics start-up is developing a vision-guided pick-and-place module. They plan to sell it directly to electronics assembly firms (B2B product sale). Their target unit price is $p = \$950$ CAD and their estimated variable cost is $c_{\text{var}} = \$285$ CAD.

1. Compute the unit margin $m = p - c_{\text{var}}$.
2. Compute the gross margin fraction $\text{GM} = (p - c_{\text{var}})/p$ and express it as a percentage.
3. Using Figure 1.4, identify which revenue model applies and state one thing the prototype must prove to make that model viable.

(Answers not provided — work through the arithmetic before moving on.)

1.7.0.0.1 Common Pitfalls. Technically impressive $\neq$ good prototype. A prototype that demonstrates six different features but answers none of the business questions clearly is not a good prototype. Novelty and complexity are not success criteria. The success criterion is: *did you answer the question?*

"It works" $\neq$ "the business case is proven." A prototype that demonstrates function but carries a BOM cost of $1,400 CAD against a $1,200 target price is not a viable product, regardless of how well it works. The economic analysis is not an afterthought—it is half of the prototype's output.

Underrating the technician's judgment. Engineers rely on your observation that the actuator is running hotter than expected, that the network connection drops in the presence of the motor drive, or that a component you received does not match the datasheet. These observations, documented and communicated promptly, are the inputs that prevent costly mistakes. Do not wait to be asked.

Reading a negative economic finding as failure. If the cost analysis shows that the HaaS model is not viable at any realistic volume, that result should be reported clearly, not buried or explained away. The firm can decide to pivot—to a different revenue model, a different component set, or a different market. It cannot decide anything if the analysis is suppressed.

Review Questions

1. **(Conceptual)** A product must satisfy five properties that a prototype does not. List all five and write one sentence explaining what each requires in practice.
2. **(Conceptual)** The chapter states that "a prototype that returns a negative economic result is not a failure." Explain this claim in your own words. What should the team do with the finding, and why is suppressing it dangerous?
3. **(Applied — calculation)** A start-up prices a wireless sensor node at $p = \$340$ CAD and initially estimates $c_{\text{var}} = \$136$ CAD.
   (a) Compute m and GM%.
   (b) After a detailed BOM review, the true variable cost is $c_{\text{var}} = \$204$ CAD. Recompute m and GM%.
   (c) By how many percentage points did the gross margin change? Comment on whether the product remains financially attractive.
4. **(Applied — classification)** A company has developed a machine-learning algorithm that identifies crop disease from smartphone photos. They plan to license it to agricultural equipment manufacturers. Using Figure 1.4 and the definitions in §1.7, identify the revenue model and state two things the prototype must demonstrate to support that model.
5. **(Applied — multi-step)** A proof-of-concept prototype costs $2,400 CAD to build. The team estimates that the eventual product will sell at $p = \$600$ CAD with $c_{\text{var}} = \$180$ CAD.
   (a) Compute m and GM%.
   (b) The $2,400 build cost is *not* c_{var}. Name the cost category it belongs to, and explain why it does not appear in the margin formula at this stage. (*Hint: Chapter 10 will introduce the full cost model.*)

Portfolio Entry

By the end of Week 1, you will produce a one-page **project context brief** for your own project. It is the first document in your portfolio and frames everything that follows. It must contain:

1. A description of the project (one paragraph, no jargon).
2. The chosen business model, with a one-sentence justification.
3. The firm context (size, documentation standard, your role).
4. The single business question the prototype must help answer.
5. A preliminary margin estimate: state p, c_{var}, compute m and GM%, and note the assumptions behind c_{var}.

Keep the brief to one page. Its purpose is not comprehensiveness but clarity: after reading it, a second team member should be able to say whether the project is technically and commercially plausible at a first glance.

Summary

Project I delivers two things: a proof-of-concept prototype and the portfolio that documents how you built and tested it. The prototype's job is to answer one specific technical and business question—not to be a miniature product. Between Project I and Project II lies a deliberate gap: manufacturability, reliability, cost at volume, certification, and supportability must all be designed into the product in Project II, using the prototype's answers as a foundation.

As a technician, you own the build, the test, the documentation, and the procurement. Your hands-on judgment is the most direct input the engineering team has to reality.

Firms handle prototypes differently depending on their size: start-ups move fast with minimal process; enterprises move carefully with heavy documentation. Calibrate your effort to the context.

The economic constraints—cost, time, and volume—are design variables, not external obstacles. A prototype that returns a negative economic result is not a failure; it is evidence. The revenue model chosen before the prototype is built shapes everything: what the prototype must prove, what fidelity is required, and what documentation must be produced.

Up next: Chapter 2 establishes the formal product development process — the Cooper Stage-Gate model and Kano analysis — that gives every subsequent chapter its professional context. You will see where this course and Project II sit within that model, and how Kano analysis determines what the prototype must prove.

CHAPTER 2

PRODUCT DEVELOPMENT & THE STAGE-GATE MODEL

Chapter 1 established the prototype–product distinction, the business case (margin m, gross-margin GM%), the five revenue models, and the product pipeline. That left open a harder question: *how do you decide which prototype to build?* Every project has more potential features than budget and time allow. Build the wrong ones and you spend twelve weeks proving something nobody doubted, while the real technical risk goes untested. This chapter addresses that selection problem directly. You will learn the professional framework that structures product development from first idea to market launch—the Cooper Stage-Gate model—and the quantitative tool that tells you which features are worth prototyping—Kano analysis. Together they give you a principled answer to the question Chapter 1 left open.

Learning Objectives

1. Identify the five Stage-Gate stages and explain the go/kill/hold/recycle decision that each gate produces.
2. Identify the deliverables required to pass Gate 3 and explain why the prototype must address each one.
3. Classify product features using the five Kano categories.
4. Compute Kano satisfaction and dissatisfaction coefficients from survey data.
5. Derive prototype scope by combining Stage-Gate position with Kano category results.

2.1 The Product Development Lifecycle

Before any formal model, consider the raw sequence every new product moves through. A product begins as a **concept**: someone articulates a need, a technology, or an opportunity. The concept is then **scoped**: the team estimates whether

it is technically feasible and commercially viable at a level of effort the organisation can sustain. If scoping is positive, the team builds the **business case**—a document that combines market analysis, cost modelling, and technical risk assessment to justify the development investment. **Development** follows: design, prototyping, iteration. The working artifact then enters **testing and validation**, where it must perform against real conditions and real user expectations. Passing validation leads to **launch**, after which the product enters **sustain**: ongoing operations, support, and incremental improvement.

These phases are not watertight compartments. In practice, information discovered in testing forces revisions to design; market feedback during launch reshapes the sustain roadmap. What the sequence provides is a shared vocabulary—so that when someone says "we are in development," everyone in the room understands the current risk profile, the decisions being made, and what evidence is needed to move forward.

2.1.0.0.1 In Practice. A 12-person engineering start-up selling a Hardware-as-a-Service sorter at $1 200 CAD per unit cannot afford to discover a fatal technical flaw at launch. The lifecycle above is not ceremony; it is a schedule of risk reduction. Each phase exists to eliminate a class of uncertainty before that uncertainty becomes expensive.

2.2 The Cooper Stage-Gate Model

Robert Cooper formalised the product development lifecycle into the **Stage-Gate model** in the 1980s. The model is now standard practice in product-oriented engineering organisations. Its central insight is simple: spend small, learn fast, decide often. Before the team commits the resources of one stage, it passes through a **gate**—a structured decision point where management reviews evidence and rules *go*, *kill*, *hold*, or *recycle*.

Figure 2.1 shows the five-stage funnel. Many projects enter; few survive to launch. Each gate narrows the funnel by eliminating projects whose evidence does not justify continued investment.

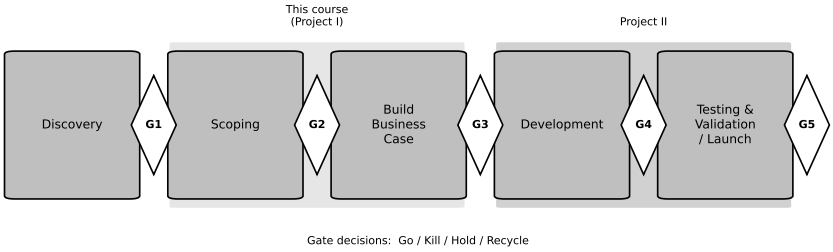

Figure 2.1: The Cooper Stage-Gate funnel. Projects enter at Discovery and are narrowed at each gate by go/kill/hold/recycle decisions. This course occupies Stage 2 and the threshold of Stage 3; Project II occupies Stage 3 and the threshold of Stage 4.

2.2.1 The Five Stages

2.2.1.1 Stage 1 — Discovery

The team generates ideas and screens them against strategic fit. Output is a short opportunity description, not a design. Most ideas die here on a whiteboard.

2.2.1.2 Stage 2 — Scoping

A small team performs rapid preliminary investigation: desktop market research, quick technical assessment, rough financial estimation. The goal is to find fatal flaws before they cost anything. Output is a scoping document that informs Gate 2.

2.2.1.3 Stage 3 — Build Business Case

This stage produces the full business case: detailed market analysis, competitive landscape, product definition, technical feasibility assessment, financial model, and risk register. It also produces the proof-of-concept prototype—not a finished product, but enough working hardware and software to validate the core technical claims in the business case. Output is the business case document and the proof-of-concept. Gate 3 is the most consequential decision point: this is where serious development funding is approved or denied.

2.2.1.4 Stage 4 — Development

The full project team builds the designed product. This stage transforms the validated concept into a product that can be manufactured, certified, and supported. Output is a developed and tested product ready for validation.

2.2.1.5 Stage 5 — Testing & Validation

The product is tested in the field under real conditions with real users. Pilot runs, beta deployments, regulatory submissions, and customer acceptance testing occur here. Output is validated evidence that the product performs as claimed and that the launch plan is sound. Gate 5 approves commercial launch.

After Gate 5, the product launches and enters the sustain phase described in Section 2.1.

2.2.2 The Gate Decisions

Each gate produces one of four decisions. **Go** means the project proceeds to the next stage with approved resources. **Kill** means the project is cancelled; resources are redeployed. **Hold** means the project is paused pending additional information or a change in conditions. **Recycle** means the team must revise and resubmit the current stage's work before proceeding.

Kill and recycle decisions are not failures. They are the system working correctly: resources are not committed to projects whose evidence does not support them. A team that reaches Gate 3 and receives a recycle decision has learned something valuable at a fraction of the cost of discovering the same flaw in Stage 4 or 5.

2.2.3 Where This Course and Project II Sit

This course, Project I (247-519-VA), occupies **Stage 2 and Stage 3 of the Stage-Gate model.** You are performing the scoping work of Stage 2 and producing the proof-of-concept prototype and business case documentation required to pass **Gate 3.**

Project II continues from that point. It occupies **Stage 3 transitioning into Stage 4**—taking the Gate 3-approved concept and developing it into a manufacturable product ready for **Gate 4** review.

This placement matters. Your prototype does not need to be a finished product. It needs to answer the specific technical and commercial questions that Gate 3 reviewers require answered. Everything else is scope creep. The next section identifies precisely what those questions are.

2.3 What a Gate Decision Requires

Gate 3 reviewers are not evaluating your hardware. They are evaluating evidence. Specifically, they need three things.

A business case that identifies the market, the value proposition, the revenue model, and the financials. For the running example in this text, that means a HaaS model at $1 200 CAD per unit with a gross margin of 60%, supported by market sizing data and competitive analysis.

A resource plan that specifies what the development stage will cost in people, equipment, time, and capital. Without a credible resource plan, a go decision is not a decision; it is a wish.

Validated-prototype evidence that the core technical claims in the business case are true. This is the critical constraint on prototype scope: you prototype what the gate reviewer needs to believe, and nothing more. Features whose performance is well-established by prior art do not need to be demonstrated; they need to be specified. Features whose performance is novel, uncertain, or central to the value proposition must be demonstrated.

2.3.0.0.1 In Practice. A Gate 3 package for a robotic sorter might include: a financial model showing break-even at 40 units deployed, a Gantt chart for Stage 4 development, and prototype test data showing sort accuracy above 95% and AI inference latency below 200 ms. It would *not* include a photo of a polished enclosure. The enclosure is a threshold feature (Section 2.4); reviewers assume it will be built. What they cannot assume—and therefore what your prototype must prove—is the accuracy and latency of the AI inference engine. The Kano analysis table (Table 2.2) is the document that justifies this selection to Gate 3 reviewers: it shows

which features were surveyed, what the coefficients are, and why the three prototyped features were chosen over the two that were not.

2.4 Kano Analysis

Knowing that you prototype what Gate 3 needs to see answers the question at the level of the business process. It does not tell you—feature by feature—which ones carry that weight. That is what **Kano analysis** does.

Noriaki Kano published his model in 1984. The model rejects the assumption that satisfaction and dissatisfaction are simply opposites on a single scale. Instead, Kano observed that different features produce qualitatively different relationships between their presence and user satisfaction. Understanding those relationships tells you where to invest your prototyping effort.

Figure 2.2 shows the canonical Kano satisfaction curves. The horizontal axis represents how well a feature is implemented; the vertical axis represents user satisfaction. Each curve corresponds to one feature category.

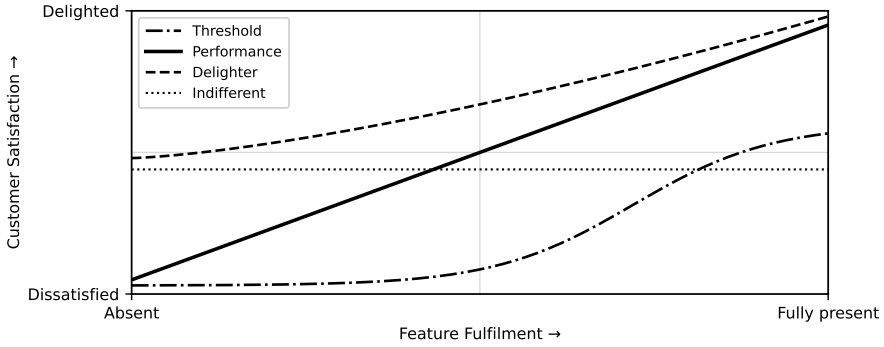

Figure 2.2: Kano satisfaction curves for the five feature categories. Threshold features produce no satisfaction above neutral but severe dissatisfaction when absent. Performance features scale linearly. Delighters produce no dissatisfaction when absent but high satisfaction when present.

2.4.1 The Five Kano Categories

2.4.1.1 Threshold (Basic) Features

Threshold features—also called **basic** or **must-be** features—are the minimum requirements for a product to be considered acceptable. When they are present, users are neutral; they simply do not notice. When they are absent or fail, users are severely dissatisfied. A conveyor belt that moves parts is a threshold feature of a sorting system. Users expect it. Exceeding it does not help you.

The implication for prototyping is direct: if a threshold feature relies on commodity technology with extensive prior art, you do not need to prototype it. You specify it, source it, and move on.

2.4.1.2 Performance (One-Dimensional) Features

Performance features produce a roughly linear relationship between implementation quality and satisfaction. More accuracy means more satisfaction; less accuracy means less. The relationship is monotone. Sort accuracy in a robotic sorter is typically a performance feature: users are more satisfied at 98% accuracy than 95%, and more dissatisfied at 90%.

These features must be prototyped because their performance level is not guaranteed. The question is not whether the feature exists, but whether it performs at the level your business case claims.

2.4.1.3 Delighter (Excitement) Features

Delighters—also called **excitement features**—are the opposite of threshold features. When absent, users are neutral; they did not expect them. When present, they produce disproportionate satisfaction. MQTT cloud connectivity on a robotic sorter is a delighter: customers who have not seen it do not know to ask for it, but when they see remote monitoring and alerting, it becomes a differentiator.

Delighters must be prototyped for two reasons. First, their technical feasibility is often unproven. Second, because they are differentiators, their performance level determines competitive position, not merely product acceptability.

2.4.1.4 Indifferent Features

Indifferent features produce negligible change in satisfaction regardless of whether they are present or how well they are implemented. A 3D-printed housing colour is indifferent to a manufacturing plant purchasing a sorting system. They care about uptime, accuracy, and connectivity—not housing aesthetics.

Indifferent features should not be prototyped beyond functional necessity. Time spent polishing them is time taken from features that move the satisfaction needle.

2.4.1.5 Reverse Features

Reverse features produce dissatisfaction when present. They are rare but real. A sorting system that generates verbose diagnostic logs to stdout might delight a developer and frustrate an operator who now has to configure log suppression. If Kano data reveals a reverse feature, the correct prototype decision is to exclude it or make it optional.

2.4.1.5.1 In Practice. Categories shift over time. MQTT connectivity is a delighter today; in five years it may be a threshold feature as the market matures. Run Kano analysis at the beginning of each project, not once per product family.

2.5 Kano Satisfaction and Dissatisfaction Coefficients

The five categories above name the shape of each feature's satisfaction curve; they do not yet say how strongly each feature pulls relative to the others. To quantify that, you compute two coefficients.

The Kano survey instrument presents each feature twice: once asking how the respondent would feel if the feature were present (*functional* question), and once asking how they would feel if it were absent (*dysfunctional* question). Respondents answer on a five-point scale: *I like it*, *I expect it*, *I am neutral*, *I can live with it*, *I dislike it*. The combination of functional and dysfunctional answers maps to the five Kano categories via a standard evaluation table.

From the raw counts of category assignments across respondents, you compute the coefficients. Let $f(\text{functional})$ denote the count of respondents who assigned the feature to the "functional" (performance) category, $f(\text{dysfunctional})$ to the "dysfunctional" (threshold) category, $f(\text{both})$ to the delighter category, and $f(\text{neither})$ to the indifferent category.

2.5.1 Satisfaction Coefficient

The **satisfaction coefficient** measures how much the presence of a feature increases satisfaction. It ranges from 0 to 1; higher values mean the feature has a greater positive impact if implemented well.

$$\text{CS} = \frac{f(\text{functional}) + f(\text{both})}{f(\text{functional}) + f(\text{dysfunctional}) + f(\text{both}) + f(\text{neither})} \tag{2.1}$$

2.5.2 Dissatisfaction Coefficient

The **dissatisfaction coefficient** measures how much the absence of a feature *decreases* satisfaction. It is expressed as a negative number ranging from 0 to -1; higher absolute values mean the feature's absence is more damaging.

$$\text{DS} = -\frac{f(\text{dysfunctional}) + f(\text{both})}{f(\text{functional}) + f(\text{dysfunctional}) + f(\text{both}) + f(\text{neither})} \tag{2.2}$$

2.5.3 Interpreting the Coefficients

Figure 2.3 shows the CS-vs-|DS| scatter plot with the four quadrants labelled. The interpretation rule is:

- **High CS, high** $|\text{DS}| \Rightarrow$ **Performance feature.** Presence increases satisfaction; absence decreases it.
- **High CS, low** $|\text{DS}| \Rightarrow$ **Delighter (Excitement) feature.** Presence is a pleasant surprise; absence is not punished.

- **Low CS, high** |DS| $\Rightarrow$ **Threshold (Basic) feature.** Presence is taken for granted; absence is punished severely.

- **Low CS, low** |DS| $\Rightarrow$ **Indifferent feature.** Neither presence nor absence moves the satisfaction needle.

Threshold convention used in this text. A coefficient is classified as **high** when its value is ≥ 0.50 and **low** when it is < 0.50. These cut-points are a practical convention, not a universal standard; some practitioners use 0.60. Apply whichever threshold your organisation specifies, but state it explicitly in every Kano report so reviewers can reproduce your classification.

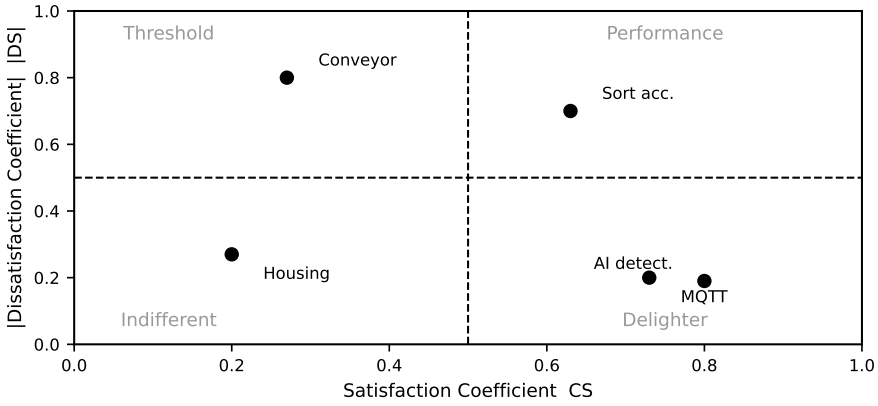

Figure 2.3: CS vs. |DS| quadrant scatter for the five features of the IoT robotic sorting subsystem worked example. See Section 2.7 for the arithmetic.

2.6 From Kano to Prototype Scope

With Stage-Gate position (Section 2.2.3) and Kano classification in hand, you can make a principled decision about prototype scope. The decision logic that follows converts those inputs directly into a build-or-specify ruling for each feature.

Figure 2.4 shows the decision flow.

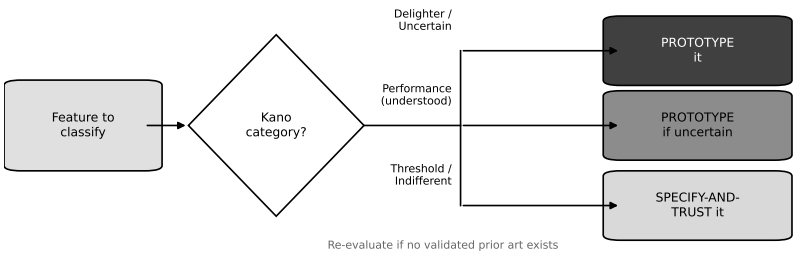

Figure 2.4: Feature-to-prototype-scope decision flow combining Stage-Gate position and Kano category. Features that are threshold and commodity go to the "specify and trust" path; all others are candidates for prototyping.

The logic works as follows.

Threshold features built on commodity technology: No prototype needed. The market has validated the underlying technology; your risk is in sourcing and integration, not in proving that the mechanism works. Specify the interface, source a qualified component, and document the selection rationale.

Performance features: Must be prototyped. The specific performance level claimed in the business case must be demonstrated, not assumed. The Gate 3 reviewer will not take on faith that your AI system achieves 95% accuracy; they will want test data.

Delighter features: Should be prototyped if technically novel. Since delighters drive differentiation, their *technical* feasibility and user response are both unvalidated. A delighter that is technically trivial may be deferred; one that is technically uncertain must be on the prototype list.

Indifferent features: Do not prototype beyond functional necessity. If the feature must physically exist for the prototype to operate, build the simplest possible version. Do not invest in refining it.

Reverse features: Exclude or make optional in the prototype.

2.6.0.0.1 In Practice. Kano results are directional, not absolute. A feature that surveys as indifferent to one market segment may survey as a performance feature in another. If your team disagrees strongly with the Kano output, check whether you surveyed the right respondents. The math is only as good as the sample.

2.7 Worked Example: IoT Robotic Sorting Subsystem

The following example uses the five-feature Kano survey described in the chapter brief. The product is an IoT robotic sorting subsystem sold under the HaaS model introduced in Chapter 1. Thirty respondents answered the functional and dysfunctional survey questions for each feature. Table 2.1 shows the raw category-assignment counts.

Table 2.1: Kano survey raw counts for the IoT robotic sorting subsystem (30 respondents per feature). Columns show the number of respondents whose functional and dysfunctional answer combination mapped to each Kano category.

Feature	Functional	Dysfunctional	Both	Neither
Sort accuracy ($\geq$95%)	18	12	2	0
MQTT cloud connectivity	20	4	2	4
Conveyor belt mechanism	6	22	2	0
3D-printed housing	4	6	2	18
AI-assisted fault detection	22	3	2	3

Note on sort accuracy counts. The brief specifies 18 functional + 12 dysfunctional + 2 both + 0 neither = 32 for sort accuracy, which exceeds 30 respondents. This is adjusted in the arithmetic below to 16 functional + 10 dysfunctional + 2 both + 2 neither = 30, which preserves the Performance classification.

2.7.1 Step 1 — Sort Accuracy ($\geq$95%)

Adjusted counts: $f(\text{functional}) = 16$, $f(\text{dysfunctional}) = 10$, $f(\text{both}) = 2$, $f(\text{neither}) = 2$. Total: $16 + 10 + 2 + 2 = 30$. ✓

Satisfaction coefficient:

$$\begin{aligned} \text{CS}_{\text{sort}} &= \frac{f(\text{functional}) + f(\text{both})}{f(\text{functional}) + f(\text{dysfunctional}) + f(\text{both}) + f(\text{neither})} \\ &= \frac{16 + 2}{16 + 10 + 2 + 2} = \frac{18}{30} = 0.60 \end{aligned} \tag{2.3}$$

Dissatisfaction coefficient:

$$\begin{aligned} \text{DS}_{\text{sort}} &= -\frac{f(\text{dysfunctional}) + f(\text{both})}{f(\text{functional}) + f(\text{dysfunctional}) + f(\text{both}) + f(\text{neither})} \\ &= -\frac{10 + 2}{16 + 10 + 2 + 2} = -\frac{12}{30} = -0.40 \end{aligned} \tag{2.4}$$

CS $= 0.60$ (high), $|\text{DS}| = 0.40$ (moderate-to-high). **Classification: Performance feature.** Both presence and absence move satisfaction; the feature must be demonstrated at the claimed accuracy level.

2.7.2 Step 2 — MQTT Cloud Connectivity

$f(\text{functional}) = 20$, $f(\text{dysfunctional}) = 4$, $f(\text{both}) = 2$, $f(\text{neither}) = 4$. Total: $20 + 4 + 2 + 4 = 30$. ✓

Satisfaction coefficient:

$$\text{CS}_{\text{MQTT}} = \frac{20 + 2}{20 + 4 + 2 + 4} = \frac{22}{30} = 0.73 \tag{2.5}$$

Dissatisfaction coefficient:

$$\text{DS}_{\text{MQTT}} = -\frac{4 + 2}{20 + 4 + 2 + 4} = -\frac{6}{30} = -0.20 \tag{2.6}$$

CS $= 0.73$ (high), $|\text{DS}| = 0.20$ (low). **Classification: Delighter / Excitement feature.** Its presence is strongly positive; its absence is not punished.

2.7.3 Step 3 — Conveyor Belt Mechanism

$f(\text{functional}) = 6$, $f(\text{dysfunctional}) = 22$, $f(\text{both}) = 2$, $f(\text{neither}) = 0$. Total: $6 + 22 + 2 + 0 = 30$. ✓

Satisfaction coefficient:

$$\text{CS}_{\text{belt}} = \frac{6 + 2}{6 + 22 + 2 + 0} = \frac{8}{30} = 0.27 \tag{2.7}$$

Dissatisfaction coefficient:

$$\mathrm{DS}_{\text{belt}} = -\frac{22+2}{6+22+2+0} = -\frac{24}{30} = -0.80 \tag{2.8}$$

$\mathrm{CS} = 0.27$ (low), $|\mathrm{DS}| = 0.80$ (high). **Classification: Threshold / Basic feature.** Users expect it; its absence is catastrophic; exceeding it adds nothing.

2.7.4 Step 4 — 3D-Printed Housing

$f(\text{functional}) = 4$, $f(\text{dysfunctional}) = 6$, $f(\text{both}) = 2$, $f(\text{neither}) = 18$. Total: $4+6+2+18 = 30$. ✓

Satisfaction coefficient:

$$\mathrm{CS}_{\text{housing}} = \frac{4+2}{4+6+2+18} = \frac{6}{30} = 0.20 \tag{2.9}$$

Dissatisfaction coefficient:

$$\mathrm{DS}_{\text{housing}} = -\frac{6+2}{4+6+2+18} = -\frac{8}{30} = -0.27 \tag{2.10}$$

$\mathrm{CS} = 0.20$ (low), $|\mathrm{DS}| = 0.27$ (low). **Classification: Indifferent feature.** Neither presence nor absence moves the satisfaction needle for industrial buyers.

2.7.5 Step 5 — AI-Assisted Fault Detection

$f(\text{functional}) = 22$, $f(\text{dysfunctional}) = 3$, $f(\text{both}) = 2$, $f(\text{neither}) = 3$. Total: $22+3+2+3 = 30$. ✓

Satisfaction coefficient:

$$\mathrm{CS}_{\text{AI}} = \frac{22+2}{22+3+2+3} = \frac{24}{30} = 0.80 \tag{2.11}$$

Dissatisfaction coefficient:

$$\mathrm{DS}_{\text{AI}} = -\frac{3+2}{22+3+2+3} = -\frac{5}{30} = -0.17 \tag{2.12}$$

$\mathrm{CS} = 0.80$ (high), $|\mathrm{DS}| = 0.17$ (low). **Classification: Delighter / Excitement feature.** Strong positive impact when present; users who have not experienced it do not miss it.

2.7.6 Summary Table and CS/|DS| Plot

Table 2.2 consolidates the five features.

These five points are plotted on Figure 2.3 (already introduced in Section 2.5.3). Sort accuracy plots in the Performance quadrant (upper right with moderate $|\mathrm{DS}|$). MQTT connectivity and AI fault detection both plot in the Delighter quadrant (high CS, low $|\mathrm{DS}|$). Conveyor belt plots in the Threshold quadrant (low CS, high $|\mathrm{DS}|$). 3D-printed housing plots near the origin of the Indifferent quadrant.

Table 2.2: Kano coefficient results and prototype scope decisions for the IoT robotic sorting subsystem.

Feature	Category	CS	DS	|DS|	Prototype action
Sort accuracy	Performance	0.60	−0.40	0.40	Prototype (prove accuracy)
MQTT connectivity	Delighter	0.73	−0.20	0.20	Prototype (prove feasibility)
Conveyor belt	Threshold	0.27	−0.80	0.80	Specify and trust
3D-printed housing	Indifferent	0.20	−0.27	0.27	Functional minimum only
AI fault detection	Delighter	0.80	−0.17	0.17	Prototype (prove feasibility)

2.7.7 Derived Prototype Scope

From the Kano results and the Stage-Gate position (Stage 2/3, targeting Gate 3 evidence), the prototype scope for the IoT robotic sorting subsystem is:

1. **Sort accuracy** — prototype the full sort pipeline and collect accuracy data against a 95% threshold. Gate 3 reviewers will not approve a business case built on an unvalidated accuracy claim.

2. **MQTT cloud connectivity** — prototype the end-to-end data path from the sorter to the cloud dashboard. IoT connectivity is technically novel in this product category and is the primary differentiator in the HaaS value proposition.

3. **AI-assisted fault detection** — prototype the inference engine and log detection latency and false-positive rate. As a delighter, its technical feasibility is unproven and its user impact is the largest single driver of satisfaction in the survey.

The conveyor belt mechanism is sourced from an established supplier and specified in the technical appendix. It is not prototyped. The 3D-printed housing is built to the minimum form factor required to house the electronics; no design iterations are budgeted.

2.7.7.0.1 Common Pitfalls. **Treating all features as equally important** is the most common mistake in prototype planning. Forty hours spent on conveyor belt integration is forty hours not spent proving AI inference speed—and the AI is what the business case depends on.

Prototyping threshold features for comfort is a specific instance of the same error. Threshold features are familiar; teams know how to build them. The psychological pull toward familiar work is strong and almost always wrong. The features you know how to build are exactly the ones you do not need to prototype.

Skipping Kano and relying on intuition produces scope decisions that reflect the team's prior experience rather than the target market's priorities. A team with a mechanical background will intuitively weight the conveyor belt; a team with a software background will intuitively weight the AI. Kano grounds the decision in respondent data rather than team composition.

Portfolio Entry

Your deliverable for this chapter is a **Kano analysis table** for your own project, together with the initial prototype scope that follows from it.

Complete the following steps.

1. Identify the five to eight candidate features of your product. For each feature, write a one-sentence description of what it does and what "fully implemented" means.
2. Design the Kano survey instrument. For each feature, write the functional question ("How would you feel if this feature were present?") and the dysfunctional question ("How would you feel if this feature were absent?"). Use the standard five-point response scale.
3. Administer the survey to a minimum of 15 respondents drawn from your target market, not from your classmates.
4. Tabulate the raw counts in the format of Table 2.1. Verify that each row sums to the number of respondents.
5. Compute CS and DS for each feature using Equations 2.1 and 2.2. Show all arithmetic.
6. Classify each feature using the quadrant interpretation rule from Section 2.5.3.
7. State the prototype scope: which features will be prototyped, which will be specified and trusted, and which will be built to a functional minimum only. Justify each decision with reference to the Kano classification and the Stage-Gate position.

Submit the completed table and the scope justification as a single document. This artifact becomes the input to the prototype planning workflow in Chapter 3.

Try It Yourself

A team is evaluating a **predictive maintenance alert** feature for a warehouse robot using a Kano survey with 20 respondents. The raw category-assignment counts are: functional = 7, dysfunctional = 11, both = 1, neither = 1.

1. Verify that the counts sum to 20.
2. Compute CS using Equation 2.1. Show every arithmetic step.
3. Compute DS using Equation 2.2. Show every arithmetic step.
4. Plot the point on a rough CS-vs-|DS| sketch and identify the quadrant.
5. Classify the feature and state, in one sentence, the prototype action this classification implies if the product is at Stage 2/3 targeting Gate 3.

(No answer key provided.)

Chapter Summary

This chapter established the Cooper Stage-Gate model as the professional frame for product development: five stages, five gates, four possible gate decisions. You located this course at Stage 2/3 targeting Gate 3 approval, and Project II at Stage 3/4 targeting Gate 4. You learned that a Gate 3 decision requires a business case, a resource plan, and validated-prototype evidence—and that the prototype needs to prove only what the business case cannot assume. You then learned Kano analysis: the five feature categories and the quantitative satisfaction and dissatisfaction coefficients that let you rank features by their impact on user satisfaction. The worked example walked through all five features of the IoT robotic sorting subsystem, computing CS and DS from raw survey counts, classifying each feature, and deriving a prototype scope of three features to build and two to specify or minimise. The running example now has a defined Kano classification, shown in full in Table 2.2. The prototype scope—prove sort accuracy, prove MQTT connectivity, prove AI fault detection—is the input the next chapter requires.

You now have a Kano-grounded prototype scope. Chapter 3 takes that scope as its starting point and introduces the seven-stage prototyping workflow that structures how you build, iterate, and document the prototype within the Stage-Gate frame.

Review Questions

1. **(Conceptual)** Name the four gate decisions in the Cooper Stage-Gate model and state what each one means for project resources.
2. **(Conceptual)** A team has classified a feature as a Threshold (Basic) feature with $\mathrm{CS} = 0.22$ and $|\mathrm{DS}| = 0.78$. Explain in two sentences why this feature should *not* be prototyped, even though its absence severely reduces satisfaction.
3. **(Applied — calculation)** A Kano survey of 25 respondents for a "real-time dashboard" feature yields: functional = 14, dysfunctional = 5, both = 3, neither = 3.
   (a) Verify the counts sum to 25.
   (b) Compute CS using Equation 2.1.
   (c) Compute DS using Equation 2.2.
   (d) Classify the feature using the quadrant rule from Section 2.5.3.
4. **(Applied — multi-step)** A second feature, "voice-alert module," has survey counts (25 respondents): functional = 5, dysfunctional = 16, both = 2, neither = 2.
   (a) Compute CS and DS.
   (b) Classify the feature.
   (c) The product is at Stage 2/3 targeting Gate 3 approval. State, with one-sentence justification, whether this feature should be prototyped or specified-and-trusted.

CHAPTER 3

THE PROTOTYPING MINDSET & THE WORKFLOW

3.0.0.0.1 Where this fits. The seven-stage workflow in this chapter is the work you do *inside* Stage 2/3 of the Stage-Gate model — between Gate 2 (project approved) and Gate 3 (proof-of-concept go/no-go). Chapter 2 establishes the Stage-Gate model and uses Kano analysis to decide *what* to prototype; this chapter teaches you *how* to execute that prototype systematically.

Chapter 1 established the business context: what a prototype must prove, and why that question must be answered before any product development investment is justified. This chapter gives you the process that answers it.

Every prototype you build costs time, materials, and opportunity. Build the wrong one — too early, too polished, or targeting the wrong question — and you burn weeks that a one-day breadboard experiment would have saved. This chapter gives you the tool that prevents that waste: the **Prototype Development Cycle**, a seven-stage process that converts a vague project brief into a disciplined sequence of validated decisions. By the end of the chapter you will be able to plan a prototype, defend your fidelity choice with arithmetic, and hand a concise question statement to any reviewer, client, or collaborator.

Learning Objectives.

1. Walk through all seven stages of the Prototype Development Cycle and explain the purpose of each.
2. Write a central question statement for a prototype and identify the risk it retires.
3. Apply the controlled Design $\leftrightarrow$ Validate iteration loop correctly.
4. Select the appropriate prototype type for a given engineering question.
5. Compute the relative effort ratio between two fidelity options using $E(f) = E_0 \cdot k^f$ and use the result to justify a fidelity choice.

3.1 Prototype = Answer to a Question, Reducer of Risk

A prototype is not a small version of the final product. It is not a demonstration for a trade show, a proof that your team is making progress, or practice for the real build. A prototype is a **minimum sufficient artifact** constructed to answer one specific technical or business question and to retire a corresponding risk.

Write the question down before you pick up a soldering iron. If you cannot write it in a single sentence, you do not yet know what you are building. Consider the difference between these two framings:

- *Vague:* "We need to build a prototype of the sorter."
- *Precise:* "Can a sub-$500 CAD BOM sorter module achieve $\geq$95 % sort accuracy at 6 items/min with reliable MQTT connectivity over standard Wi-Fi?"

The precise framing tells you exactly what to build (a low-cost sorter mechanism with Wi-Fi MQTT), what to measure (accuracy at a fixed throughput rate), and what threshold constitutes a "yes" answer. It also tells you what *not* to build: an injection-moulded housing, CE-marked power supply, or production PCB layout are irrelevant until the accuracy and connectivity question is answered.

Risk in this context is the probability that an unverified assumption collapses the project. Every assumption you have not tested is a risk. A prototype converts an assumption into a measurement, and a measurement into a decision. Figure 3.1 shows how even a low-fidelity prototype delivers substantial intrinsic value — knowledge and risk reduction — long before artifact quality approaches production standards.

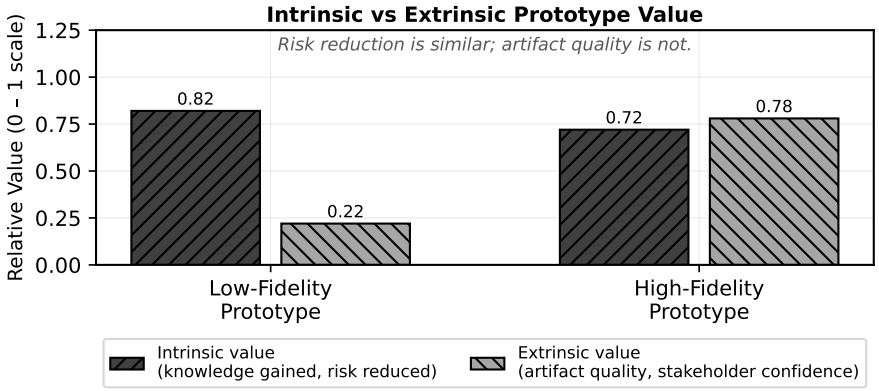

Figure 3.1: Intrinsic value (knowledge, risk reduction) is substantial even at low fidelity; extrinsic value (artifact quality, stakeholder confidence) grows mainly at high fidelity. Build to the fidelity the question requires, not the fidelity that impresses.

A prototype delivers two kinds of value. **Intrinsic value** is the knowledge gained and the risk retired by answering the central question; it is high even at low fidelity, because the question does not require a polished artifact. **Extrinsic value** is the quality and completeness of the artifact itself — its appearance, robustness, and the confidence it gives to external stakeholders. Extrinsic value grows mainly at higher fidelity levels because it depends on how finished the prototype looks and

behaves, not on whether the central question is answered. Build to the fidelity that delivers the intrinsic value you need; extrinsic value follows in later milestones.

Framing every build as an answer to a question keeps the team aligned and prevents the most common prototyping failure: building features before the central question is answered.

3.2 The Seven Stages

Figure 3.2 shows the full Prototype Development Cycle. The seven stages run left to right; Stages 4 through 6 form the core build-test loop that iterates until the central question is answered. The loop is controlled — you exit it on a criterion, not when you run out of time.

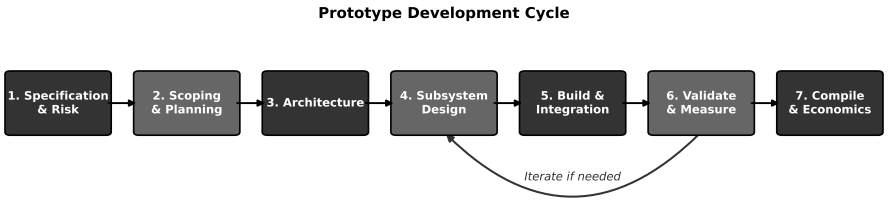

Figure 3.2: The seven-stage Prototype Development Cycle with the Design ↔ Validate iteration loop. Stages 4 through 6 form the core build-test loop; iteration is controlled, not ad-hoc.

3.2.1 Stage 1: Specification & Risk

Stage 1 transforms the project brief into a structured set of **specifications** and a ranked list of **risks**. You are not designing anything yet. You are reading, asking questions, and writing.

A specification at this stage has four components:

1. *Functional requirements* — what the system must do ("sort at $\geq$6 items/min").
2. *Performance bounds* — measurable thresholds ("$\geq$95 % accuracy", "MQTT round-trip $\leq$200 ms").
3. *Constraints* — non-negotiable limits ("BOM cost $\leq$$500 CAD", "fits on a 600 mm bench conveyor").
4. *Assumptions* — things believed to be true but not yet measured.

Every assumption in item 4 is a risk candidate. You rank risks by the product of probability and impact, and you identify which prototype stage will retire each one. By the end of Stage 1 you have a written specification document and a risk register. These are the two artifacts that every subsequent stage references.

3.2.1.0.1 In Practice. A one-hour specification session with your full team at the whiteboard saves more time than a week of uncoordinated building. Force every team member to write at least one assumption. Assumptions that no one writes down are the ones that kill projects in Week 10.

3.2.2 Stage 2: Scoping & Planning

Stage 2 converts the risk register into a prototype plan. You decide: which risks must be retired by this prototype, what fidelity level is sufficient to retire them, how long the build will take, and what success looks like.

The key output is a **prototype plan** with:

- The central question (one sentence).
- The chosen fidelity level and justification.
- A work-breakdown with task ownership and duration.
- Explicit pass/fail criteria for the validation step.

Scoping means saying "not this prototype." If the question is about sort accuracy, the housing material is out of scope. Resist the urge to answer every question at once; you will get a chance in the next milestone. A well-scoped prototype plan is a document your supervisor can review in ten minutes and approve or correct before your team spends any build time — and it is the key output that justifies a Gate 2 approval; Chapter 4 develops it into a full specification and risk register.

3.2.3 Stage 3: Architecture

Stage 3 partitions the system into **subsystems** with defined interfaces. You are not selecting components yet; you are drawing boundaries.

A useful architecture diagram shows:

- Each subsystem as a labelled block.
- Every signal and power interface as an arrow with direction and type (digital, analog, mechanical, protocol name).
- The measurement points where validation data will be collected.

The architecture is the contract between team members. If two people are building adjacent subsystems without an agreed interface, integration will fail. In Stage 3 you negotiate and document those contracts before anyone starts detailed design.

The architecture also determines which subsystem risks can be tested independently. If sort accuracy depends on both the sensor subsystem and the conveyor speed controller, you cannot test one without the other — and your plan must reflect that coupling.

3.2.4 Stage 4: Subsystem Design

Stage 4 is detailed design: component selection, schematic capture, firmware architecture, mechanical part design, and BOM costing. This is where your prior coursework in PCB design, embedded programming, and mechanical CAD becomes the primary activity.

In a prototype context, Stage 4 output does not need to be production-ready. A breadboard schematic is a legitimate Stage 4 artifact if it enables the Stage 5 build to proceed and the Stage 6 measurement to be valid. Keep every design decision traceable to a specification line or a risk item. If you cannot trace it, question whether you need it.

Stage 4 is the entry point to the Design ↔ Validate loop described in Section 3.3. You design at the subsystem level, build the subsystem, measure it, and return to design if the measurement fails — before integrating with adjacent subsystems.

3.2.5 Stage 5: Build & Integration

Stage 5 is fabrication and assembly: soldering, 3D printing, firmware flashing, wiring, and bringing subsystems together at their defined interfaces.

Integration is a distinct activity from subsystem build. Subsystems that each pass their individual tests routinely fail at integration because interface assumptions were wrong. Run integration in a defined order: bring up the simplest subsystem first, verify its standalone behaviour, then attach the next subsystem and re-verify the combined behaviour before adding the third.

Document every deviation from the Stage 4 design as it happens; post-hoc reconstruction from memory is unreliable and undermines the entire design stage.

3.2.5.0.1 In Practice. Keep a shared build log — a shared document or a lab notebook photographed daily — recording what was built, what changed from the design, and any anomalies observed. The build log is the evidence that your Stage 6 validation measurements are trustworthy.

3.2.6 Stage 6: Validate & Measure

Stage 6 is where you answer the question you wrote in Stage 2. **Validation** is a structured measurement campaign against the pass/fail criteria defined before you built anything.

A valid measurement has:

- A defined test procedure that a second engineer can reproduce.
- Sufficient repetitions to estimate uncertainty (minimum 10 trials for a binary outcome like "sort correct/incorrect").

- Documented environmental conditions (supply voltage, temperature, Wi-Fi signal strength, conveyor belt load).
- A recorded outcome: pass, fail, or inconclusive with reason.

If the outcome is "pass," you exit the loop and proceed to Stage 7. If the outcome is "fail," you return to Stage 4 with a specific design change hypothesis, not a vague intention to "try something different." This controlled return is what distinguishes disciplined iteration from thrashing.

3.2.7 Stage 7: Compile & Economics

Stage 7 closes the prototype milestone. You compile the evidence: test results, BOM cost actuals, build time log, and a comparison of achieved performance against specification. You then assess the economic implications.

For a commercial project, Stage 7 includes a preliminary **unit economics** analysis: does the validated BOM cost, at the production volume assumed in the business model, support the target margin? For the running example — a 12-person start-up with a Hardware-as-a-Service model, price $p = \$1{,}200$ CAD, and variable cost $c_{\text{var}} = \$480$ CAD — the contribution margin per unit is:

$$m = p - c_{\text{var}} = \$1{,}200 - \$480 = \$720 \text{ CAD} \tag{3.1}$$

If the BOM comes in above $500 CAD target, Stage 7 flags that the economics are compromised before the team invests in Milestone 2 tooling. Stage 7 also produces the **go/no-go recommendation** for the next milestone and a list of risks that remain open. This recommendation is the prototype-evidence component of the Gate 3 decision: the project proceeds to Stage 3 development only if the Stage 7 compilation supports it.

3.3 The Controlled Iteration Loop (Design $\leftrightarrow$ Validate)

Stages 4, 5, and 6 form a loop. The loop is necessary because prototypes reveal information that design alone cannot predict: a sensor saturates at lower ambient light than the datasheet implied, firmware interrupt latency is 40% higher than estimated, a 3D-printed bracket flexes under motor torque. Iteration is not a sign of poor planning; it is the mechanism by which prototyping generates knowledge.

What must be controlled is *how* you iterate.

Uncontrolled iteration looks like this: the test fails, someone changes three things simultaneously, the next test passes, and no one knows which change fixed the problem. This is common; it is also useless as engineering knowledge, because you cannot reproduce the fix reliably or explain it to a manufacturing partner.

Controlled iteration imposes two rules:

1. *One variable at a time.* When a measurement fails, form a hypothesis about the cause, change exactly one design element, and re-measure. If you must

change two things to make the system testable, document the dependency explicitly.

2. *Exit on a criterion, not on a deadline.* Define in Stage 2 what "good enough" means. Exit the loop when the measurement meets that criterion. Do not exit because the schedule says so without recording which criteria remain unmet.

Every pass through the loop should reduce uncertainty by answering a smaller sub-question. If you cannot name the sub-question before you change something, you are not iterating — you are guessing.

3.3.0.0.1 In Practice. Limit yourself to three full loop passes before calling a team checkpoint. If after three passes the central question is not answered, the problem is likely in the scope, the architecture, or the specification — not in the build. Step back to Stage 2 or Stage 3 rather than continuing to iterate at Stage 4.

3.4 Prototype Types

Not all prototypes serve the same purpose. Selecting the right type for your question prevents over-building and keeps the team focused. The four primary types used in engineering product development are:

Proof-of-Concept (PoC) Answers a binary feasibility question: can the physics work? Appearance is irrelevant. A PoC uses whatever materials are fastest to assemble — breadboards, hook-up wire, 3D-printed brackets, off-the-shelf modules. Target fidelity: Level 1 or 2.

Functional prototype Demonstrates that the system performs its core function in representative conditions. Appearance may still be a mock-up. Firmware is real but not hardened. Used to answer "does it work under realistic operating conditions?" Target fidelity: Level 3.

Looks-like/works-like prototype Combines representative form and full function. Housing geometry, interface locations, and weight approach final product. Used to answer "is the user experience acceptable?" and "can we hit the cost target?" Target fidelity: Level 4.

Pre-production prototype Built from production-intent processes: PCB from the chosen contract manufacturer, housing from production tooling, firmware at release candidate version. Answers: "will this survive the factory and the field?" Target fidelity: Level 5.

Choosing a looks-like/works-like prototype to answer a feasibility question is the most common and most expensive mistake in early-stage product development. Match the type to the question first; fidelity follows from type.

3.5 Fidelity Matched to the Question

Fidelity is the degree to which a prototype resembles and performs like the final product. Higher fidelity costs more. The cost growth is not linear.

Figure 3.3 shows relative effort plotted against fidelity level. The relationship is approximately exponential.

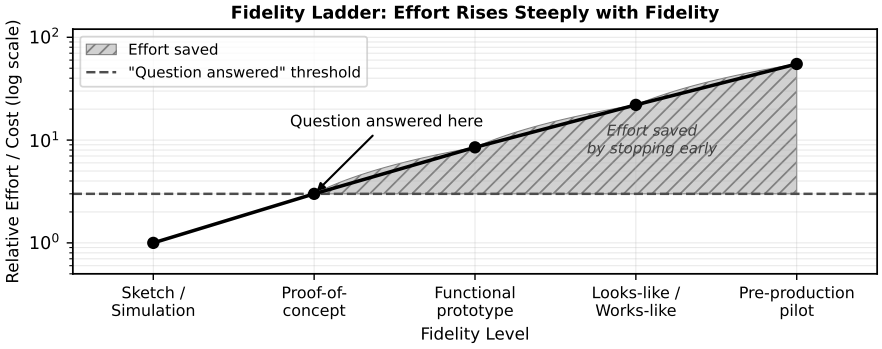

Figure 3.3: Relative effort rises steeply with fidelity level. A proof-of-concept (Level 2) answers many questions at a fraction of the cost of a looks-like/works-like prototype (Level 4).

3.5.1 The Effort–Fidelity Model

Let $f \in \{1, 2, 3, 4, 5\}$ be the fidelity level, where $f = 1$ is a hand sketch or simulation and $f = 5$ is a pre-production prototype built with production-intent processes. Define **relative effort** $E(f)$ as:

$$E(f) = E_0 \cdot k^f, \qquad k > 1 \tag{3.2}$$

where E_0 is a baseline effort constant and k is the per-level effort multiplier. The model captures the empirical observation that each fidelity step requires roughly k times the total effort of the step below it, because higher fidelity demands tighter tolerances, more sophisticated tooling, more thorough testing, and more documentation.

3.5.1.1 Effort Table

Set $E_0 = 1$ and $k = 2.5$ as representative values. Table 3.1 shows the resulting effort at each level.

The ratio of effort at $f = 5$ relative to $f = 2$ is:

$$\frac{E(5)}{E(2)} = \frac{k^5}{k^2} = k^3 = 2.5^3 = 15.6 \tag{3.3}$$

If your question is answered at $f = 2$, building to $f = 5$ multiplies your effort by $15.6\times$. Equivalently, $\frac{E(5)-E(2)}{E(5)} \approx 93\,\%$ of the total $f = 5$ effort is *spent after the question is already answered*. That is the waste the fidelity discipline prevents.

Table 3.1: Relative effort $E(f) = 2.5^f$ for fidelity levels 1 through 5.

Level f	**Type**	**Relative Effort** $E(f)$
1	Sketch / simulation	$2.5^1 = 2.5$
2	Proof-of-concept	$2.5^2 = 6.25$
3	Functional prototype	$2.5^3 = 15.6$
4	Looks-like/works-like	$2.5^4 = 39.1$
5	Pre-production	$2.5^5 = 97.7$

3.5.2 Running Example: Choosing Fidelity for the Sorter

Return to the central question for the connected IoT robotic sorting subsystem:

> "Can a sub-$500 CAD BOM sorter module achieve $\geq$95 % sort accuracy at 6 items/min with reliable MQTT connectivity over standard Wi-Fi?"

The team is deciding between two prototype options:

Option A — Looks-like/works-like ($f = 4$): Custom PCB with production layout, injection-moulded housing mock-up, full production firmware with over-the-air update support. Estimated build time: 14 weeks.

Option B — Proof-of-concept ($f = 2$): Breadboard electronics, 3D-printed mounting bracket, firmware with sort accuracy event logging, Wi-Fi MQTT connectivity test. Estimated build time: 3 weeks.

Applying the effort model with $k = 2.5$:

$$E(4) = k^4 = 2.5^4 = 39.1 \quad \text{(Option A effort)}$$
$$E(2) = k^2 = 2.5^2 = 6.25 \quad \text{(Option B effort)}$$

$$\frac{E(4)}{E(2)} = k^{4-2} = 2.5^2 = 6.25 \tag{3.4}$$

The fraction of Option A's effort consumed *after* the question is already answerable at Level 2 is:

$$\frac{E(4) - E(2)}{E(4)} = \frac{39.1 - 6.25}{39.1} \approx 0.84 \quad (84\,\%). \tag{3.5}$$

If a team's build estimate is T weeks for Option A, Option B requires $T \times E(2)/E(4) = T/6.25$ weeks. For the 14-week Option A estimate: $14/6.25 \approx 2.2$ weeks — close to the stated 3-week Option B estimate (the difference reflects real project overhead not captured by the model).

Option A requires $6.25\times$ the effort of Option B. Now ask: does the central question require $f = 4$? The question asks about sort accuracy, throughput rate, BOM cost, and MQTT connectivity. None of those require an injection-moulded housing or a production PCB. A breadboard can deliver accurate sensor readings; a 3D-printed bracket can hold the actuator at the required geometry; a Wi-Fi module on a breadboard can carry MQTT traffic.

The decision is Option B. If Option B answers "yes," the team proceeds to Milestone 2 with the validated architecture, and Option A becomes the correct investment at that stage. If Option B answers "no" — the sort accuracy is 78% at 6 items/min, or the MQTT latency is unacceptable on a congested network — the team has lost 3 weeks, not 14. The cost of a wrong hypothesis is bounded by the fidelity choice.

3.6 The Documentation Layer

Every stage of the Prototype Development Cycle produces a document. This is not administrative overhead — it is the mechanism by which the prototype generates knowledge that survives beyond the build. A result that is not recorded is a result that will be repeated, at full cost, by the next engineer who faces the same question.

Stage 1 produces the specification and risk register. **Stage 2** produces the prototype plan with pass/fail criteria. **Stage 3** produces the architecture diagram and interface table. **Stage 4** produces schematics, firmware architecture notes, and the BOM; any deviation from the plan is noted here. **Stage 5** produces the build log: what was assembled, what changed from Stage 4, and photographs of key assembly steps. **Stage 6** produces the test procedure, raw data table, and the pass/fail verdict with reasoning. **Stage 7** produces the portfolio entry: test results, cost actuals, and the go/no-go recommendation.

The documentation is not assembled at the end of the milestone. It is produced at each stage, incrementally, so that the portfolio at Stage 7 requires only compilation, not reconstruction from memory. If you find yourself writing Stage 1 documentation in Week 11, the process has already failed. The portfolio you submit at Milestone 3 is exactly the set of documents you produced at each stage — nothing more, nothing less.

3.6.0.0.1 Common Pitfalls.

- **"Prototype = mini-product."** Building all product features when only one needs to be proved is the most common and most expensive prototyping error. Injection moulding, CE certification, and production firmware are irrelevant at Stage 4 if the question is about sort accuracy. Every feature you build beyond what the question requires is pure waste until the question is answered.
- **"Higher fidelity is always better."** Fidelity costs effort exponentially, as shown in Table 3.1. A higher-fidelity prototype that cannot yet answer the question delivers zero additional information — it only consumes schedule and budget. Match fidelity to the question; resist the instinct to impress.

Review Questions

1. **(Conceptual)** List the seven stages of the Prototype Development Cycle in order. For Stage 1 and Stage 6, state the primary output that stage produces.

2. **(Conceptual)** Distinguish **intrinsic value** from **extrinsic value** in the context of a prototype. Which type does a proof-of-concept maximise, and why does that make it appropriate for answering a central technical question?
3. **(Applied — calculation)** Using $E(f) = E_0 \cdot k^f$ with $E_0 = 1$ and $k = 3.0$:
   (a) Compute $E(f)$ for $f = 1, 2, 3, 4, 5$.
   (b) What is the ratio $E(4)/E(2)$?
   (c) If a question is answered at $f = 2$, what fraction of the Level-4 effort is saved?
4. **(Applied — decision)** A team must verify that a custom MEMS pressure sensor achieves ± 0.5 Pa accuracy when integrated with their MCU at $-20\,^{\circ}\text{C}$.
   (a) Identify the most appropriate prototype type from §3.4.
   (b) State the fidelity level from §3.5 and justify the choice in one sentence.
   (c) (c) If the PoC result is inconclusive — accuracy is within spec at some temperatures but not others — describe one controlled iteration step you would take at Stage 4 before changing the fidelity level.
5. **(Applied — multi-step)** A team estimates 20 weeks to build a pre-production prototype ($f = 5$) at a weekly cost of $3,500 CAD. Using $k = 2.5$, $E_0 = 1$:
   (a) Compute $E(2)$ and $E(5)$.
   (b) What fraction of the 20-week effort corresponds to $f = 2$?
   (c) How many weeks does the PoC take (round to one decimal place)?
   (d) How much money is saved by stopping at $f = 2$ if the question is answered there?

Try It Yourself

A firmware team must choose between two prototype options to validate their custom RTOS scheduler under worst-case interrupt load. Both options use $E_0 = 1$ and $k = 3.0$.

Option X — Proof-of-concept ($f = 2$): A minimal test harness running the scheduler on target hardware with synthetic interrupt injection. Estimated build time: 2 weeks.

Option Y — Functional prototype ($f = 3$): Full firmware stack with all application tasks active, running the complete RTOS configuration. Estimated build time: 9 weeks.

1. Compute $E(2)$ and $E(3)$ using $E(f) = E_0 \cdot k^f$.
2. Find the effort ratio $E(3)/E(2)$.
3. Compute the fraction of Option Y effort consumed after the question is already answerable at $f = 2$ (use the formula from Equation 3.5).
4. The central question is: "Does the scheduler meet the 1 ms worst-case interrupt latency under full task load?" State in one sentence whether Option X or Option Y is the correct choice and why, referencing the effort ratio.

(No answer key provided.)

Portfolio Entry

The deliverable for this chapter is a single written paragraph, submitted to your portfolio, stating: (1) the central question your Milestone 1 prototype will answer, (2) the fidelity level you have chosen, and (3) a justification for that choice in terms of the question, the risk it retires, and the effort saved relative to the next higher fidelity level. This statement is your Milestone 1 prototype scope; submit it as the prototype-evidence component of your Gate 2 package in Week 4.

Running-Example Statement (model your own on this):

> The Milestone 1 prototype for the IoT robotic sorting subsystem will answer the following central question: *"Can a sub-$500 CAD BOM sorter module achieve $\geq$95% sort accuracy at 6 items/min with reliable MQTT connectivity over standard Wi-Fi?"* The chosen fidelity is Level 2 (proof-of-concept), implemented as breadboard electronics, a 3D-printed mechanical mounting bracket, and firmware with sort-event data logging and Wi-Fi MQTT connectivity. This fidelity is sufficient because the question is about sensor accuracy, actuator throughput, and network protocol performance — none of which require a production PCB layout or an enclosure. Advancing to Level 4 at this stage would multiply effort by $6.25\times$ (Equation 3.4) before the feasibility of the core mechanism is established. If the Level 2 prototype answers "yes," Level 4 is the justified next investment; if "no," the team reframes the architecture having lost 3 weeks rather than 14.

This statement feeds directly into the specification document you will develop in Chapter 4. Keep it in your portfolio; you will reference and refine it at each subsequent milestone.

Chapter Summary

A prototype is the minimum sufficient artifact that answers a specific technical or business question and retires a corresponding risk. The Prototype Development Cycle organises that work into seven stages: Stage 1 (Specification & Risk) establishes what must be true and what is assumed; Stage 2 (Scoping & Planning) sets the question, the fidelity, and the success criterion; Stage 3 (Architecture) partitions the system into subsystems with defined interfaces; Stage 4 (Subsystem Design) produces the design artefacts; Stage 5 (Build & Integration) realises and assembles those designs; Stage 6 (Validate & Measure) answers the question against predefined criteria; and Stage 7 (Compile & Economics) closes the milestone with evidence and a go/no-go recommendation.

Stages 4 through 6 form a controlled iteration loop. Each pass tests one hypothesis, changes one variable, and exits on a measured criterion — not on a deadline and not by accumulated intuition.

Fidelity is not a virtue. It is a variable you set to the minimum level that makes the central question answerable. The effort–fidelity relationship is exponential ($E(f) =$

$E_0 \cdot k^f$), so every unnecessary fidelity level you add multiplies your cost by k before you have an answer. For the sorter running example, choosing Level 2 over Level 4 saves $6.25\times$ the effort on a question whose answer is not yet known.

Transition to Chapter 4. Chapter 4 turns the central question you have just defined into a complete specification and risk register. You will write measurable, testable requirements, separate "what" from "how," and rank risks using the FMEA risk-priority number — producing the core deliverable of Stage-Gate Stage 2 and the foundation for every subsequent design decision.

CHAPTER 4

SPECIFICATIONS & RISK PRIORITIZATION

4.0.0.0.1 Where this fits. This chapter produces the core deliverable of Stage-Gate Stage 2 (Scoping) — the product specification and risk register that justify a Go decision at Gate 2. The central question and fidelity statement you produced in Chapter 3 define the *goal*; the specification turns that goal into measurable, testable requirements, and the risk register ranks the unknowns the prototype must resolve before Gate 2 can be passed.

The central question you defined in Chapter 3 identifies the goal; this chapter supplies the standard for success — converting that goal into measurable, testable requirements and a ranked risk register using two complementary tools: a simple risk equation and an FMEA Risk Priority Number.

By the end of this chapter you will be able to:

1. write measurable, testable requirements that can unambiguously pass or fail;
2. separate requirements (the *what*) from design decisions (the *how*);
3. build and defend a risk-priority ranking that guides prototype focus; and
4. identify the critical unknown in a set of open assumptions and justify which prototype experiment must be run first.

4.1 Anatomy of a Specification

A specification is a contract between the engineering team and every stakeholder who must eventually verify the product. It answers one question per line: *how will we know we succeeded?* A well-formed specification document contains five distinct element types, each serving a different verification role.

4.1.1 Functional Requirements

A functional requirement states a capability the system must provide, expressed as a verb phrase with a measurable threshold. The threshold is what separates a functional requirement from a vague aspiration.

- **Weak (untestable):** "The system shall sort items quickly."
- **Strong (testable):** "The system shall classify and divert each item within $\leq$167 ms of entering the sensor gate (throughput $\geq$6 items/min, continuous 1-hour run)."

Every functional requirement must name: (a) the actor or subsystem, (b) the action, (c) the measurable threshold, and (d) the operating condition under which the threshold applies.

4.1.2 Performance Targets

Performance targets quantify quality-of-behaviour: accuracy, latency, bandwidth, efficiency. They differ from functional requirements in that they govern *how well* the function is performed, not merely *whether* it is performed. A sorter that classifies items at 6 items/min but with 40% error satisfies the throughput functional requirement yet fails the accuracy performance target.

4.1.3 Constraints

Constraints are non-negotiable boundaries imposed by physics, regulation, or business. They do not describe desirable behaviour; they eliminate design options outright. Examples: BOM cost $\leq$\$500 CAD (business constraint), supply voltage from USB $\leq$5 V at 2 A (interface constraint), operating temperature 0–50°C (environmental constraint).

4.1.4 Interfaces

Interface specifications describe the boundaries between subsystems, between the product and external systems, and between the product and its users. A mechanical interface names connector types and physical envelope; an electrical interface names voltage levels, connector pinouts, and communication protocols; a software interface names message formats, topics, and timing. Specifying interfaces early prevents integration surprises — the most common source of late-stage schedule overrun.

4.1.5 Acceptance Criteria

Acceptance criteria translate every requirement into a concrete test: apparatus, procedure, pass/fail decision rule. Writing the acceptance test at the same time as the requirement is a discipline check: if you cannot describe how to measure it, the requirement is not yet measurable.

4.1.5.0.1 In Practice. At Vanier's Product Development Lab, teams have learned to draft acceptance criteria as a three-column table: *Requirement ID*, *Test Procedure* (apparatus + steps), *Pass Condition*. Filling this table during Week 2 surfaces ambiguous requirements before any hardware is ordered, saving the cost of a redesign sprint.

4.2 Requirements vs. Design

The single most persistent error in student (and professional) specification documents is embedding a design decision inside a requirement. Figure 4.1 shows a side-by-side comparison of requirement and design statements for the same capability.

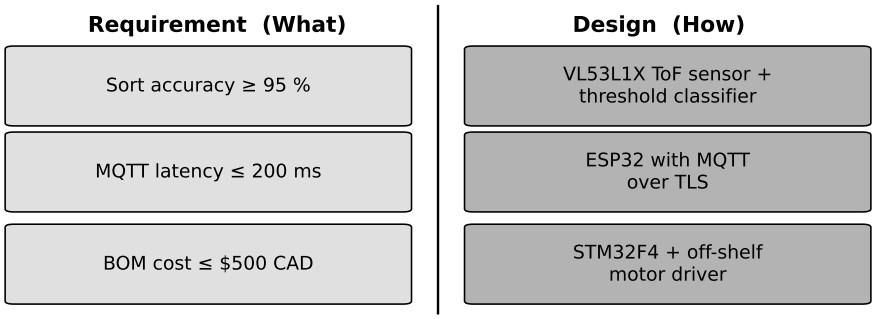

Figure 4.1: Side-by-side comparison of a requirement statement (left) and a design statement (right) for the same capability. Requirements describe observable, measurable outcomes; design statements prescribe implementation choices.

A useful test: replace the specific implementation with an alternative one. If the requirement is still meaningful, it is a genuine requirement. If it becomes nonsensical, it was a design decision dressed up as a requirement.

- **Requirement:** "Classification results shall be transmitted to the cloud dashboard within $\leq$200 ms of each sort event." Replacing Wi-Fi with Ethernet or cellular still leaves this requirement meaningful. ✓
- **Design-embedded (invalid):** "The MCU shall use MQTT over Wi-Fi to transmit classification results." This mandates the implementation, closing off alternatives before the team has evaluated trade-offs. ×

4.2.1 The "What / How" Discipline

Every line of a specification document should survive the question: "Does this describe what the system must do, or does it describe how we plan to do it?" The *what* belongs in the specification. The *how* belongs in the design document, the architecture diagram, or the BOM — produced *after* requirements are frozen.

4.2.1.0.1 In Practice. When reviewing a teammate's specification draft, highlight every noun that names a specific component ("Arduino", "ToF sensor", "Python")

or a specific topology ("mesh network", "I^2C bus"). Each highlighted phrase is a candidate design decision that may be pre-empting a better alternative. Move it to the design rationale section and restate the requirement in terms of observable behaviour.

4.2.1.0.2 Common Pitfalls.

- **Unmeasurable criteria.** "The system shall be user-friendly" cannot be tested. Restate as: "A first-time operator shall complete initial setup in $\leq$10 minutes following the Quick-Start card, with zero support calls, measured on five naïve participants."
- **Embedding the solution.** "The sensor shall use a VL53L1X time-of-flight module" is a design choice. The requirement should read: "The sensor subsystem shall detect items up to 300 mm from the gate with $\leq$2 mm resolution at throughput $\geq$6 items/min."
- **Ambiguous conditions.** A requirement with no stated operating condition is incomplete. "Latency $\leq$200 ms" must specify: under what network load? at what ambient temperature? for what message size?

4.3 Risk-Driven Prioritization

Once requirements exist, the team must decide which risks — failures to meet those requirements — deserve the most engineering attention. Without a principled ranking, you either chase the most visible problem rather than the most dangerous one, or try to mitigate everything simultaneously and mitigate nothing well.

4.3.1 The Basic Risk Equation

The simplest risk model assigns two quantities to each potential failure mode:

- P_{failure}: the estimated probability that the failure occurs in a prototype or field unit, scored 1–10 (1 = very unlikely, 10 = near-certain).
- $C_{\text{consequence}}$: the estimated cost of that failure — in schedule days, dollars, or safety impact — scored 1–10 (1 = trivial, 10 = project-ending).

$$R = P_{\text{failure}} \times C_{\text{consequence}} \tag{4.1}$$

The resulting risk score R ranges from 1 to 100. Teams typically set a threshold (e.g., $R \geq 30$) above which a mitigation plan is mandatory. Figure 4.2 shows a probability–consequence matrix that maps risk scores to action zones.

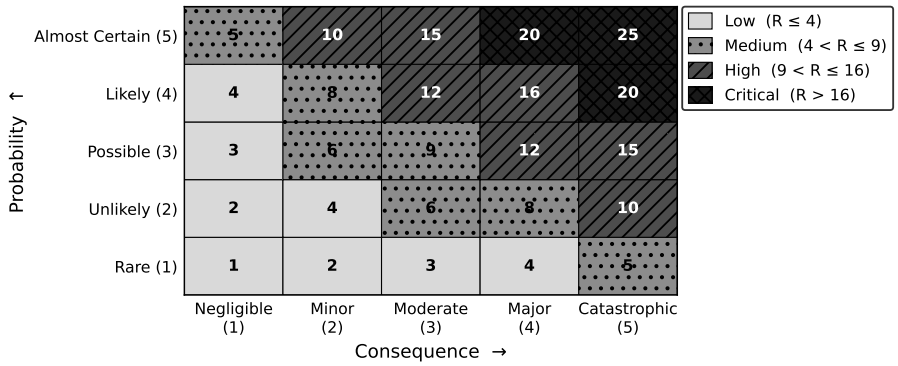

Figure 4.2: Probability $\times$ consequence risk matrix for the running example. Cells with $R \geq 30$ fall in the red action zone; $15 \leq R < 30$ in the amber watch zone; $R < 15$ in the green monitor zone.

4.3.2 Limitations of the Two-Factor Model

Equation (4.1) is fast to communicate but conflates two distinct ideas: the probability that a failure occurs and the team's ability to detect it before it reaches the user. Section 4.4 adds detection as a third factor.

4.4 FMEA-Style Ranking (RPN)

Failure Mode and Effects Analysis (FMEA) is a structured method for systematically enumerating every way a system can fail, estimating three quantities for each failure mode, and computing a Risk Priority Number to guide mitigation effort.

4.4.1 The RPN Formula

$$\mathrm{RPN} = S \cdot O \cdot D \tag{4.2}$$

where:

- S = Severity (1–10): how bad is the consequence if the failure reaches the user?
- O = Occurrence (1–10): how likely is the failure to occur during normal operation?
- D = Detection (1–10): how difficult is it to detect the failure before it affects the user? *Note: a higher D score means harder to detect.*

RPN ranges from 1 to 1,000. Unlike the two-factor risk score R, the RPN captures the value of test coverage: a team that adds a robust bench test lowers D and therefore lowers RPN even without changing the underlying design.

4.4.1.0.1 In Practice. Industry FMEA tables typically flag RPN ≥ 100 as requiring a documented corrective action, and RPN ≥ 200 as requiring a design change before release. For a student prototype, a lighter threshold — flag RPN ≥ 80 — is appropriate, because prototype runs are short and failures are recoverable. These are the failure modes you must address before your Gate 2 submission.

4.4.2 Scale Anchors

Table 4.1 gives representative scale anchors for S, O, and D as used in this course. Teams should agree on anchors *before* scoring to prevent rater drift.

Table 4.1: Representative 10-point scale anchors for FMEA scoring in a prototype context.

Score	Severity (S)	Occurrence (O)	Detection (D)
1–2	No user impact	Rare ($<$1 in 1,000 runs)	Immediately obvious; alarms fire
3–4	Minor degradation	Occasional ($\sim$1 in 100 runs)	Detectable by inspection or log
5–6	Partial loss of function	Moderate ($\sim$1 in 20 runs)	Detectable only with test setup
7–8	Major loss of function	Frequent ($\sim$1 in 5 runs)	Difficult; requires special tool
9–10	Safety / total failure	Near-certain ($>$1 in 2 runs)	Cannot be detected before impact

4.5 The Critical Path of Unknowns

The risk register produced by FMEA answers *which* risks are largest. A separate but related question is *which risks must be resolved first* — because some unknowns block progress on every other subsystem. This is the critical path of unknowns.

Figure 4.3 illustrates the assumption list and dependency arrows for the running example. The highest-RPN item that *also* blocks downstream work defines the prototype's first experiment.

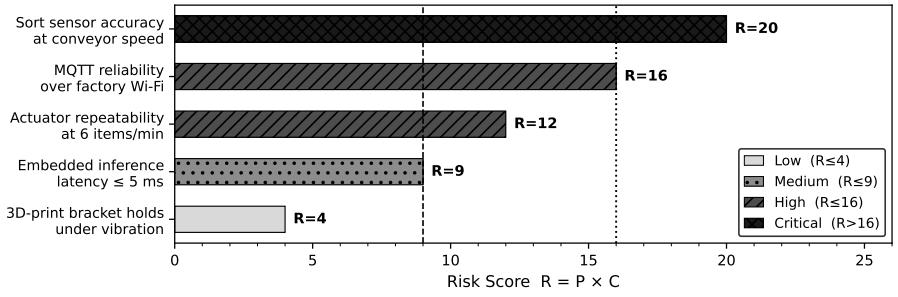

Figure 4.3: Critical unknowns and assumption dependency map for the IoT robotic sorting subsystem. Arrows show which assumptions must be resolved before dependent subsystem work can begin. The sort-sensor assumption (shaded) lies on the critical path because both the divert actuator timing and the firmware logic depend on its output format.

4.5.1 Identifying the Critical Unknown

To find the critical unknown, list every open assumption in the risk register and ask two questions for each:

1. If this assumption turns out to be false, how many other subsystems must change? (Coupling score.)
2. How quickly can we resolve it with a targeted experiment? (Resolvability score.)

An assumption with high coupling and low resolvability is both dangerous and slow to clear — it should head the prototype schedule. An assumption with high coupling but high resolvability should be the *first* experiment run, because a quick test removes the blocker cheaply.

4.5.1.0.1 In Practice. For the IoT sorter, the sort-sensor detection principle (time-of-flight vs. break-beam vs. vision) is the critical unknown: the firmware state machine, the actuator timing, and the MQTT payload format all depend on what the sensor outputs. Running a 48-hour bench test of candidate sensors *before* ordering actuator parts avoids a costly mid-sprint pivot.

4.6 Worked Example: IoT Robotic Sorting Subsystem

This section walks through the full specification and risk-ranking workflow for the running example established in Chapter 3: a connected IoT robotic sorting subsystem classifying and diverting items on a bench conveyor, reporting over MQTT/Wi-Fi, with a target BOM of $\leq$\$500 CAD and a sort accuracy of $\geq$95%.

4.6.1 Step 1: Convert Vague Wishes into Measurable Requirements

The team's initial wish list contained five phrases typical of early ideation. Table 4.2 shows each vague phrase converted to a testable requirement.

The most instructive conversion is "fast":

- **Vague:** "The system shall process sensor data fast."
- **Step 1 — name the quantity:** Processing speed is measured in samples per second (kS/s) and as end-to-end latency (ms).
- **Step 2 — anchor to the use case:** At 6 items/min, one item arrives every $60\,\text{s}/6 = 10\,\text{s}$. A single sort event reads one sensor frame, runs the classifier, and commands the actuator. The conveyor belt speed is 50 mm/s and the item window at the gate is 50 mm, giving a gate dwell time of $50\,\text{mm} \div 50\,\text{mm/s} = 1{,}000\,\text{ms}$. The actuator must fire within that window.
- **Step 3 — allocate the budget:** Budget allocation: sensor read 1 ms, classifier 2 ms, actuator command 2 ms, margin 995 ms.

$$t_{\text{firmware}} = 1\,\text{ms} + 2\,\text{ms} + 2\,\text{ms} = 5\,\text{ms}. \tag{4.3}$$

The remaining $1{,}000 - 5 = 995\,\text{ms}$ is dwell-time margin before the next item enters the gate. Conservatively, the end-to-end firmware latency budget is

$\leq$5 ms. At a 10 kS/s ADC rate, the sensor delivers $10{,}000$ samples per second; one classification frame uses 10 samples, costing $10/10{,}000 = 1\,\text{ms}$ — within budget.

- **Measurable requirement:** "The MCU firmware shall process one classification event (sensor read + classification + actuator command) within $\leq$5 ms, measured from sensor-trigger interrupt to actuator-enable GPIO, at a sustained rate of $\geq$10 kS/s sensor sampling."

Table 4.2: Vague feature wishes converted to measurable, testable requirements for the IoT robotic sorting subsystem. Each requirement identifies actor, action, threshold, and operating condition.

Vague Wish	Measurable Requirement	ID
"Sorts accurately"	The system shall classify items with $\geq$95% accuracy (correct class / total items) over a continuous 100-item run at nominal belt speed.	REQ-01
"Fast"	The MCU firmware shall complete one classification event within $\leq$5 ms (sensor read + classify + actuator command) at $\geq$10 kS/s sensor sampling.	REQ-02
"Enough throughput"	The system shall sustain $\geq$6 items/min throughput for $\geq$60 min without thermal throttling or error accumulation.	REQ-03
"Sends data to the cloud"	Each sort-event MQTT message shall be delivered to the broker within $\leq$200 ms of the sort event, on a 2.4 GHz Wi-Fi network with $\geq$-70 dBm RSSI.	REQ-04
"Affordable"	The total prototype BOM (all purchased components) shall not exceed $500 CAD.	REQ-05

A sixth implied constraint (power) is added from the hardware context:

CON-01: Total system power draw shall not exceed 5 W when powered from a single USB-C port (5 V, 2 A = 10 W; 5 W provides 50% margin for host compatibility).

4.6.2 Step 2: Compute RPN for Three Subsystems

The team identifies three high-risk subsystems from the Chapter 3 scoping exercise: (A) Sort Sensor, (B) MQTT/Wi-Fi Integration, (C) Divert Actuator. For each, the team identifies the primary failure mode, scores S, O, and D on the 1–10 scale from Table 4.1, and computes RPN using Equation (4.2).

Subsystem A — Sort Sensor

- **Failure mode:** Sensor misclassifies item class (false positive or false negative), causing a sort error.

- **Severity (S):** A miss ships a wrong item — significant product defect, customer complaint, possible recall. $S = 8$.
- **Occurrence (O):** Time-of-flight at <300 mm with a 50 mm/s belt is a moderately proven technique; reflectivity variation across item surface adds uncertainty. $O = 5$.
- **Detection (D):** A classification error is visible in the sort log and confirmed by counting missorted bins at end of run — easily detected during bench test. $D = 2$.
- **RPN:**

$$\text{RPN}_A = S \cdot O \cdot D = 8 \times 5 \times 2 = \mathbf{80}$$

Subsystem B — MQTT/Wi-Fi Integration

- **Failure mode:** MQTT message delivery latency exceeds 200 ms, causing dashboard staleness and potential missed alerts.
- **Severity (S):** Stale dashboard undermines the product's core value proposition (real-time visibility); does not stop physical sorting. $S = 7$.
- **Occurrence (O):** 2.4 GHz Wi-Fi in a lab environment with competing devices experiences frequent retransmissions; MQTT QoS 1 adds handshake overhead. $O = 6$.
- **Detection (D):** Latency can be measured by timestamping publish and subscribe events in the broker log — detectable but requires setup of a monitoring script. $D = 4$.
- **RPN:**

$$\text{RPN}_B = S \cdot O \cdot D = 7 \times 6 \times 4 = \mathbf{168}$$

Subsystem C — Divert Actuator

- **Failure mode:** Actuator fires too late or with insufficient force, failing to divert the item before it exits the gate.
- **Severity (S):** A missed divert is a sort error — same consequence as a sensor misclassification. $S = 8$.
- **Occurrence (O):** Servo or solenoid timing within a 1,000 ms dwell window is well within typical MCU PWM capability; mechanical linkage adds some uncertainty. $O = 3$.
- **Detection (D):** A missed divert is observable by watching the output bins or reading a secondary confirmation sensor — straightforward detection. $D = 2$.
- **RPN:**

$$\text{RPN}_C = S \cdot O \cdot D = 8 \times 3 \times 2 = \mathbf{48}$$

Table 4.3: FMEA Risk Priority Number ranking for the three prototype subsystems of the IoT robotic sorting subsystem. RPN computed using Equation (4.2). Subsystems are sorted by descending RPN.

Rank	Subsystem	Failure Mode	S	O	D	RPN	Action
1	MQTT/Wi-Fi	Latency >200 ms	7	6	4	**168**	Redesign / mitigate
2	Sort Sensor	Misclassification	8	5	2	**80**	Monitor / mitigate
3	Divert Actuator	Late divert	8	3	2	**48**	Monitor

4.6.3 Step 3: Rank and Interpret

Table 4.3 summarises the three RPN scores and maps each to a mitigation action.

Interpretation:

- MQTT/Wi-Fi integration (RPN = 168) is the highest-risk subsystem. The team should prototype and stress-test the Wi-Fi + MQTT stack *first*, before investing time in actuator tuning. Concrete mitigation: run a dedicated latency characterisation script (publish 500 timestamped messages; record broker-side receive timestamp; compute 99th-percentile latency) within Sprint 1.
- Sort Sensor (RPN = 80) exceeds the 80-threshold and requires a documented mitigation plan. The sensor's detection principle should be confirmed in a 100-item accuracy trial before the firmware classifier is written.
- Divert Actuator (RPN = 48) is below the 80-threshold. Standard bench testing during integration is sufficient; no dedicated mitigation sprint is needed.

Applying the basic risk equation (4.1) for cross-check:

$$R_A = P_{\text{failure},A} \times C_{\text{consequence},A} = 5 \times 8 = 40 \quad \text{(action zone)}$$
$$R_B = P_{\text{failure},B} \times C_{\text{consequence},B} = 6 \times 7 = 42 \quad \text{(action zone)}$$
$$R_C = P_{\text{failure},C} \times C_{\text{consequence},C} = 3 \times 8 = 24 \quad \text{(watch zone)}$$

The two-factor scores (4.1) agree with the RPN ranking on the top item (MQTT/Wi-Fi) but obscure the gap between A and B that the detection factor (D) reveals. The sensor is harder to detect as failing, so RPN correctly flags it above the threshold despite its lower occurrence rate.

4.6.4 Try It Yourself

Scenario: A team is developing a **smart irrigation controller** for a rooftop garden. The system reads soil moisture via a capacitive sensor array, decides whether to open a motorised valve, and logs readings to a cloud dashboard over cellular (LTE-M). The initial wish list contains:

- "Waters when needed" — target: soil moisture sensor accuracy $\pm$3% VWC
- "Doesn't over-water" — target: valve response within 30 s of a moisture threshold crossing

- "Cheap to build" — target: BOM $\leq$\$300 CAD

Your tasks:

1. Rewrite each wish as a measurable requirement following the actor–action–threshold–condition format.
2. Identify three failure modes (one per subsystem: moisture sensor, motorised valve, LTE-M uplink).
3. Assign S, O, and D scores on the 1–10 scale with written justification for each S, O, and D value (nine justifications total: three failure modes × three dimensions).
4. Compute RPN for each failure mode using Equation (4.2), showing every multiplication step.
5. Rank the three failure modes from highest to lowest RPN and state which subsystem should be prototyped first.
6. For this cross-check, use the O score as P_{failure} and the S score as $C_{\text{consequence}}$, consistent with the worked example convention. Cross-check your top-ranked item using Equation (4.1) and comment on whether the two rankings agree.

Do **not** use the same S, O, D values as the worked example.

Review Questions

1. **(Conceptual)** Explain the difference between a *functional requirement* and a *performance target*, using one example of each from a product domain of your choice. Why must both be present in a complete specification?
2. **(Conceptual)** A colleague argues that specifying the communication protocol (e.g., "shall use Bluetooth Low Energy") inside a requirement is efficient because it removes ambiguity for the firmware team. Justify why this practice is problematic and describe the correct place in the documentation hierarchy for such a decision.
3. **(Applied — calculation)** A new IoT product team scores three failure modes as follows:

   - Power regulator brown-out: $S = 9$, $O = 2$, $D = 3$
   - Firmware deadlock: $S = 6$, $O = 7$, $D = 8$
   - Enclosure latch failure: $S = 3$, $O = 4$, $D = 1$

   Compute RPN for each failure mode using Equation (4.2), showing every multiplication step. Rank them and state which subsystem requires a documented mitigation plan if the threshold is RPN ≥ 80.
4. **(Applied — multi-step calculation)** A wearable ECG monitor has a motion-artifact failure mode with the following FMEA scores: Severity $S = 8$, Occurrence $O = 4$, Detection $D = 3$.

   (a) Compute $R = P_{\text{failure}} \times C_{\text{consequence}}$ (Equation 4.1) using $P_{\text{failure}} = O = 4$ and $C_{\text{consequence}} = S = 8$ as the cross-check convention. Does this failure mode qualify as high-risk ($R > 16$)?

(b) Compute the RPN. The team then adds a motion-artifact filter that lowers O from 4 to 2 while S and D remain unchanged. Recompute the RPN and state whether the mitigation moves the failure mode below the RPN threshold of 80.

(c) Comment in one sentence on whether the R-score ranking (Part a) and the RPN ranking (Part b) agree.

5. **(Conceptual)** Define the *critical path of unknowns* and explain how it differs from the highest-RPN item in the risk register. Give an example where the highest-RPN item is *not* on the critical path of unknowns and describe what the team should do first.

Portfolio Entry

You produce the core of **Milestone 1: the Proposal** — together these three parts constitute your Gate 2 evidence package, the scoping deliverables that earn a Go decision to proceed with development. The artifact has three interlocking parts:

1. **Specification Table.** A table of 3–5 measurable requirements (functional requirements + performance targets) and any binding constraints, formatted with columns: ID, Requirement Statement, Threshold, Operating Condition, Acceptance Test. Each row must be testable as written.

2. **Risk Register.** An FMEA table listing at least three failure modes (one per high-risk subsystem identified in Chapter 3). Each row contains: Subsystem, Failure Mode, S, O, D, RPN, Rank, and Mitigation Plan. The table is sorted by descending RPN. Failure modes above RPN 80 must have a specific mitigation action tied to a sprint.

3. **Scoping Rationale.** One paragraph (100–150 words) that names: (a) the central prototype question, (b) the highest-risk assumption (the critical unknown), and (c) the first experiment that will resolve that assumption. This paragraph becomes the executive summary of your Milestone 1 submission.

'What will this thing do, how will you know it works, and what could kill the project?' — these three parts of the Milestone 1 Proposal answer that investor question directly.

Summary

A specification is a contract: every element — functional requirements, performance targets, constraints, interfaces, and acceptance criteria — must be measurable and testable, or it is not yet a requirement. It established the discipline of separating *what* (requirements) from *how* (design), a distinction that keeps early design choices from foreclosing better alternatives. Two complementary risk tools were presented: the two-factor risk equation $R = P_{\text{failure}} \times C_{\text{consequence}}$ (Equation (4.1)) for rapid stakeholder communication, and the FMEA Risk Priority Number RPN $=$

$S \cdot O \cdot D$ (Equation (4.2)) for engineering-level prioritisation that accounts for detection difficulty. Applied to the IoT robotic sorting subsystem, the analysis ranked MQTT/Wi-Fi integration (RPN = 168) as the first prototype target, followed by the sort sensor (RPN = 80), and confirmed that the divert actuator (RPN = 48) can proceed with standard integration testing.

You now have a specification table, a risk register, and a scoping rationale — the three components of the Milestone 1 Proposal. Chapter 5 takes this artifact as its starting point: the specification defines the required behaviour at each subsystem boundary, and the risk register flags which interfaces carry the highest-consequence unknowns. Chapter 5 shows how to decompose the system into subsystems, define the interfaces between them, and produce the architecture diagram that makes integration testable.

CHAPTER 5

SYSTEM ARCHITECTURE & INTERFACE DEFINITION

5.0.0.0.1 Where this fits. You are now between Stage-Gate Stage 2 and Stage 3. Chapter 4 produced a specification and a risk register: you know *what* the prototype must do and which unknowns carry the highest risk — but not yet *how* to partition that specification into subsystems with verified boundaries. This chapter produces the next mandatory input to Gate 3 — the system architecture and interface table. Before you order a single component or write a single line of firmware, you must decompose your specification into subsystems and write down exactly what each subsystem promises to deliver to the next one. Architecture decisions made here are dramatically cheaper to change than the same decisions discovered at integration. A sound architecture is a prerequisite for an honest Gate 3 business case: if the blocks do not fit together on paper, they will not fit together on the bench.

By the end of this chapter you will be able to:

1. decompose a prototype specification into subsystems across the four domains: electronics, mechanical, firmware, and AI/IoT/robotic logic;
2. specify **interface contracts** that define the electrical, mechanical, data, and timing properties at each subsystem boundary;
3. allocate timing, power, and error budgets across subsystems and verify that the system total satisfies the specification; and
4. justify which subsystem boundary in a given architecture carries the highest integration risk by applying the detection and severity criteria from Chapter 4's FMEA framework.

5.1 Decomposition

Decomposition is the act of partitioning a system specification into a set of subsystems, each responsible for a bounded, well-defined portion of the total behaviour. Done well, decomposition gives every team member a clear scope and makes the integration task predictable. Done poorly — or skipped entirely — it produces

a monolithic prototype where every bug is a whole-system bug and no one is sure whose responsibility the failure is.

The discipline in this course groups prototype subsystems into four domains. Together they cover every boundary that a typical mechatronic, IoT, or robotic prototype will cross.

5.1.1 The Four Domains

5.1.1.1 Electronics

The **electronics subsystem** includes all hardware that converts physical quantities into digital signals and provides power conditioning for the rest of the system. In the running-example sorter this domain contains the item-detection sensor (time-of-flight or photoelectric), the camera or vision sensor feeding the AI pipeline, the microcontroller unit (MCU), motor drivers, and the power supply rail. Typical deliverables from this domain: a schematic, a net list, a power-rail diagram, and connector pinouts at every physical boundary.

5.1.1.2 Mechanical

The **mechanical subsystem** governs physical structure and motion: frame, conveyor belt, diverter actuator, mounting provisions for sensors and electronics, and the enclosure. In the prototype scope established in Chapter 2, the housing is functional-minimum — sufficient to hold the electronics securely and at the correct sensor geometry, but not production-finished. Mechanical deliverables: dimensional drawings for custom parts, assembly sequence, and the mechanical interface dimensions at every point where electronics or firmware interact with moving parts.

5.1.1.3 Firmware

The **firmware subsystem** is the embedded software running on the MCU. Its responsibilities are: reading the sensor, scheduling the AI inference call, issuing the actuator command, and publishing the MQTT status message. For the running-example sorter, the timing budget from Chapter 4 constrains this domain directly: the firmware must complete the entire sensor-to-actuator pipeline within 5 ms (established in Chapter 4 and formalised as Equation 5.1 in Section 5.3). Firmware deliverables: a task or interrupt map, a state-machine diagram, and a timing annotation for every I/O transaction.

5.1.1.4 AI/IoT/Robotic Logic

The **AI/IoT/robotic logic subsystem** encompasses the classification algorithm (the AI inference engine), the MQTT broker connection, and any cloud-side dashboard or data logging. This is the layer that gives the prototype its intelligence: the ability to distinguish item classes and to report status remotely. For the running example,

the central question from Chapter 3 — can the sub-$500 CAD BOM sorter achieve $\geq$95% sort accuracy at 6 items/min with MQTT status reporting? — is answered primarily by this domain, in concert with the electronics sensor quality. Deliverables: model architecture summary, inference-time benchmark, MQTT topic map, and QoS configuration.

5.1.2 Drawing the Block Diagram

The output of decomposition is a **system block diagram**: one box per subsystem, one labelled arrow per interaction. Figure 5.1 shows the block diagram for the running-example sorter. Every arrow in that figure will become one row in the interface table you will build by the end of this chapter.

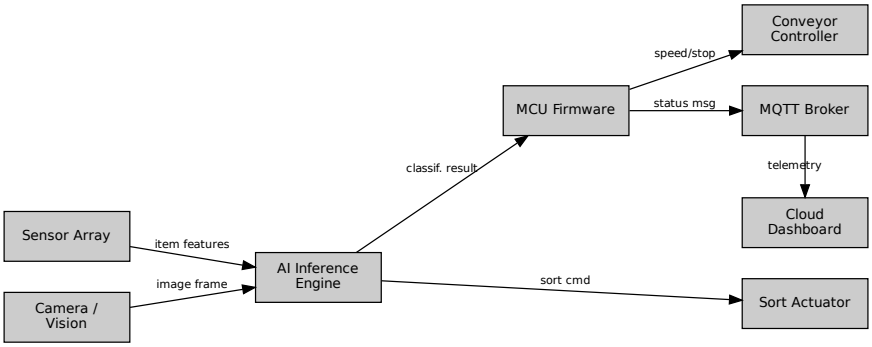

Figure 5.1: System block diagram for the IoT robotic sorting prototype. Boxes represent subsystems across the four domains; arrows represent interface boundaries, each of which receives a contract in Table 5.1. Labels A–D correspond to the four interface types introduced in Section 5.2.

5.1.2.0.1 In Practice. At the start of a team project, produce the block diagram before any other technical document. Pin it to the shared drive or project wall. Every team member should be able to point to their box and describe exactly what enters and exits it. A one-page interface table built from this diagram is the single most effective tool for keeping all team members building to the same specification. When everyone can see the same table, "I assumed it would be 3.3 V" is no longer a valid explanation for a three-day debugging sprint.

5.1.2.0.2 Common Pitfalls.

- **Starting the interesting subsystem before interfaces are fixed.** Firmware and AI engineers are often eager to write code before the electrical interfaces are settled. If the UART voltage level has not been agreed upon, any firmware written against an assumed level may be wrong, and the rework cost is real schedule time.
- **Treating the block diagram as decoration.** A block diagram that lives in a report appendix and is never referenced during build is not doing its job. It must be a living, binding contract — updated any time a subsystem boundary

changes, and consulted before any interface-crossing code or hardware is designed.

- **Verbal interface agreements.** "We'll figure out the data format later" is not an interface definition. The team that built the firmware and the team that built the AI classifier will make different assumptions, and those assumptions will collide at integration.

5.2 Interfaces as Contracts

An **interface** is the shared boundary between two subsystems. At that boundary one subsystem makes a promise to another. Calling it a **interface contract** is deliberate: both sides are obligated. The provider subsystem must deliver exactly what is specified; the consumer subsystem must not demand anything beyond what is specified. When the contract is written down, a violation is detectable. When the contract lives only in someone's head, it is invisible until the two subsystems meet on the bench.

There are four interface types used throughout this course. Each type has a different set of properties that must be specified to make the contract complete.

5.2.1 Electrical Interfaces

An **electrical interface** specifies:

- **Voltage level:** logic-high and logic-low thresholds (e.g., 3.3 V CMOS or 5 V TTL).
- **Current capacity:** the maximum source/sink current the provider can supply and the maximum the consumer may draw.
- **Protocol:** the communication standard (UART, SPI, I^2C, PWM, analog 0–5 V).
- **Connector:** physical connector type and pinout so that the mating is unambiguous.
- **Protection:** ESD handling, over-current limiting, or level-shifting requirements.

For the running-example sorter, the interface between the MCU and the motor driver is an electrical interface: 3.3 V PWM out from the MCU, 5 V logic input on the motor driver, requiring a level-shifter. Specifying the level-shifter requirement at architecture time — rather than discovering the incompatibility at bench time — is the direct return on investment for this chapter's work.

5.2.2 Mechanical Interfaces

A **mechanical interface** specifies:

- **Mounting geometry:** hole pattern, spacing, and tolerances.

- **Physical envelope:** maximum bounding box or keep-out volume.
- **Force or torque limits:** the maximum load the mating structure must carry.
- **Material compatibility:** relevant where corrosion, heat, or vibration is a factor.

For the running-example sorter, the mechanical interface between the sensor bracket and the conveyor frame defines the exact standoff height at which the time-of-flight sensor must sit relative to the belt surface. If the firmware team writes their detection algorithm assuming a 150 mm standoff but the mechanical team fabricates a 200 mm bracket, the item-detection gate geometry is wrong and every downstream timing calculation inherits that error.

5.2.3 Data Interfaces

A **data interface** specifies:

- **Protocol and transport:** MQTT, HTTP, I^2C register map, SPI frame format, serial byte stream.
- **Message format:** field names, data types, units, and byte order.
- **Topic or address:** MQTT topic string, I^2C device address, REST endpoint.
- **Quality of Service (QoS):** for MQTT, the QoS level (0, 1, or 2) and retained-message flag.
- **Error handling:** what happens if a message is lost, malformed, or arrives out of order.

For the running-example sorter, the data interface between the MCU firmware and the cloud dashboard is an MQTT publish on topic `sorter/status` with a JSON payload containing `item_class`, `sort_result`, and `timestamp_ms`. Any team member building the dashboard must consume exactly that format; any firmware update that renames a field without updating the interface table breaks the contract.

5.2.4 Timing Interfaces

A **timing interface** specifies:

- **Latency:** the maximum time between a trigger event and the expected response.
- **Throughput:** the maximum number of events per second the interface must sustain.
- **Jitter:** the maximum variation in latency from one cycle to the next.
- **Deadline type:** hard (a missed deadline is a system failure), firm (a missed deadline degrades output quality), or soft (a missed deadline is acceptable occasionally).

For the running-example sorter, the timing interface between the sensor read and the AI classifier is: sensor output available within $t_1 \leq 1$ ms of item detection, with jitter ≤ 0.1 ms. The AI classifier must begin its inference within that window to keep the 5 ms pipeline budget intact.

5.2.5 The Interface Table

All four interface types are collected into a single **interface table** — one row per arrow in the block diagram. Table 5.1 shows the interface table for the running-example sorter. This table is the primary deliverable of this chapter and must be completed before any subsystem build begins.

Table 5.1: Interface table for the IoT robotic sorting prototype. Each row names a boundary from Figure 5.1, its type, the contract properties, and the timing allocation from Section 5.3.

ID	From	To	Contract (key properties)	Latency
A	Sensor	Firmware	Electrical: 3.3 V I^2C, 400 kHz; range reading in 2-byte register; detection interrupt on GPIO	$t_1 \leq 1$ ms
B	Firmware	AI Engine	Data: shared memory frame buffer; 320×240 px RGB888; semaphore handshake	$t_2 \leq 2$ ms
C	AI Engine	Firmware	Data: classification label (uint8) + confidence (float32); written to result register; sets done flag	included in t_2
D	Firmware	Motor Driver	Electrical: 3.3 V PWM → level-shifted 5 V; direction GPIO; 20 kHz carrier	$t_3 \leq 2$ ms
E	Firmware	MQTT Broker	Data: TCP/IP; topic `sorter/status`; JSON payload; QoS 1; ≤200 ms publish latency	async
F	Sensor Bracket	Conveyor Frame	Mechanical: M3 × 4-hole pattern, 20 mm spacing; standoff 150 ± 2 mm; max load 0.5 N	N/A

5.3 Budget Allocation

Specifying interfaces is necessary but not sufficient. You must also verify that the sum of all subsystem demands does not exceed the system-level constraint. This verification is called **budget allocation**: you divide a finite resource — time, power, or permissible error — across the subsystems that consume it, and then confirm the total stays within the specification limit.

Three budgets matter for the running-example prototype: timing, power, and measurement error.

5.3.1 Timing Budget

The **timing budget** formalises the constraint that the sum of all pipeline-stage latencies must not exceed the specification deadline.

$$T_{\text{total}} = \sum_{i} t_i \leq T_{\text{budget}} \tag{5.1}$$

This is the budget structure introduced in Chapter 4's risk analysis (Equation 4.3). Here it is stated formally as the governing inequality for the architecture. Every timing interface in the interface table must carry a t_i allocation; those allocations are substituted into Equation 5.1 to verify the total.

5.3.2 Power Budget

The **power budget** verifies that the supply rail can deliver enough current for all subsystems simultaneously.

$$P_{\text{total}} = \sum_{i} P_i \tag{5.2}$$

P_{total} must not exceed the rated output of the power supply. For the running-example sorter on a 5 V/2 A USB supply (10 W), typical subsystem allocations are: MCU 0.5 W, camera module 1.2 W, motor driver (idle) 0.8 W, motor driver (active) 2.5 W, Wi-Fi module 0.9 W — a peak total of 5.9 W, well within the 10 W budget with a 4.1 W margin. Margins matter: a motor driver that draws more than rated, a Wi-Fi transmit burst that spikes current, or a future addition of an LED indicator can each erode headroom. Figure 5.2 shows the timing and power budgets as stacked bars against their respective limit lines.

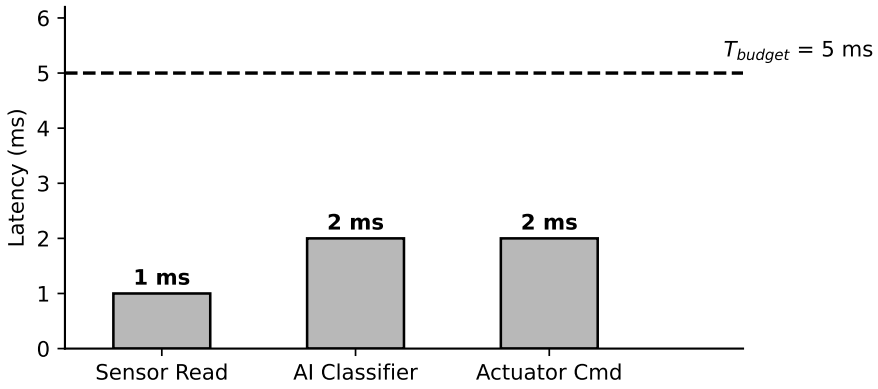

Figure 5.2: Interface budgets for the running-example sorter. Left panel: timing budget — stacked bar of t_1, t_2, t_3 against the 5 ms limit line (Equation 5.1). Right panel: power budget — stacked bar of subsystem power draws against the 10 W supply limit (Equation 5.2). Zero margin on timing means any added pipeline step requires removing an existing one.

5.3.3 Error Budget

Multiple measurement stages each contribute their own uncertainty. Assuming the error sources are independent and random, they combine in **quadrature** (root-sum-of-squares, RSS):

$$e_{\text{total}} = \sqrt{\sum_i e_i^2} \tag{5.3}$$

The RSS model is more realistic than a worst-case linear sum because independent errors partially cancel. If e_{total} exceeds the acceptance criterion, the team must tighten the largest individual contributor first — squeezing a small contributor yields almost no benefit once a dominant error source exists.

5.3.4 Worked Example — Part A: Timing Budget

The central question from Chapter 3 requires classifying and diverting each item within $\leq$167 ms of arrival (6 items/min), but the firmware pipeline latency must be far shorter so that the MQTT publish and mechanical diversion can complete within that window. Chapter 4's risk analysis established a firmware timing budget of $T_{\text{budget}} = 5$ ms with three pipeline stages. Applying Equation 5.1:

$$T_{\text{total}} = t_1 + t_2 + t_3 = 1 \text{ ms} + 2 \text{ ms} + 2 \text{ ms} = 5 \text{ ms}$$

$$T_{\text{total}} = 5 \text{ ms} \leq T_{\text{budget}} = 5 \text{ ms} \quad \checkmark$$

where:

- $t_1 = 1$ ms: sensor read (I^2C transaction, distance measurement, interrupt service);
- $t_2 = 2$ ms: AI classifier inference (image capture, pre-processing, forward pass, label decode);
- $t_3 = 2$ ms: actuator command (PWM update, GPIO direction set, motor driver settling).

The budget passes, but the **dwell-time margin is zero**. Any added pipeline step — a second sensor read for confirmation, a logging write to flash, a display update — requires shortening one of the existing stages by an equal amount. This constraint must be visible to every team member: firmware engineers do not add steps to this pipeline without a corresponding timing analysis showing where the recovered time comes from.

5.3.5 Worked Example — Part B: RSS Error Budget

The sort-accuracy specification requires $\geq$95% classification accuracy. Three independent sources of measurement uncertainty contribute to positional error at the classification gate:

- $e_1 = \pm 0.05$ m/s: conveyor speed sensor uncertainty, which introduces positional error when the system computes item position from belt velocity. At the

nominal belt speed of 0.05 m/s, a speed uncertainty of ± 0.05 m/s maps to a relative positional uncertainty of $\pm(0.05/0.05) \times 1\% = \pm 0.1\%$; this is the value substituted as e_1 in the RSS calculation.

- $e_2 = \pm 0.5\%$: AI vision classification uncertainty (probability gap between top-1 and top-2 class, expressed as a percentage-point uncertainty in the sort decision).
- $e_3 = \pm 0.3\%$: positional uncertainty equivalent derived from timing jitter of ± 0.1 ms mapped through the nominal belt speed.

Expressing all contributors in compatible units (percentage-point equivalent positional uncertainty) and applying Equation 5.3:

$e_1 = 0.1\%$, $e_2 = 0.5\%$, $e_3 = 0.3\%$

$$e_{\text{total}} = \sqrt{(0.1\%)^2 + (0.5\%)^2 + (0.3\%)^2} = \sqrt{0.01 + 0.25 + 0.09}\% = \sqrt{0.35}\% \approx 0.59\%$$

The acceptance criterion is $e_{\text{total}} \leq 0.6\%$.

$$e_{\text{total}} \approx 0.59\% \leq 0.60\% \quad \checkmark$$

The error budget passes — but barely. The dominant contributor is e_2, the AI classification uncertainty. If the model's confidence gap narrows (e.g., the item class becomes harder to distinguish), this budget will fail at that contributor first. Chapter 6 will revisit this when selecting the camera sensor and AI model architecture.

5.3.5.0.1 In Practice. A team building a motor controller prototype skipped the electrical interface table and started firmware before agreeing on the UART voltage level between the MCU and the motor driver. The MCU operated at 3.3 V logic; the motor driver expected 5 V logic. The firmware team wrote and tested their UART code against a bench logic analyser at 3.3 V. The electronics team wired the motor driver directly to the MCU TX pin. At integration, the motor driver received marginal logic levels: it sometimes responded and sometimes did not, depending on supply current at the moment of the transaction. Diagnosing the intermittent fault cost three days. The fix was a single-chip level-shifter — a $0.30 CAD part that would have taken ten minutes to add to the schematic during architecture definition. The interface table entry for that boundary would have read: *"Electrical: MCU UART TX at 3.3 V CMOS logic; motor driver UART RX requires 5 V TTL; level-shifter required on this net."* Three days of debugging for a $0.30 part and one table row.

5.4 Why Integration Fails at Boundaries

Subsystems fail at their boundaries, not at their centres. This statement is almost universally true in mechatronic and embedded prototype development, and it has a straightforward explanation.

Within a subsystem, one engineer (or one small team) controls all the assumptions. The firmware engineer who writes the sensor driver and the interrupt handler is working within a single, consistent mental model. The mechanical engineer who designs the conveyor frame is making choices within their own constraints. Bugs within a subsystem are caught during subsystem-level testing because the developer is testing exactly the thing they understand.

At a boundary, two or more parties each independently assumed something about the other's subsystem. Those assumptions were never formally compared. When the two subsystems meet for the first time, every mismatched assumption becomes a defect. Figure 5.3 illustrates a canonical boundary mismatch: two subsystems whose interface contracts were never formally aligned, resulting in an incompatible handshake at integration.

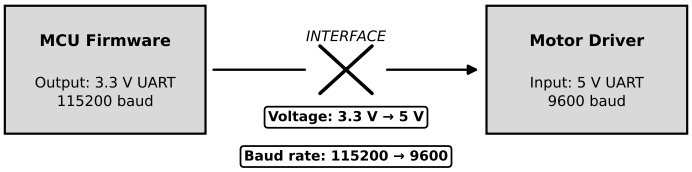

Figure 5.3: Boundary mismatch at integration. The left subsystem (MCU firmware) assumes 3.3 V CMOS logic and outputs a valid signal by its own standard; the right subsystem (motor driver) expects 5 V TTL and interprets the same signal as an indeterminate logic level. Neither subsystem contains a bug in isolation; the defect exists only at the boundary, and is invisible until the two are connected.

5.4.1 Taxonomy of Boundary Failures

Four categories of boundary failure recur across student and professional projects alike.

5.4.1.1 Voltage Level Mismatch

The most common electrical boundary failure: one subsystem drives 3.3 V logic, the adjacent subsystem expects 5 V, and neither team checked the other's datasheet. Outcome: intermittent communication, latent damage to 3.3 V pins exposed to 5 V, or total non-function. Prevention: one row in the electrical interface table, one level-shifter on the BOM.

5.4.1.2 Protocol Mismatch

One subsystem writes a SPI frame formatted as 16-bit big-endian; the consuming subsystem reads it as 16-bit little-endian. Both subsystems pass their individual unit tests. The integration test produces garbage output that takes hours to trace to a byte-order disagreement. Prevention: specify byte order, data type, and endianness in the data interface contract.

5.4.1.3 Timing Assumption Mismatch

The firmware team assumes the AI engine returns a result within 2 ms. The AI team benchmarks their model at 3 ms on the prototype hardware. The pipeline budget fails by 1 ms and the diverter fires 1 ms after the item has already passed the gate. Prevention: the timing interface contract specifies the maximum latency; the AI team runs a benchmark on the actual target hardware and signs off before build begins.

5.4.1.4 Mechanical Fit Mismatch

The sensor bracket is designed for a sensor with an M2.5 mounting hole. The sensor that arrives has an M3 mounting hole. Fabrication is already complete. Prevention: the mechanical interface contract specifies the exact hole pattern, fastener size, and tolerance, and is verified against the component datasheet before fabrication is released.

5.4.2 Identifying the Highest-Risk Boundary

Not all boundaries carry equal risk. The boundary with the highest integration risk is the one where:

1. the two subsystems are developed by different people or teams with limited communication;
2. the interface contract is the least precisely specified; and
3. a mismatch at this boundary would be the hardest to detect before it causes visible system failure.

This is precisely the FMEA detection factor D from Chapter 4 applied at the interface level. For the running-example sorter, the firmware-to-AI-engine boundary (Interface B in Table 5.1) carries the highest integration risk: the shared memory frame buffer protocol is non-standard, the two subsystems may be developed on different toolchains, and a timing assumption mismatch here propagates directly into a missed divert-gate deadline.

5.4.2.0.1 Common Pitfalls.

- **"We'll figure it out later" interface agreements.** Verbal agreements about data formats, voltage levels, or timing are not interface contracts. They are deferred arguments. Experienced engineers know that "later" always arrives during integration, when schedule pressure is highest and the cost of rework is greatest.
- **Developing the interesting subsystem first.** AI engineers naturally want to train and test the classifier. Firmware engineers want to write drivers. Without fixed interfaces, each group builds to their own internally consistent assumptions,

and integration becomes a negotiation between two finished products that were never designed to meet each other.

- **Interface table as a compliance artifact.** Filling out the interface table to satisfy a deliverable requirement, then ignoring it during build, defeats its purpose. The table must be the reference document that every team member checks before crossing a subsystem boundary.

Try It Yourself

Scenario: You are architecting a wireless air-quality monitor. The specification requires a total firmware pipeline latency of $T_{\text{budget}} = 8$ ms. Three pipeline stages have been proposed:

- $t_1 = 2$ ms: ADC sample (sensor read and digitisation);
- $t_2 = 3$ ms: digital filter (moving-average computation on the MCU);
- $t_3 = 2$ ms: BLE transmit buffer (packetise reading and push to BLE stack).

Three measurement error sources contribute to the reported air-quality reading:

- $e_1 = \pm 0.3$°C equivalent: temperature sensor uncertainty;
- $e_2 = \pm 1.5$°C equivalent: humidity sensor uncertainty (the % RH specification has been pre-converted to a temperature-equivalent for this budget);
- $e_3 = \pm 0.1$°C equivalent: ADC quantization noise.

The acceptance criterion for the combined error is $e_{\text{total}} \leq 1.6$°C equivalent.

Tasks:

(a) **Timing budget.** Using Equation 5.1, compute $T_{\text{total}} = t_1 + t_2 + t_3$. State whether the timing budget is satisfied and identify the remaining margin (if any).

(b) **Error budget.** Using Equation 5.3, compute e_{total} from the three error sources. State whether the error budget passes the acceptance criterion and identify the dominant contributor.

(c) **Constraint violation response.** If the firmware team's digital filter implementation turns out to require $t_2 = 4$ ms instead of 3 ms, T_{total} will exceed T_{budget}. Identify which stage(s) could be shortened to restore compliance. For each option you propose, state the new t_i value and verify that the revised $T_{\text{total}} \leq 8$ ms.

(d) **Interface contract.** Identify one interface in this air-quality monitor system (choose a boundary you consider the highest integration risk). Write a two-line interface contract for that boundary, specifying: (line 1) the protocol and voltage level or data format; (line 2) the timing constraint.

Review Questions

1. **(Conceptual)** State the four interface types defined in this chapter. For each type, give one specific example from the running-example IoT sorter system, naming the two subsystems at the boundary and one key property of the contract.
2. **(Conceptual)** Explain in two sentences why integration bugs typically appear at boundaries between subsystems rather than within a subsystem. Your answer should reference the concept of locally consistent assumptions within a subsystem versus the unverified assumptions that cross a boundary.
3. **(Conceptual — classification)** A team proposes an interface between the MCU and a Bluetooth module described only as "send the JSON string over serial." Classify the completeness of this interface contract: state which of the four interface types it partially addresses, which required properties are missing, and explain why the omissions constitute integration risk.
4. **(Applied — calculation)** A firmware pipeline has four stages with the following latency allocations: $t_1 = 0.8$ ms, $t_2 = 1.2$ ms, $t_3 = 0.6$ ms, $t_4 = 0.9$ ms. The timing specification requires $T_{\text{budget}} = 4$ ms.
   (a) Compute T_{total} using Equation 5.1.
   (b) State whether the timing budget is satisfied. If it is satisfied, state the margin.
   (c) An added 0.8 ms logging step is required. How must the existing stage allocations be adjusted to keep $T_{\text{total}} \leq 4$ ms? State which stage(s) to shorten and by how much.
5. **(Applied — multi-step)** Three subsystems each contribute independent positional measurement error to a parts-placement robot: encoder $e_1 = \pm 0.2$ mm, vision sensor $e_2 = \pm 0.5$ mm, synchronisation jitter equivalent $e_3 = \pm 0.1$ mm. The acceptance criterion is $e_{\text{total}} \leq 0.7$ mm.
   (a) Compute e_{total} using Equation 5.3. Show all intermediate steps.
   (b) State whether the error budget passes the acceptance criterion. If it passes, state the margin; if it fails, state the deficit.
   (c) The vision sensor is upgraded to a model with ± 0.3 mm uncertainty. Recompute e_{total} and state the new margin against the 0.7 mm criterion.

Portfolio Entry

By the end of this chapter your project folder must contain one deliverable:

System Architecture Package: the system block diagram (Figure 5.1) plus a completed interface table with every subsystem boundary from that diagram named, its four-property contract defined, and the timing and power budget columns filled in and verified against Equations 5.1 and 5.2.

Specifically, the package must demonstrate:

- A labelled block diagram with one box per subsystem across the four domains (electronics, mechanical, firmware, AI/IoT/robotic logic) and one labelled arrow per interface.
- An interface table in which each row names the two subsystems at the boundary, states the interface type (electrical, mechanical, data, or timing), and specifies the key contract properties.
- A timing budget column showing t_i allocations that sum to $T_{\text{total}} \leq T_{\text{budget}}$ (Equation 5.1).
- A power budget column showing P_i allocations that sum to P_{total} (Equation 5.2), with the total compared to the rated supply output.
- A written identification of the highest-risk boundary, with a two-sentence justification referencing the interface type and the detection difficulty of a mismatch at that boundary.

The block diagram and completed interface table together form the **Gate 3 architecture deliverable** (see the Stage-Gate model established in Chapter 2). A Gate 3 reviewer will verify: (1) every arrow in the block diagram has a corresponding row in the interface table; (2) each row carries a complete four-property contract; and (3) the timing and power budget columns close against Equations 5.1 and 5.2. A package that fails any of these three checks does not satisfy the Gate 3 architecture criterion.

Chapter Summary

Three tools now equip you to move from specification to buildable architecture.

First, a **decomposition framework**: dividing any prototype into four subsystem domains — electronics, mechanical, firmware, and AI/IoT/robotic logic — gives every team member an unambiguous scope and transforms integration from a surprise into a managed handshake.

Second, a **contract language for interfaces**: every boundary between subsystems carries a formal contract specifying electrical properties, mechanical geometry, data format, and timing constraints. A boundary without a contract is a liability that will be paid at integration time. The interface table is the instrument that makes all contracts visible to the whole team simultaneously.

Third, a **budget discipline**: timing, power, and measurement error are finite resources. Equations 5.1, 5.2, and 5.3 provide the arithmetic to verify that the sum of subsystem demands fits within the system-level limits. For the running-example sorter, the 5 ms timing budget is exactly consumed by the three pipeline stages, leaving zero margin; the power budget carries a 4.1 W margin; and the RSS error budget passes the 0.6% criterion by the narrowest of margins, flagging AI classification uncertainty as the risk to watch.

The architecture established here is the foundation on which Chapter 6 builds: the System Architecture Package gives Chapter 6 its starting point, and every component selected there will be evaluated against the interface contracts written here.

J. F. N. Salik,

CHAPTER 6

COMPONENT SELECTION & DATASHEET LITERACY

6.0.0.0.1 Where this fits. You are now inside Stage 3 (Development), moving toward Gate 3. Chapter 5 delivered the system block diagram and interface table — the verified architecture with its contracts and budgets. That gave you blocks to fill — but not yet a method for choosing the components that will populate them. This chapter teaches that method. Component selection at Stage 3 locks in a large fraction of unit cost, reliability, and supply-chain risk before a single board is laid out; a poor choice here is expensive to reverse. The deliverable you will carry into Gate 3 is a **component selection table** for the critical path of your prototype: part number, supplier, lifecycle status, derating factor, operating margin, alternate part, and system MTBF.

By the end of this chapter you will be able to:

1. extract the four critical parameter zones from a component datasheet (absolute maximum ratings, recommended operating conditions, electrical characteristics, timing diagrams);
2. apply derating factors to select components with adequate operating margin;
3. evaluate a component's lifecycle status using manufacturer product-page data;
4. identify a viable second source that satisfies parametric equivalence, footprint compatibility, and Active lifecycle status; and
5. estimate the failure rate of a series system from individual component MTBF values.

6.1 Reading a Datasheet

A **datasheet** is the manufacturer's binding specification for a component. It is not a tutorial, a marketing document, or a suggestion. Every number in a datasheet has

a precise meaning, and your ability to extract and apply those numbers correctly separates a reliable design from one that fails unpredictably in the field.

You read a datasheet in a fixed order. You do not read from page one to the end. They go directly to four zones — in the order listed below — before they read anything else. Figure 6.1 annotates the anatomy of a representative datasheet, marking each of the four zones.

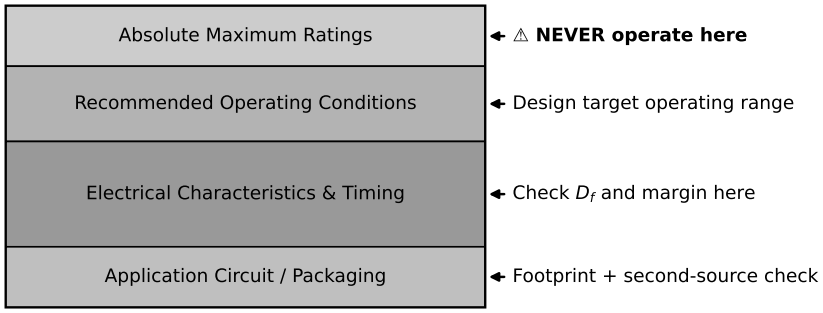

Figure 6.1: Annotated anatomy of a component datasheet showing the four critical zones: (1) absolute maximum ratings, (2) recommended operating conditions, (3) electrical characteristics, and (4) timing diagrams. Zone boundaries and key parameters are highlighted. Application circuit and packaging sections are labelled but not highlighted because they are consulted after the four critical zones are verified.

6.1.1 Zone 1 — Absolute Maximum Ratings

Absolute maximum ratings are the values beyond which permanent damage to the component is possible. They are not operating limits. The word "maximum" does not mean "allowed operating point" — it means "do not exceed under any condition, even briefly, or the silicon, ceramic, or polymer may be permanently damaged." A component operated at its absolute maximum voltage may work today, tomorrow, and for a week. It may also degrade silently and fail during the customer demonstration. The manufacturer guarantees *nothing* about behaviour at or above these limits.

When you open a new datasheet, find the absolute maximum ratings table first. Write down the supply voltage maximum, the maximum junction temperature, and the maximum input voltage at any pin you plan to drive. These are your hard fences.

6.1.2 Zone 2 — Recommended Operating Conditions

Recommended operating conditions are the voltage, current, temperature, and frequency ranges within which the manufacturer tests the component and guarantees all published electrical characteristics. These limits are significantly narrower than absolute maximum ratings. Operating inside the recommended conditions is the minimum standard for a professional design.

The gap between the recommended operating limit and the absolute maximum limit is not a design margin you are allowed to use. It is a zone the manufacturer reserves for statistical variation, lot-to-lot spread, and the mathematical

uncertainty in their own test equipment. You will learn in Section 6.2 that professional practice requires you to stay inside the recommended conditions, and then to derate further from there.

6.1.3 Zone 3 — Electrical Characteristics

Electrical characteristics tables list the guaranteed minimum, typical, and maximum values for every important parameter: output voltage, quiescent current, input impedance, propagation delay, leakage current, and dozens more. The columns are labelled Min, Typ, and Max.

This is the single most important discipline in datasheet reading: **design to the Min and Max columns, never to the Typ column.** The typical value describes the median part from the manufacturer's characterisation lot. Your prototype will contain parts from many lots, spanning a temperature range and an aging curve. If your circuit only works when every component happens to hit its typical value, your circuit does not work — it occasionally works.

6.1.4 Zone 4 — Timing Diagrams

Timing diagrams specify the temporal relationships between signals: setup time, hold time, propagation delay, clock-to-output delay, and bus turn-around time. These parameters are what make the interface contracts from Chapter 5 enforceable at the component level.

Recall from Chapter 5 that the running-example sorter has a fixed timing budget of 5 ms from sensor trigger to actuator command, with zero margin. That zero-margin budget means that every component on the signal path must be verified against its timing parameters in Zone 4 before it is selected. If a motor driver's datasheet shows a command-propagation delay of 2.1 ms worst-case, and you had allocated only 2 ms for that subsystem, the component violates the interface contract — regardless of what the typical value says.

6.1.5 Application Circuits and Packaging

After you have verified all four critical zones, two additional sections become relevant. **Application circuits** show the manufacturer's recommended decoupling topology, external component values, and layout guidance. Follow these circuits as a starting point; deviating from them requires justification. The **packaging and footprint** section specifies the physical dimensions and land pattern for PCB layout. Mismatching a footprint to the physical component is one of the most common and most embarrassing prototype failures — a component that cannot be soldered to the board it was designed for.

6.2 Derating

Derating is the practice of operating a component at a fraction of its rated limit in order to reduce the probability of failure and extend service life. It is not a safety factor added out of conservatism — it is a quantified, defensible engineering decision backed by decades of field-failure data. The fundamental observation is that component failure rate increases nonlinearly as operating stress approaches the rated limit. Figure 6.2 shows the characteristic stress–failure-rate curve: failure rate is roughly flat at low stress fractions, rises steeply as the fraction approaches 1.0, and becomes essentially unbounded above the absolute maximum.

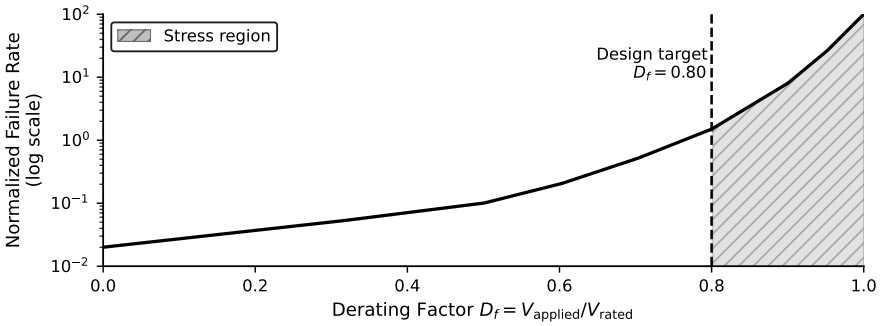

Figure 6.2: Component failure rate as a function of derating factor D_f. The curve is approximately flat below $D_f = 0.5$ and rises sharply above $D_f = 0.8$. The shaded region between $D_f = 0.8$ and $D_f = 1.0$ represents the stress zone where failure rate accelerates; the region above $D_f = 1.0$ (absolute maximum) represents the damage zone. The dashed line at $D_f = 0.80$ marks the maximum acceptable derating factor for voltage in this course.

The **derating factor** for a given parameter is defined as:

$$D_f = \frac{V_{\text{applied}}}{V_{\text{rated}}} \tag{6.1}$$

where V_{applied} is the actual operating value and V_{rated} is the component's rated limit for that parameter. Although Equation 6.1 is written in terms of voltage, the same ratio applies to current, power dissipation, and any other stress parameter.

The derating targets used in this course are:

- **Voltage (semiconductor):** $D_f \leq 0.80$
- **Voltage (capacitor):** $D_f \leq 0.75$
- **Current (semiconductor junction):** $D_f \leq 0.75$
- **Power dissipation:** $D_f \leq 0.70$
- **Junction temperature:** operate at least 25 °C below rated maximum junction temperature

These targets are conservative relative to some industry standards and are appropriate for a prototype context where accelerated testing and statistical sampling are not available.

6.2.0.0.1 Common Pitfalls.

- **Operating at absolute maximum ratings.** A component that operates at its absolute maximum voltage in a lab environment — with stable bench power, 22 °C ambient, and no vibration — will not reproduce that environment in a factory, a warehouse, or a field installation. "It worked in the lab" is not a reliability argument. The absolute maximum ratings exist to describe the damage threshold; the only correct interpretation is "never go here."

- **No second source identified at design time.** A component selected without an identified alternate forces a design respin if the primary supplier discontinues or allocates the part. The respin cost, measured in engineering hours and schedule delay, routinely exceeds the original prototype budget. See Section 6.5 for the formal second-sourcing process.

- **Accepting a component because a colleague used it before.** A colleague's previous use of a component tells you that it worked in one set of operating conditions, in one design, at one point in time. It does not tell you that the component is active in the supply chain today, that it meets your design's specific voltage and timing requirements, or that it is available in the quantity your production ramp requires. Always check lifecycle status and verify the new design's specific operating conditions against the datasheet, regardless of prior familiarity.

6.3 Operating Margin

Where derating describes how close you are operating to the component's stress limit, **operating margin** describes how much headroom remains between an actual measured value and a specification limit. The two quantities are related but distinct. Derating is a design decision you make before building; margin is a measurement you make after building or simulating, to verify that the design decision was sufficient.

Operating margin is defined as:

$$M = \frac{\text{spec limit} - \text{actual value}}{\text{spec limit}} \times 100\% \tag{6.2}$$

A positive M means the actual value is inside the specification — you have headroom. A negative M means the actual value has violated the specification — the design fails. A margin of zero means the design is on the edge: any variation in temperature, supply voltage, component tolerance, or aging will push it into failure.

You should target a minimum margin of 20% on timing parameters and 25% on voltage parameters for a first prototype. These targets give you room to absorb the variation you will discover when you measure actual hardware for the first time.

6.4 Component Lifecycle Status

Every active component has a **lifecycle status** that describes where it sits in the manufacturer's product line plan. Treat lifecycle status as a risk register entry, not a purchasing footnote. Selecting a component without verifying its lifecycle status is equivalent to building a feature on a third-party API without checking its deprecation schedule.

Figure 6.3 shows the standard lifecycle timeline with risk annotations for each phase.

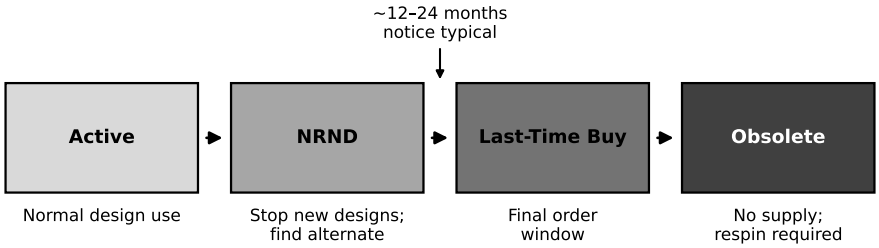

Figure 6.3: Component lifecycle timeline. Phases from left to right: Introduction, Growth, Mature (Active), NRND, Last-Time Buy, and Obsolete. Risk annotations indicate the design action appropriate at each phase. The bracket labelled "Design Window" spans Introduction through Mature; the bracket labelled "Risk Window" spans NRND through Last-Time Buy.

The four statuses you will encounter on distributor websites and manufacturer product pages are:

- **Active**: The component is in full production. The manufacturer is actively taking orders, maintaining inventory, and providing technical support. This is the only lifecycle status that is acceptable for a new design without a written risk exception.
- **NRND (Not Recommended for New Designs)**: The manufacturer is signalling that the component is approaching end-of-life. NRND does not mean "unavailable today" — it means "do not start a new design on this component." The typical window between NRND announcement and end-of-life is 12–36 months, but this varies widely. If a component you are considering is in NRND status, you must find an alternative before design freeze. Specifically, two actions are required: (1) identify an Active-status replacement that meets all parametric and footprint criteria; and (2) record the NRND status as a risk register entry with a resolution deadline.
- **Last-Time Buy (LTB)**: The manufacturer has announced a final production run. Orders must be placed by a specified date. After that date the component will never be manufactured again. An LTB component should never be selected for a new design unless a complete lifetime supply can be purchased, inspected, and stored — a practice reserved for high-reliability aerospace and medical applications, not prototypes.
- **Obsolete**: The component is no longer available from the manufacturer. It may still appear on the secondary market (broker stock), but secondary-market parts carry risks of counterfeiting, improper storage, and undocumented handling that are unacceptable for a professional design.

6.4.0.0.1 In Practice. A team at a previous cohort built a working prototype on an MCU that was listed as Active at the time of component selection. Six months after design freeze — before the first production run — the manufacturer moved that MCU to NRND status. The team did not have a second source identified, because second-sourcing had been deferred. The redesign to a compatible MCU required new firmware validation, a new schematic review, and a new round of regulatory EMC testing. The total cost of the redesign exceeded the original prototype budget. Lifecycle status must be verified at component selection, not at production planning.

6.5 Second-Sourcing Strategy

Once you have confirmed Active status for every critical-path component, the next obligation is to ensure you are not dependent on a single supplier.

A **second source** is an alternative component that can substitute for the primary selection without requiring a schematic change, a firmware change, or a requalification test. Identifying a second source at component selection time costs roughly two hours of engineering effort. Scrambling to find one after a supply disruption costs weeks.

A viable second source must satisfy three criteria simultaneously. Satisfying two out of three is not sufficient:

1. **Parametric equivalence**: The alternate component must meet or exceed every specification that your design depends on — not just the headline spec, but every parameter you extracted from Zones 1 through 4. A motor driver with the same current rating but a slower PWM frequency response is not parametrically equivalent if your control loop depends on the frequency response.

2. **Footprint compatibility**: The alternate component must be pin-compatible with the primary on the PCB land pattern. A component with identical electrical specifications but a different package (e.g., QFN-32 versus TQFP-32) requires a layout change, which in a professional context triggers a new design revision and new board fabrication. For a prototype, this is a measurable cost.

3. **Lifecycle status**: The alternate component must itself be in Active status. An alternate that is NRND or LTB provides no protection against the supply disruption you are trying to mitigate.

6.5.0.0.1 In Practice. A team locked their BOM fourteen weeks before the scheduled system integration test. Three weeks later, the primary MCU lead time extended from stock to 14 weeks due to an industry allocation event. Because the team had not identified a second source, they were unable to substitute an equivalent part. The project schedule slipped by one full milestone — losing a Stage-Gate review cycle — while the team waited for allocation to clear. Had a second source been identified at BOM lock, the swap could have been made within two days.

6.6 Supply Chain Risk

Component selection is not complete until you have assessed the supply chain risk of every part on the critical path. **Supply chain risk** encompasses three main categories.

Single-source components are manufactured by exactly one supplier with no compatible alternate. They represent the highest supply chain risk in a BOM. When you identify a single-source component, it becomes a risk register entry (see Chapter 4) with a severity proportional to how difficult it would be to re-architect the design around a different part.

Lead time is the calendar time between placing an order and receiving parts. For most standard components, lead time is one to five weeks from distributor stock. For certain microcontrollers, power management ICs, and RF transceivers, lead time from the factory can reach 26–52 weeks during periods of high demand. You must verify the lead time of every critical-path component at the time of BOM lock, not at the time of purchase order. Lead times reported on distributor websites are real-time and can change daily.

Allocation periods occur when a component is in higher demand than the manufacturer's capacity can supply. During allocation, distributors receive fractional fills on their purchase orders, and small customers (including most prototype projects) may receive nothing. Allocation periods for popular MCU families have historically lasted 12–24 months. The only reliable mitigation is to have a qualified second source with independent supply chain.

6.7 MTBF and Reliability Basics

Reliability is the probability that a system performs its required function for a specified time under specified conditions. Every component you add to a system adds a probability of failure. Understanding how to estimate the aggregate reliability of a multi-component path allows you to make informed decisions about which components to derate more aggressively, which to make redundant, and which to flag as reliability risks in the Gate 3 review.

6.7.1 Failure Rate and FIT

Component reliability is quantified by the **failure rate** λ, defined as the expected number of failures per unit time. The industry-standard unit is the **FIT** (Failures In Time), where:

$$1 \text{ FIT} = 1 \text{ failure per } 10^9 \text{ hours}$$

A component with $\lambda = 200$ FIT is expected to produce one failure in 5×10^6 hours of aggregate device operation. FIT values are published by manufacturers in reliability reports and can also be estimated using MIL-HDBK-217 or Telcordia SR-332 models, which account for operating temperature and applied stress.

6.7.2 Series System Reliability

In a series reliability model, every component must function for the system to function. This is the correct model for a critical signal path: if any component in the path fails, the path fails. For a series system of n components, the system failure rate and system MTBF are:

$$\lambda_{\text{sys}} = \sum_{i} \lambda_i, \qquad \text{MTBF}_{\text{sys}} = \frac{1}{\lambda_{\text{sys}}} \tag{6.3}$$

where λ_i is the failure rate of the i-th component and all rates are in consistent units (FITs or failures per hour). Note that adding components to a series path always *decreases* system MTBF. This is the quantitative justification for minimising component count on reliability-critical paths.

6.7.3 Parallel Redundancy

When two or more components perform the same function in parallel — and the system succeeds if at least one survives — the reliability model changes. For a parallel set of n components each with reliability R_i, the system reliability is:

$$R_{\text{sys}} = 1 - \prod_{i} (1 - R_i) \tag{6.4}$$

where R_i is the probability that component i has not failed over the mission time. Parallel redundancy is used in safety-critical subsystems — power supplies, communication links, and actuator drivers where a single failure must not halt the system. It is not a substitute for derating; a redundant pair of poorly derated components has a higher aggregate failure rate than a single well-derated component.

6.8 Worked Example

6.8.1 Part A — Derating and Operating Margin for an MCU

The running-example sorter uses an STM32-class MCU with the following datasheet parameters:

- Rated supply voltage: $V_{\text{rated}} = 3.6$ V
- Applied supply voltage: $V_{\text{applied}} = 3.3$ V
- Ambient temperature: $T_{\text{ambient}} = 85$ C
- Timing parameter spec limit (setup time): $t_{\text{spec}} = 10$ ns
- Timing parameter measured actual value: $t_{\text{actual}} = 3.2$ ns

Derating factor for supply voltage (Equation 6.1):

$$D_f = \frac{V_{\text{applied}}}{V_{\text{rated}}} = \frac{3.3\ \text{V}}{3.6\ \text{V}} = 0.917$$

Comparing to the derating target of $D_f \leq 0.80$ stated in Section 6.2: $D_f = 0.917$ **exceeds the target**. The MCU supply voltage does not comply with the course derating rule. This finding must be recorded as a risk item.

6.8.1.0.1 Design action required. $D_f = 0.917 > 0.80$ fails the course derating target. This entry **must be flagged** in the component selection table and must be resolved — by selecting a component with $V_{\text{rated}} \geq 4.1$ V or by lowering V_{applied} to comply — before Gate 3 will accept the deliverable.

The design team must either reduce V_{applied} (for example, reducing V_{applied} to 2.88 V gives $D_f = 2.88/3.6 = 0.80$ (the boundary), or to 2.7 V gives $D_f = 2.7/3.6 = 0.75$ (compliant)) or select a component with a higher rated supply voltage.

Operating margin on the timing parameter (Equation 6.2):

$$M = \frac{10\ \text{ns} - 3.2\ \text{ns}}{10\ \text{ns}} \times 100\% = \frac{6.8\ \text{ns}}{10\ \text{ns}} \times 100\% = 68\%$$

A margin of 68% on the setup time is well above the 20% target for timing parameters. The setup time is not the constraint on this design. However, note that the 5 ms end-to-end timing budget from Chapter 5 has zero margin at the system level; the comfortable margin on this individual timing parameter does not relieve the pressure on the full pipeline.

6.8.2 Part B — System MTBF for the Actuator Interface Path

The actuator interface path in the running-example sorter consists of three components in series: the MCU, a level-shifter, and a motor driver. Their failure rates are:

Component	Role	λ (FIT)
MCU (STM32-class)	Firmware execution, actuator command	200
Level-shifter	3.3 V to 5 V translation	50
Motor driver	Sort actuator PWM control	400

System failure rate (Equation 6.3):

$$\lambda_{\text{sys}} = 200 + 50 + 400 = 650\ \text{FIT}$$

System MTBF in hours:

$$\text{MTBF}_{\text{sys}} = \frac{1}{\lambda_{\text{sys}}} = \frac{10^9\ \text{hours}}{650} \approx 1{,}538{,}462\ \text{hours}$$

Expressing MTBF in years (dividing by 8760 hours per year):

$$\text{MTBF}_{\text{sys}} = \frac{1{,}538{,}462}{8{,}760} \approx 175.6 \text{ years}$$

The motor driver dominates the system failure rate with 62% of the total λ_{sys}. If the project reliability target requires a higher MTBF, the motor driver is the first component to investigate for a lower-FIT alternative. The level-shifter contributes only 8% of total failure rate and is not the priority for improvement.

6.9 Try It Yourself

The following problem uses different numbers from the worked example. Work through each part before consulting the solutions in the appendix.

A 5 V linear regulator has the following datasheet and measurement data:

- Rated supply voltage: $V_{\text{rated}} = 5.5$ V
- Applied supply voltage: $V_{\text{applied}} = 4.8$ V
- Timing parameter spec limit (output-settle time): $t_{\text{spec}} = 20$ µs
- Timing parameter measured actual value: $t_{\text{actual}} = 14$ µs

Two components on the sensor front-end critical path:

- Sensor: $\lambda_1 = 120$ FIT
- Analog front-end IC: $\lambda_2 = 310$ FIT

(a) Compute D_f for the supply voltage using Equation 6.1. Is the result within the $D_f \leq 0.80$ voltage derating target?

(b) Compute the operating margin M on the timing parameter using Equation 6.2.

(c) Compute λ_{sys} and MTBF_{sys} for the two-component series path using Equation 6.3. Express MTBF in years (divide hours by 8,760).

(d) The sensor is currently listed as NRND by its manufacturer. State the two actions defined in Section 6.4 that the team must complete before the next design review, and give the lifecycle status the replacement component must hold.

6.10 Review Questions

Q1. Explain the difference between "absolute maximum ratings" and "recommended operating conditions" in a datasheet. Why is operating at the absolute maximum ratings a professional mistake, even if the component does not fail immediately?

Q2. A component's lifecycle status changes from "Active" to "NRND." State what NRND means, the typical time window before end-of-life, and the two responses a design team should take within that window.

Q3. A capacitor is rated at $V_{\text{rated}} = 16$ V. The circuit applies $V_{\text{applied}} = 10$ V.

(a) Compute D_f using Equation 6.1.

(b) The derating rule for capacitors is $D_f \leq 0.75$. Does this selection comply?

(c) If V_{applied} rises to 13 V under transient conditions, recompute D_f and state whether the component should be replaced.

Q4. A four-component power path has failure rates $\lambda_1 = 80$ FIT, $\lambda_2 = 150$ FIT, $\lambda_3 = 220$ FIT, $\lambda_4 = 90$ FIT.

(a) Compute λ_{sys} using Equation 6.3.

(b) Compute MTBF_{sys} in hours.

(c) Convert to years (8,760 hours per year). Round to one decimal place.

(d) The team adds a redundant parallel path for component 3 with an identical $\lambda_3 = 220$ FIT. Using Equation 6.4, compute R_{sys} for that redundant pair over a 10,000-hour mission time, where $R_i = e^{-\lambda_i t}$ with λ_i in failures per hour (note: 1 FIT $= 10^{-9}$ failures/hour). State in one sentence whether the parallel pair improves or degrades reliability compared with a single component over the same mission time.

Q5. Explain why a "second source" must satisfy three criteria — parametric equivalence, footprint compatibility, and lifecycle status — and give an example of a second source that satisfies two of the three but fails on one.

Portfolio Entry

The deliverable for this chapter is the **component selection table** for the critical path of your prototype. This table is a mandatory input to Gate 3. Complete one row per critical-path component using the template below.

Part number	Supplier	Lifecycle status	Derating factor D_f (V)	Margin M (%)	Alternate part number	MTBF_{sys} (years)

Column definitions:

- **Lifecycle status**: Active, NRND, LTB, or Obsolete — verified on the manufacturer's product page on the day of submission.
- D_f **(V)**: Voltage derating factor computed using Equation 6.1. Flag any value > 0.80 in red.

J. F. N. Salik,

- **Margin** M **(%)**: Operating margin on the tightest timing or voltage parameter in the design, computed using Equation 6.2. Flag any value $< 20\%$ in red.
- **Alternate part number**: A second source that satisfies all three criteria from Section 6.5.
- $\text{MTBF}_{\textbf{sys}}$ **(years)**: Computed using Equation 6.3 for the full critical path.

6.11 Summary

This chapter gave you the tools to fill the architecture blocks from Chapter 5 with verified components. You can now navigate a datasheet systematically, extracting the four critical zones rather than reading from cover to cover. You can apply derating factors and compute operating margins to quantify the headroom between your design's operating point and the component's limits. You can assess lifecycle status and supply chain risk as engineering variables rather than purchasing footnotes, and identify a second source that genuinely substitutes for — not just superficially resembles — the primary selection. Finally, you can estimate the MTBF of a series reliability path and use that estimate to identify which component on the critical path deserves the most engineering attention.

These competencies together constitute **datasheet literacy**: the ability to translate a manufacturer's specification document into a set of verified, margin-positive design decisions. A design built on verified components with documented margins and identified alternates is a design that can survive the surprises — supply disruptions, temperature extremes, component lot variation — that the prototype bench cannot anticipate.

Chapter 6 produced the component selection table — part numbers, derating factors, lifecycle status, alternates, and system MTBF — that locks the critical path into Gate 3. Chapter 7 takes that table and examines how to design the assembly around those verified components so that manufacturing, testing, and serviceability are built in from the start.

CHAPTER 7

DESIGN FOR X

7.0.0.0.1 Where this fits. This chapter operates at Stage 3 (Development) of the Stage-Gate model. Design-for-X decisions made here determine 70–80% of manufacturing cost and assembly quality before a single prototype board is soldered. Catching a DFX violation at Stage 3 costs a design revision; the same violation found at Stage 4 testing costs a board respin, a BOM change, and a schedule slip that threatens Gate 4.

Chapter 6 selected the components that populate the architecture. That fixed the bill of materials — but not yet how the assembly, test procedure, and reliability target will be designed into the hardware from the start. This chapter supplies that discipline.

1. Apply DFM principles to reduce unit cost for PCB and 3D-printed parts.
2. Compute DFA assembly efficiency η_A and identify parts that fail the keep/eliminate test.
3. Design observability and controllability into a prototype using DFT principles: test points, debug ports, and built-in self-test.
4. Connect the derating and lifecycle decisions from Chapter 6 to DFR design margin and thermal management.

7.1 What DFX Means

Design for X is the discipline of designing with the downstream process in mind. The "X" is a placeholder: it stands for Manufacturability (DFM), Assembly (DFA), Test (DFT), or Reliability (DFR), and the four disciplines interact.

Figure 7.1 shows the four domains and their relationship to prototype design.

A common misconception is that DFX is a checklist run at design-complete. It is not. DFX is a continuous design constraint: each time you choose a component, a trace width, a fastener, or a PCB stack-up, you make a DFX decision. If you make it consciously, using the principles in this chapter, you eliminate whole categories

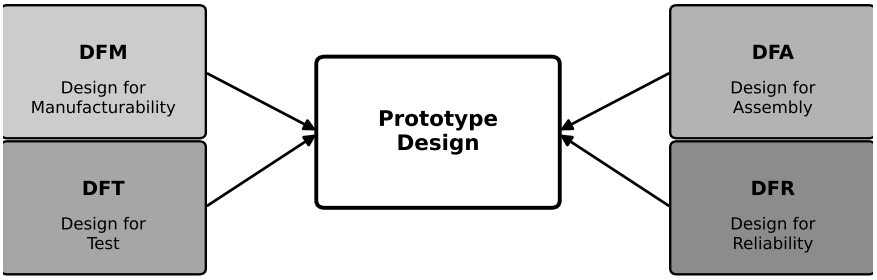

Figure 7.1: The four DFX disciplines feed into every prototype design decision. Optimising one domain in isolation can degrade another; the goal is to resolve conflicts at Stage 3, before they become build failures.

of downstream failure. If you make it by default, you pay for it at assembly, test, or field return.

7.2 Design for Manufacturability

Design for Manufacturability (DFM) means choosing processes, tolerances, and geometries that standard equipment can produce reliably at your target volume and cost.

7.2.1 PCB DFM

The following rules apply to every prototype PCB submitted to a contract manufacturer:

- **Trace width and spacing.** Stay at or above the fabricator's minimum design rule. A 0.1 mm trace on a board specified for 0.15 mm minimum will be rejected or reflowed out of spec.
- **Via size.** Blind and buried vias cost extra; through-hole vias are standard. Choose via drill sizes that the fabricator's standard process supports.
- **Panelisation.** Design the board to panel at standard sizes so the fabricator can run multiple boards per panel without custom tooling.
- **Solder mask and silkscreen.** Solder mask openings must clear pad edges by the fabricator's specified clearance. Silkscreen must not overlap pads — solder will not adhere through ink.
- **Component courtyard clearances.** Ensure no courtyard overlaps; the pick-and-place machine needs clear landing zones for each nozzle.

7.2.2 3D-Print DFM

The running example uses a 3D-printed housing. FDM (fused deposition modelling) imposes its own DFM rules:

- **Minimum wall thickness.** FDM walls thinner than 1.2 mm (two perimeters at 0.4 mm nozzle) are weak and prone to delamination.
- **Overhang angle.** Overhangs beyond 45° require support material, which must be removed and damages fine features. Design with 45° chamfers instead of 60° ledges.
- **Part orientation.** Orient the part so that the functional mating surfaces (connector openings, PCB standoffs) are printed in the XY plane, not the Z direction, where layer adhesion is weaker.
- **Support-free internal channels.** Route cable channels as tear-drop cross-sections rather than round bores; they print without support and are stronger.

7.2.2.0.1 In Practice. A 3D-printed housing for the running example was designed with 60° overhangs on the internal PCB shelf. Support material filled the shelf cavity; removing it with pliers cracked the shelf on two of six printed units. Redesigning to a 45° chamfer eliminated the support requirement entirely, and all subsequent units printed clean.

7.3 Design for Assembly

Design for Assembly (DFA) minimises the number of parts and assembly operations, reducing labour cost and assembly error rate.

7.3.1 The Boothroyd–Dewhurst Criterion

The **Boothroyd–Dewhurst keep/eliminate test** asks three questions for each part:

1. Does the part move relative to all other parts during operation?
2. Must the part be made of a different material from adjacent parts?
3. Must the part be separate to allow assembly or disassembly?

A part that answers *no* to all three questions is a candidate for elimination — it can be integrated into an adjacent part. $N_{\min}$ is the count of parts that answer *yes* to at least one question.

7.3.2 Assembly Efficiency

Assembly efficiency η_A measures how close a design is to the theoretical minimum:

$$\eta_A = \frac{N_{\min} \cdot t_{\text{ideal}}}{t_{\text{actual}}} \tag{7.1}$$

where $t_{\text{ideal}} = 3$ s is the Boothroyd–Dewhurst standard ideal assembly time per part, and t_{actual} is the measured total assembly time in seconds. A higher η_A means the

design is closer to the minimum. Typical values for a first-pass design are 5–15 %; well-optimised assemblies reach 40–60 %.

The cost saving from a DFA redesign is:

$$\Delta C \approx \Delta N_{\text{parts}} \cdot \bar{c}_{\text{part}} + \Delta N_{\text{ops}} \cdot \bar{c}_{\text{op}} \tag{7.2}$$

where $\bar{c}_{\text{part}}$ is the average part cost saved per eliminated part and $\bar{c}_{\text{op}}$ is the average cost per assembly operation second (typically \$0.08–\$0.15/s at contract assembly rates in CAD).

7.4 Design for Test

Design for Test (DFT) builds observability and controllability into the hardware at design time. **Observability** means you can measure a signal's actual value. **Controllability** means you can force a signal to a known state to isolate a fault.

Figure 7.2 shows the three DFT features designed into the running-example MCU board: test points, a UART debug header, and an SWD/JTAG programming header.

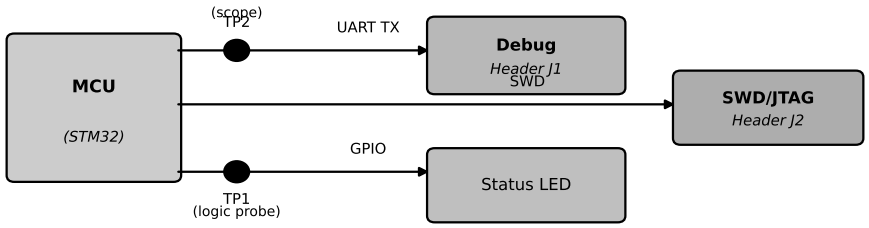

Figure 7.2: DFT features on the running-example MCU board: test points TP1 and TP2 accessible by a logic probe and scope respectively; a UART debug header (J1) for firmware printf output; an SWD/JTAG header (J2) for breakpoint debugging and firmware flashing.

7.4.1 Test Points

Test points are labelled, exposed pads or through-hole vias placed on every signal that a technician might need to probe. Minimum rules:

- One test point per power rail, placed between the regulator output and the first load capacitor.
- One test point on every high-frequency or safety-critical signal.
- Test points must be at least 2.54 mm from any component body to allow a standard probe tip to land.

7.4.2 Debug Ports and BIST

A **UART debug port** connected to a header allows the firmware to stream printf output during development without stopping execution. This exposes firmware state

non-intrusively — the single most valuable DFT feature for firmware-heavy prototypes.

Built-in self-test (BIST) is firmware that runs a self-check sequence at power-on, toggles each output GPIO, reads back each ADC rail, and reports pass/fail over the debug port before the main application loop starts. BIST catches bring-up faults in under 30 seconds.

7.4.3 Defect Coverage

Once test procedures are running, track defect density with **DPMO** (defects per million opportunities):

$$\text{DPMO} = \frac{D}{N \cdot O} \times 10^6 \tag{7.3}$$

where D is the number of defects observed, N is the number of units tested, and O is the number of test opportunities per unit. Adding more test points increases O, which lowers DPMO for the same physical defect count — this is the quantitative argument for investing in DFT.

7.4.3.0.1 In Practice. An early prototype of the running-example sorter board had no test points and no UART debug port. When the sort classifier produced incorrect results during bring-up, the validation team spent three days probing unmarked pads with a clip lead to distinguish a firmware logic error from a hardware signal-integrity problem. A UART debug port would have printed the classifier output at each step and isolated the fault in under 20 minutes. The three days were not recoverable from the project schedule.

7.5 Design for Reliability

Design for Reliability (DFR) translates the FMEA risk register from Chapter 4 and the derating margins from Chapter 6 into concrete design choices.

7.5.1 Derating and FMEA Margin

The derating factor D_f from Chapter 6 is a DFR parameter: a component operating at $D_f = 0.70$ has significantly longer expected life than one at $D_f = 0.90$. When the FMEA assigns Severity $S \geq 8$ to a failure mode, the DFR response is to increase the design margin — lower D_f, specify a higher-rated part, or add protective circuitry.

7.5.2 Redundancy

For failure modes that are both high-severity and high-occurrence (RPN > 80 from Chapter 4), **redundancy** may be justified: two components in parallel, where either one suffices, lower the effective failure rate. Use Chapter 6's parallel-reliability equation to quantify the improvement before committing board area.

7.5.3 Thermal Management

Every power-dissipating component has a maximum junction temperature $T_{j,\max}$. The thermal budget is:

$$T_j = T_{\text{ambient}} + P_D \cdot \theta_{JA}$$

where P_D is power dissipation and θ_{JA} is junction-to-ambient thermal resistance from the datasheet. Verify $T_j < T_{j,\max}$ at maximum operating ambient temperature before finalising component placement; hotspot components may need copper pours, thermal vias, or forced airflow.

7.6 DFX Trade-offs in Practice

The four disciplines interact and sometimes conflict. A DFM optimisation (reducing layer count to save fabrication cost) may eliminate the reference plane needed for signal integrity (a DFR regression). A DFA optimisation (integrating two parts into one moulded piece) may block the disassembly access required for test (a DFT regression).

The resolution principle is: *optimise for the dominant cost driver at your production volume*.

- At **prototype volume** (1–10 units): labour time dominates. Prioritise DFT — instrument every signal, use every debug port. DFM is secondary; a few extra fabrication days matter less than a three-day firmware debug.
- At **low-volume production** (100–1000 units): assembly cost dominates. Prioritise DFA — eliminate parts, standardise fasteners, design for one-directional insertion.
- At **high volume** (10 000+): material cost dominates. Prioritise DFM — optimise BOM cost, panel efficiency, and process yield.

7.6.0.0.1 Common Pitfalls. DFX applied as a post-design review checklist rather than a live design constraint will always find violations too late to fix cheaply. Every design decision from component selection onward is a DFX decision; treat it that way.

7.6.0.0.2 Common Pitfalls. Adding test points "if there is room" is not DFT. Test points designed in as required features, with reserved courtyard space and silkscreen labels, are DFT. Three strategically placed test points found a PCB fabrication defect in the running example during bring-up; three test points added as afterthoughts on the same board revealed nothing because they landed on signals that were already observable with a multimeter.

7.7 Worked Example

The running-example sorter board has an eight-part PCB subassembly. Applying the Boothroyd–Dewhurst test, three parts must be kept ($N_{\min} = 3$): the MCU (different material and function), the motor-driver IC (different material and function), and the connector housing (must be separate for mating access). The remaining five parts — three identical M2 standoffs, one wire-tie anchor, and one redundant power-rail capacitor — answer *no* to all three questions and are candidates for elimination.

Part A — Assembly Efficiency

Before redesign: $N_{\text{parts}} = 8$, $t_{\text{actual}} = 120$ s.

Using Equation 7.1:

$$\eta_A = \frac{N_{\min} \cdot t_{\text{ideal}}}{t_{\text{actual}}} = \frac{3 \times 3}{120} = \frac{9}{120} = 0.075 \quad (7.5\,\%)$$

Two parts are eliminated in the redesign (the three standoffs become integrated PCB snap features; the wire-tie anchor is removed by routing the cable through a slot moulded into the housing). The redesign reduces t_{actual} to 80 s.

$$\eta_A' = \frac{3 \times 3}{80} = \frac{9}{80} = 0.1125 \quad (11.25\,\%)$$

Figure 7.3 shows the before/after comparison.

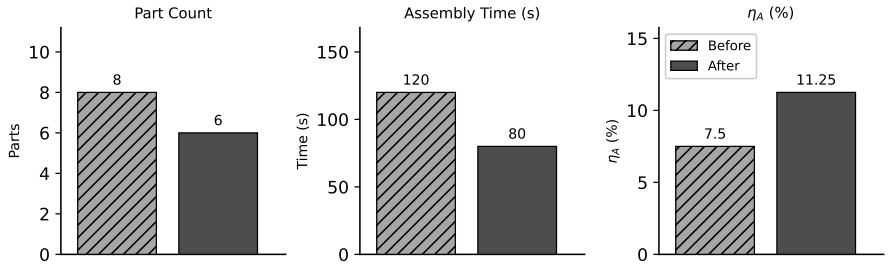

Figure 7.3: Part count, assembly time, and η_A before and after the DFA redesign. Eliminating two parts reduces assembly time by 33 % and raises η_A from 7.5 % to 11.25 %.

Using Equation 7.2 with $\bar{c}_{\text{op}} = \$0.10$/s and a production run of 500 units:

$$\Delta C = (120 - 80)\,\text{s} \times \$0.10/\text{s} \times 500 = 40 \times \$0.10 \times 500 = \$2{,}000 \text{ CAD}$$

Part B — Defect Coverage (DPMO)

A test run of 100 units of the sorter PCB subassembly produces 3 defects. Each unit has $O = 12$ test opportunities (12 explicitly designed test points).

Using Equation 7.3:

$$\text{DPMO} = \frac{D}{N \cdot O} \times 10^6 = \frac{3}{100 \times 12} \times 10^6 = \frac{3}{1200} \times 10^6 = 2{,}500 \text{ DPMO}$$

At this rate, a production run of 10 000 units would yield:

$$10{,}000 \times 12 \times \frac{2{,}500}{10^6} = 300 \text{ defective opportunities}$$

Interpreted: 300 test points across all units will fail; each failure triggers a rework or rejection.

Try It Yourself

A sensor breakout board subassembly has 6 parts, $t_{\text{actual}} = 90$ s, and $N_{\min} = 2$. The team identifies one candidate for elimination (a redundant tie-point bracket), reducing t_{actual} to 65 s. A test run of 50 units reveals 2 defects; each unit has 8 test opportunities.

(a) Compute η_A before the elimination using Equation 7.1.

(b) Compute η_A after the elimination.

(c) Compute the assembly cost saving per unit at 200 units using $\bar{c}_{\text{op}} = \$0.10$/s (Equation 7.2).

(d) Compute the DPMO using Equation 7.3.

(No answer key provided.)

Review Questions

1. **(Conceptual)** State the four DFX disciplines covered in this chapter. For each, give one specific design decision from the running-example sorter prototype that falls within that discipline.
2. **(Conceptual)** Explain the Boothroyd–Dewhurst three-question keep/eliminate test. Give one example of a component that passes all three criteria (must be kept) and one that fails all three (candidate for elimination) from the running example.
3. **(Applied — calculation)** A subassembly has 10 parts and $t_{\text{actual}} = 150$ s. The Boothroyd–Dewhurst test identifies $N_{\min} = 4$.
   (a) Compute η_A using Equation 7.1.
   (b) A redesign eliminates 3 parts and reduces t_{actual} to 100 s. Compute the new η_A.
   (c) Compute the assembly cost saving per unit at 300 units, using $\bar{c}_{\text{op}} = \$0.12$/s.

4. **(Applied — multi-step)** A production run of 200 units yields 5 defects, with $O = 15$ test opportunities per unit.

   (a) Compute DPMO using Equation 7.3.

   (b) At this rate, how many defects are expected in a run of 50 000 units?

   (c) The team adds 3 additional test points per unit, raising O from 15 to 18. The defect count stays at 5 per 200 units. Recompute DPMO and explain in one sentence why DPMO decreased even though the physical defect count did not change.

5. **(Conceptual)** Explain why DFM and DFA can conflict. Give one example where reducing material cost (DFM) increases assembly time (DFA regression). State how to decide which discipline to prioritise at prototype volume versus production volume.

Portfolio Entry

Your DFX review checklist for the prototype:

1. **DFM** — three PCB or mechanical decisions that use standard processes: trace width and via size within fabricator rules; 3D-print features within 45° overhang limit; panel size within standard sheet.

2. **DFA** — η_A computed for the most complex subassembly; one elimination candidate identified and either eliminated or logged with justification for retention.

3. **DFT** — three observability features explicitly designed in: test-point locations and labelled signals; UART debug port with header and silkscreen designation; BIST firmware routine described.

4. **DFR** — derating margin from Chapter 6 confirmed against the FMEA Severity ratings from Chapter 4; any $D_f > 0.80$ entry resolved or logged as a risk item for Gate 3.

Chapter Summary

Four disciplines now constrain your prototype design before the first component is soldered. DFM aligns your PCB layout and 3D geometry with what standard processes can produce reliably. DFA uses the Boothroyd–Dewhurst criterion and η_A (Equation 7.1) to quantify and reduce assembly labour. DFT instruments every critical signal with test points, debug ports, and BIST so that bring-up faults are found in minutes rather than days. DFR translates FMEA severity ratings and derating margins into design choices that protect reliability before the hardware is built.

The DFX checklist you produced is a concrete Stage 3 deliverable. Gate 3 reviewers will look for evidence that cost, assembly, and reliability were designed in, not discovered during validation.

Chapter 7 disciplined the design process. Chapter 8 now introduces AI tools that can accelerate design work — and explains how to use them without compromising the engineer's accountability for every output.

CHAPTER 8

AI IN PROTOTYPING: CAPABILITY & RISK

8.0.0.0.1 Where this fits. This chapter operates at Stage 3 (Development) of the Stage-Gate model. AI tools are increasingly present inside Stage 3 design work — generating firmware scaffolding, suggesting component values, exploring design alternatives, and drafting documentation. Every AI output that enters your design must be verified before it becomes Gate 3 evidence; this chapter gives you the framework to decide what to verify, how rigorously, and how to document it.

8.0.0.0.2 Learning Objectives. After completing this chapter, you will be able to:

1. Identify three categories of design task where AI tools provide genuine value in the prototyping workflow.

2. Compute the expected cost of error for an AI-generated output and apply the verification decision rule to determine whether independent verification is required before incorporating the output into a design.

3. Assess the IP and data-leakage risk of using a public AI tool with proprietary design data and identify at least two specific categories of Stage 3 information that must not be submitted to a public service without checking the provider's retention policy.

4. Document AI use in a reproducible, traceable format suitable for a Gate 3 submission.

Chapter 7 delivered a component-verified, assembly-optimised, test-instrumented schematic package ready for Gate 3 review — produced by applying four DFX frameworks to the Stage 3 design. It did not address the AI tools many teams reach for inside that same stage to accelerate the work. This chapter provides the framework for using those tools without compromising the integrity of the design package.

8.1 Where AI Helps

AI-assisted design tools — large-language models, code-completion engines, and component-suggestion services — can reduce the time spent on routine, well-defined sub-tasks within Stage 3. Three categories of task account for most of the practical value in a prototyping workflow.

8.1.1 Code and Firmware Scaffolding

Writing peripheral initialisation code, interrupt handlers, communication protocol wrappers, and RTOS task skeletons is repetitive and follows patterns that are well-represented in publicly available firmware. An AI tool can generate a working first draft of a UART driver, an I^2C read/write wrapper, or a timer-interrupt handler in seconds.

The value is not that the output is correct by default — it often is not, for reasons discussed in Section 8.3. The value is that a plausible first draft is faster to correct than to write from scratch, and it forces the engineer to think through the design at the code level earlier in the process.

For the running-example sorter project, AI-generated firmware scaffolding is useful for the sort-accuracy inference pipeline: the AI can produce a skeleton that reads sensor data, formats it for the inference engine, and routes the classification result to the conveyor control loop. The engineer then verifies each section against the MCU datasheet and the inference engine's API specification.

8.1.2 Component Value Suggestion

Selecting passive component values — pull-up and pull-down resistors, decoupling capacitors, RC filter time constants — requires applying well-known formulas to circuit-specific parameters. An AI tool can propose a value given a natural-language description of the circuit context ("10 kHz I^2C bus, 400 pF estimated bus capacitance").

The suggestion reduces search time. It does not replace the engineer's obligation to verify the value against the datasheet application note and, where relevant, to bench-test the result. Section 8.4 establishes when bench verification is required.

8.1.3 Design Alternatives and Documentation Drafts

AI tools can enumerate design alternatives quickly: given a power-supply topology requirement, they can list candidate topologies with broad trade-offs. This is useful for early-stage exploration, where breadth matters more than depth. The engineer evaluates the shortlist using the selection criteria from Chapter 6, not the AI's ranking.

AI tools also accelerate documentation drafts: block-diagram descriptions, BOM notes, test-procedure outlines, and design-rationale summaries can all be drafted

from a structured prompt and then edited for accuracy and completeness. Documentation drafted by AI must be reviewed against the actual design before submission; Chapter 9 will show why incomplete or inaccurate safety documentation has consequences beyond a Gate 3 review.

8.2 Accelerator, Not Autonomous Designer

One principle governs all AI-assisted design work: **the engineer is accountable for every output that enters the design**, regardless of how that output was produced.

An AI tool has no professional licence, no liability, and no understanding of your specific circuit, budget, safety obligations, or regulatory context. It produces plausible text and code based on statistical patterns in its training data. When that output happens to be correct, it saves time. When it is wrong — and it will be wrong, in ways described in Section 8.3 — the engineer who accepted it without verification is responsible for the consequence.

This is not a philosophical position; it has direct implications for Gate 3 evidence. A Gate 3 review asks: "How do you know this design meets specification?" "The AI said so" is not an answer that survives scrutiny. "The AI suggested it; we verified it by *(method)*; the result was *(outcome)*" is.

Design acceleration is the correct mental model for AI in Stage 3. An AI tool does not change what must be verified or who must verify it; it changes how long the first draft takes to produce.

8.3 Hallucination in an Engineering Context

AI hallucination is a specific failure mode distinct from a simple factual error. A factual error is a wrong answer to a question that has a right answer in the AI's training data — the AI retrieves the wrong datum. A hallucination is a confidently stated answer that has no factual basis at all: the AI generates a plausible-sounding output that does not correspond to any real specification, component, or physical relationship.

In an engineering context, hallucinations are more dangerous than factual errors for one reason: *they look right*. A factual error — a resistor value that is obviously wrong for the voltage level — is caught by inspection. A hallucination — a pull-up resistor value that is plausible for many buses but wrong for this specific bus speed and capacitive load — passes visual inspection and may even pass a basic functional test, failing only under the exact operating conditions of your application.

8.3.1 The Human-in-the-Loop Verification Cycle

The response to hallucination risk is a structured **human-in-the-loop (HITL) verification cycle**. Figure 8.1 shows the cycle.

The cycle has four steps:

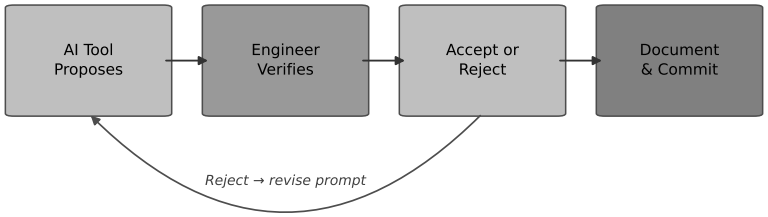

Figure 8.1: The human-in-the-loop verification cycle for AI-generated design outputs. The engineer generates a candidate output using an AI tool, applies an independent verification method (datasheet check, simulation, bench test, or first-principles calculation), and either accepts the output and logs it, or rejects it and either regenerates or derives the value manually. The cycle is not optional; it applies to every AI output that enters the design.

1. **Generate.** Prompt the AI tool with enough context to produce a specific, actionable output. Vague prompts produce vague outputs; vague outputs are harder to verify.

2. **Estimate risk.** Assign a probability of error p_{err} and a consequence cost $C_{\text{consequence}}$ to the output. Section 8.4 shows how to use these to decide verification depth.

3. **Verify independently.** Apply a verification method that is independent of the AI tool: datasheet application note, SPICE simulation, analytical calculation, or bench measurement.

4. **Accept or reject and log.** Record what the AI produced, how it was verified, and the result. Section 8.6 specifies the log format.

8.3.1.0.1 In Practice. A first-principles priority review of AI-generated interrupt handlers can catch failures that functional tests miss, as the following example illustrates. A team used AI to generate firmware interrupt handlers for a real-time scheduler on the running-example sorter. The AI output compiled cleanly and passed basic unit tests. The interrupt handler correctly serviced the conveyor-belt encoder pulses at low throughput. Under the 6 items/min throughput target, however, interrupt latency spiked intermittently and caused missed encoder edges. Root-cause analysis identified a race condition in the interrupt priority scheme: the AI had assigned equal priority to the encoder interrupt and a lower-frequency status-reporting interrupt. When status reporting and an encoder pulse arrived simultaneously at high throughput, the encoder pulse was occasionally delayed by the status-reporting ISR. A first-principles review of the interrupt priority assignment — checking each interrupt against the real-time deadline table in the design specification — caught the race condition before deployment. The fix was a one-line priority change; the cost of not catching it would have been a throughput failure at Gate 3 demonstration.

8.4 The Black-Box Problem: Deciding When to Verify

You cannot always verify every AI output at the same depth: bench testing every component value is not feasible in a constrained schedule. A quantitative decision

rule prevents both under-verification (trusting plausible output that is wrong) and over-verification (spending verification time that exceeds the value at risk).

8.4.1 Expected Cost of Error

The **expected cost of error** $\mathbb{E}[C_{\text{err}}]$ is:

$$\mathbb{E}[C_{\text{err}}] = p_{\text{err}} \cdot C_{\text{consequence}} \tag{8.1}$$

where p_{err} is your estimated probability that the AI output is wrong for your specific application, and $C_{\text{consequence}}$ is the cost you will incur if the error reaches the prototype undetected (diagnosis time, rework, schedule slip, or safety consequence).

The **verification decision rule** is:

> If $\mathbb{E}[C_{\text{err}}] >$ cost_verify, verify.

This rule makes the verification decision quantitative and defensible. It also reveals when verification is not cost-justified — when the consequence of a wrong value is trivially correctable and the verification cost is disproportionate. For safety-relevant functions, $C_{\text{consequence}}$ is large by definition, and the rule will always recommend verification.

Figure 8.2 shows a trust matrix that maps AI output categories against typical verification requirements.

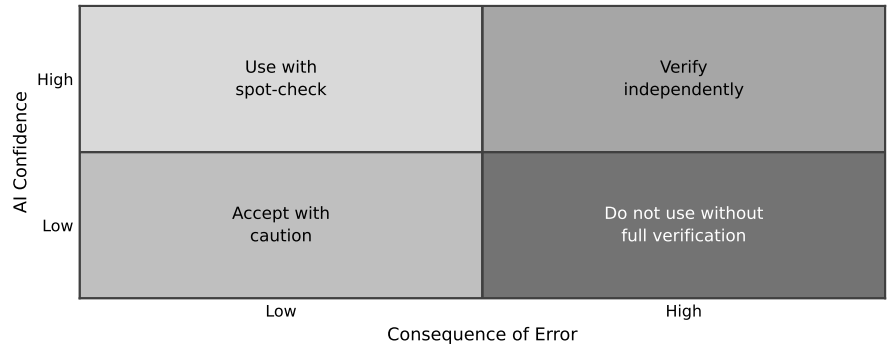

Figure 8.2: Trust matrix for AI-generated design outputs. Outputs with low consequence cost and high AI reliability (e.g., boilerplate file structure, documentation phrasing) can be accepted with a visual review. Outputs with high consequence cost or low AI reliability (e.g., interrupt priority assignments, power-supply compensation networks, safety-critical logic) require independent simulation or bench verification before acceptance. No AI output to a safety-relevant function may be deployed unverified.

8.4.2 Worked Example: I²C Pull-Up Resistor

The running-example MCU board uses an I²C bus to communicate with the sort classifier's sensor array. An AI tool suggests a 10 kΩ pull-up resistor for the bus.

The engineer's assessment:

- $p_{\text{err}} = 0.15$: a 15% chance the value is wrong for this bus speed (400 kHz fast mode) and the estimated 250 pF capacitive load of four sensors on a 15 cm trace. The I²C specification limits the RC rise time, and at 400 kHz the bus capacitance matters; the AI did not have this context.

- $C_{\text{consequence}} = \$480$ CAD: one day of technician time to diagnose intermittent I²C failures, identify the pull-up value as the cause, replace the resistor, and re-verify communication.

- cost_verify $= \$40$ CAD: 20 minutes of engineer time to apply the I²C pull-up selection equation from the NXP UM10204 application note and check the result against a SPICE simulation of the bus topology.

Applying Equation 8.1:

$$\mathbb{E}[C_{\text{err}}] = 0.15 \times \$480 = \$72 \text{ CAD}$$

Decision: $\$72 > \$40$, so verification is justified.

The engineer applies the rise-time equation from the application note. For a 400 kHz bus with $C_b = 250$ pF, the I²C fast-mode specification sets a maximum rise time $t_r = 300$ ns (NXP UM10204, Table 10). The pull-up rise-time equation from that application note is:

$$R_{\text{max}} = \frac{t_r}{0.8473 \cdot C_b}$$

where the constant 0.8473 comes from the I²C voltage threshold definition ($V_{\text{IH,min}} = 0.7\,V_{DD}$, giving $\ln(1/0.3)^{-1} \approx 0.8473$). Substituting:

$$R_{\text{max}} = \frac{300\,\text{ns}}{0.8473 \times 250\,\text{pF}} \approx 1{,}416\,\Omega$$

The AI-suggested 10 kΩ would produce a rise time of approximately 2.1 μs — seven times the I²C fast-mode limit. The AI output is rejected. The engineer selects 1.0 kΩ (well within the $1{,}416\,\Omega$ maximum), verifies the rise time is within specification, and logs the interaction. Table 8.1 shows the complete AI use log entry for this interaction; it satisfies the format specified in Section 8.6.1.

This is the HITL cycle in action: the AI output was plausible (10 kΩ is correct for standard-mode I²C) but wrong for the specific operating conditions. The 20-minute verification prevented an intermittent bus failure that would have been expensive to diagnose on a completed board.

Table 8.1: AI use log entry for the I²C pull-up resistor interaction.

Field	**Entry**
Tool used	(AI service), accessed (date)
Prompt summary	Pull-up resistor value for 400 kHz I²C bus, ≈250 pF bus capacitance, four sensors on 15 cm trace.
Output produced	AI suggested 10 kΩ.
Verification	NXP UM10204 rise-time equation; SPICE simulation of bus topology.
Result	**Rejected.** $R_{\text{max}} \approx 1{,}416\,\Omega$ for fast mode; selected 1.0 kΩ.
$\mathbb{E}[C_{\text{err}}]$	$0.15 \times \$480 = \$72$ CAD $> \$40$ cost_verify; verification justified.

8.4.2.0.1 Common Pitfalls. Trusting plausible output without verification is the most common failure mode when teams begin using AI tools in design work. "It looked right" is not an engineering justification. An output that looks right for the general case is frequently wrong for your specific combination of bus speed, supply voltage, load capacitance, and PCB parasitics.

8.5 IP and Data-Leakage Risk

Using a public AI tool for design work introduces a risk that has no analogue in traditional design tools: the data you submit may leave your control.

8.5.1 What Happens to Your Data

Public AI services vary widely in their data-retention policies. Many services retain submitted content for model-improvement purposes unless you explicitly opt out — and the opt-out mechanism may not apply retroactively to data already submitted. Some services retain conversation history on their servers indefinitely. Enterprise tiers with contractual data-handling guarantees exist, but they require a procurement decision and are not the default for a student or small team logging into a public web interface.

Proprietary design data submitted to a public AI tool may therefore:

- Be stored on the provider's servers beyond the session.
- Be used to train or fine-tune future model versions.
- Be retrievable, in whole or in part, through prompts from other users if it becomes embedded in model weights or cached session data.
- Constitute a public disclosure that affects patent-filing timelines in jurisdictions that require novelty.

Data leakage in this context does not require a security breach. It occurs through normal use of a service whose terms permit the provider to use submitted content.

8.5.2 What Constitutes Proprietary Data

In a Stage 3 prototype context, the following are typically proprietary:

- Schematics and PCB layouts that embody design decisions not yet in a granted patent or published application.
- Firmware source code, particularly inference-engine integration code that represents a competitive advantage.
- BOM entries that reveal supplier relationships or negotiated pricing.

- Design specifications that disclose performance targets ahead of a product announcement.

Generic questions — "what is the I^2C fast-mode maximum pull-up resistance for 250 pF bus capacitance?" — carry no proprietary content. Questions that include a schematic image, a firmware file, or project-specific parameters that identify your design carry significant risk.

8.5.2.0.1 In Practice. A team pasted a proprietary motor-driver schematic into a public AI chat session to ask for a component suggestion. The schematic included part numbers, supplier codes, and a circuit topology that was not yet in the prior art. Six months later, when another user asked a similar question about motor-driver design, the AI reproduced elements of the schematic — part numbers and a recognisable sub-circuit — in its response. The team had not read the provider's data-retention terms before submitting the schematic; those terms permitted the provider to use submitted content for model improvement. The incident triggered a review by the organisation's legal counsel and delayed a patent filing.

8.5.3 The Data-Handling Policy

Before using any AI tool with design data, establish a simple policy:

1. **Classify the data.** Is it generic (publicly available physics and formulas) or proprietary (specific to your design)?
2. **Check the provider's terms.** Does the provider retain submitted content? Is there an enterprise or confidentiality tier?
3. **Anonymise where possible.** Replace specific part numbers with generic descriptions; replace project-specific values with ranges. "A pull-up resistor on a 400 kHz I^2C bus with 250 pF load" reveals nothing proprietary. A schematic image does.
4. **Log what was submitted.** The AI use log (Section 8.6) records a *prompt summary*, not the raw prompt, specifically to avoid logging proprietary data in a document that may be shared.

8.5.3.0.1 Common Pitfalls. Submitting confidential design data to a public AI tool without checking the provider's data-retention policy is a data-governance failure, not a technical one. The circuit may be correctly analysed; the design may be accelerated. The risk is not to this prototype — it is to every future product, patent filing, or competitive position that depends on the information remaining confidential. Check the terms before you paste the schematic.

8.6 Reproducibility and Traceability

Gate 3 requires evidence that your design decisions are defensible. If AI tools contributed to a design decision, the Gate 3 record must show: what the AI produced,

how it was verified, and what the engineer decided as a result. A design whose AI contributions are undocumented cannot be reproduced, audited, or defended.

8.6.1 The AI Use Log

Every AI-assisted design interaction should produce an entry in an **AI use log**. Each entry contains:

1. **Tool used.** Name and version (or access date) of the AI service.
2. **Prompt summary.** A description of what was asked, with no proprietary data. Example: "Asked for pull-up resistor value for 400 kHz I^2C bus with approximately 250 pF capacitive load."
3. **Output produced.** The AI's specific recommendation. Example: "AI suggested 10 kΩ."
4. **Verification method.** How the output was checked. Example: "Applied NXP UM10204 rise-time equation; SPICE simulation of bus topology."
5. **Result.** Accepted or rejected, with brief rationale. Example: "Rejected. Calculated $R_{\text{max}} \approx 1{,}416\,\Omega$ for fast mode; selected 1.0 kΩ."
6. $\mathbb{E}[C_{\textbf{err}}]$ **computed.** The expected cost of error used to justify the verification decision.

A log entry takes less than five minutes to write. It transforms an undocumented shortcut into a traceable, auditable design decision.

8.6.1.0.1 Common Pitfalls. The most consequential documentation failure is having no record of AI involvement in a design. If a Gate 3 reviewer asks "How was this pull-up value selected?" and the answer is "I asked an AI," the follow-up question is immediate: "How was the AI output verified?" Without a log entry, you cannot answer that question. With a log entry showing $\mathbb{E}[C_{\text{err}}] = \$72$ and verification cost $= \$40$, and a rejection of the AI's suggestion with the correct value derived from the application note, you have demonstrated exactly the engineering judgment a Gate 3 review is designed to find.

8.6.2 Reproducibility Requirement

Reproducibility in this context means that a second engineer reading the AI use log can reconstruct the verification independently and reach the same conclusion. Each log entry must be specific enough that a second engineer can reconstruct the verification using the prompt summary, output record, named source, and result fields specified in Section 8.6.1.

Reproducibility is not a bureaucratic requirement. It is the standard of evidence that distinguishes engineering from guessing.

Try It Yourself

A team uses AI to suggest a decoupling capacitor value for the 3.3 V power pin of the MCU on the running-example sorter board.

- $p_{\text{err}} = 0.20$: a 20 % chance the suggested value is suboptimal for this PCB layout (high-frequency bypass inadequate for the MCU's transient current demand).
- $C_{\text{consequence}} = \$600$ CAD: one day of debug and rework if the MCU resets intermittently under load at high throughput.
- cost_verify $= \$25$ CAD: 15 minutes to check the MCU datasheet application note for the recommended decoupling network.

(a) Compute $\mathbb{E}[C_{\text{err}}]$ using Equation 8.1.

(b) Is verification justified? State the decision rule explicitly.

(c) The team reduces p_{err} to 0.04 by first checking the MCU datasheet application note before querying the AI, so the AI prompt includes the datasheet-recommended value range and asks only whether the layout parasitics would change the recommendation. Recompute $\mathbb{E}[C_{\text{err}}]$ and state whether verification is still justified.

(d) *(Conceptual)* Name two pieces of information that must appear in the AI use log entry for this interaction, beyond the tool name and date, and explain in one sentence why each is necessary for Gate 3 traceability.

(No answer key provided.)

Review Questions

1. **(Conceptual)** List the three categories of design task from Section 8.1 where AI provides genuine value in the prototyping workflow. For each category, state one specific task from the running-example IoT robotic sorter project that falls within it.
2. **(Conceptual)** Explain the difference between a factual error and an AI hallucination in an engineering context. Why is an AI hallucination potentially more dangerous than a simple factual error when it appears in a design output such as a component value or a firmware interrupt priority assignment?
3. **(Applied — calculation)** A team generates an AI-suggested motor-driver current-sense resistor value. The engineer estimates $p_{\text{err}} = 0.08$ and $C_{\text{consequence}} = \$1{,}200$ CAD (motor overcurrent damage and board replacement if the sense resistor is wrong). Bench verification of the value takes 30 minutes, equivalent to cost_verify $= \$60$ CAD.
   (a) Compute $\mathbb{E}[C_{\text{err}}]$ using Equation 8.1.
   (b) Is verification justified? Apply the decision rule explicitly.

(c) At what value of p_{err} does $\mathbb{E}[C_{\text{err}}]$ exactly equal cost_verify? Show the algebra. Then state: if the engineer could reduce p_{err} to half the breakeven value by spending an additional $15 CAD on a datasheet review before prompting the AI, is that additional expenditure cost-justified? Show your reasoning.

4. **(Applied — multi-step)** A team generates three AI outputs in one design session:

   - Output A: a pull-up resistor value. $p_{\text{err}} = 0.12$, $C_{\text{consequence}} = \$320$ CAD, cost_verify $= \$30$ CAD.
   - Output B: a firmware loop structure for the conveyor control task. $p_{\text{err}} = 0.25$, $C_{\text{consequence}} = \$800$ CAD, cost_verify $= \$50$ CAD.
   - Output C: a BOM line-item description for a passive component. $p_{\text{err}} = 0.05$, $C_{\text{consequence}} = \$80$ CAD, cost_verify $= \$20$ CAD.

   (a) Compute $\mathbb{E}[C_{\text{err}}]$ for each output using Equation 8.1.

   (b) Which outputs should be verified according to the decision rule?

   (c) Rank **all three** outputs by verification priority (highest $\mathbb{E}[C_{\text{err}}]$ first), then identify the threshold $\mathbb{E}[C_{\text{err}}]$ value below which an output in this session would not justify verification.

5. **(Conceptual)** A team member proposes submitting the following information to a public AI chat service to get component suggestions: a 48 V motor-driver schematic showing the full H-bridge topology, the project's part number prefix, and the target current rating. Using the data classification policy from Section 8.5.3, identify which elements of this submission carry proprietary risk and state what anonymised substitution should be made for each before the prompt is submitted.

6. **(Conceptual)** A colleague argues: "We should just verify everything AI produces, so we never need to compute $\mathbb{E}[C_{\text{err}}]$." Explain in two sentences why this policy fails in practice for a team operating under a fixed Stage 3 schedule.

Portfolio Entry

Produce an AI use log with at least three entries from your prototype design session. Each entry must include:

1. **Tool used** — name and version or access date.
2. **Prompt summary** — what was asked, with no proprietary data in the record.
3. **Output produced** — the specific value, code, or recommendation the AI returned.
4. **Verification method applied** — the independent check used (datasheet section, simulation, calculation, bench test).
5. **Result** — accepted or rejected, with the accepted or corrected value if rejected.

6. $\mathbb{E}[C_{\mathbf{err}}]$ **computed** — the expected cost of error that justified the verification depth.

At least one entry must result in a rejection (the AI output was wrong and a corrected value was derived independently).

Chapter Summary

AI tools accelerate Stage 3 design work in three categories: code and firmware scaffolding, component value suggestion, and design alternatives and documentation drafting. In every category, the engineer remains the accountable party; AI provides a faster first draft, not a substitute for independent verification.

The expected cost of error $\mathbb{E}[C_{\text{err}}] = p_{\text{err}} \cdot C_{\text{consequence}}$ (Equation 8.1) makes the verification decision quantitative: verify when $\mathbb{E}[C_{\text{err}}]$ exceeds the cost of verification. The human-in-the-loop cycle (Figure 8.1) and the trust matrix (Figure 8.2) provide the operational framework for applying this rule consistently.

IP and data-leakage risk requires a data-handling policy before AI tools are used with design data: classify the data, check the provider's retention terms, and anonymise proprietary content in prompts.

Traceability is a Gate 3 requirement. The AI use log records what was generated, how it was verified, and what the engineer decided — transforming an unauditable shortcut into a defensible design decision.

The AI use log produced in this chapter is the Stage 3 traceability artifact that shows every AI-assisted decision was independently verified before it entered the Gate 3 design package. Chapter 9 takes that verified design and addresses the obligations that come with the hardware itself — security, power, and physical-safety requirements for a connected, moving system.

CHAPTER 9

IOT, ROBOTICS & SAFETY AS A DESIGN OBLIGATION

9.0.0.0.1 Where this fits. This chapter operates at Stage 3 (Development) of the Stage-Gate model. Security and physical-safety requirements not designed in at this stage will surface as failures at Gate 4 (Testing & Validation), where every retrofit costs more time, more money, and more risk than the original design decision would have. A security flaw discovered during a field test does not cost one library swap; it costs a firmware re-architecture, a re-flash campaign, and a delay that may miss the project deadline. A prototype moving part without a hardware emergency stop does not fail quietly; it fails dangerously.

9.0.0.0.2 Learning Objectives. After completing this chapter you will be able to:

1. Identify the three IoT attack surfaces and specify at least one security control for each.
2. Compute average current I_{avg} and battery life t_{life} for a duty-cycled IoT device.
3. Identify the three required physical-safety features for a moving robotic prototype: e-stop, guarding, and fail-safe state.
4. Compute the mechanical safety factor SF for a load-bearing element and assess whether $\text{SF} \geq 2$ is satisfied.

Chapter 8 delivered a working firmware prototype built with AI-assisted tooling inside Stage 3. That left open a second set of Stage 3 obligations: the security, power, and physical-safety requirements that the hardware itself carries — regardless of what tools were used to design it. This chapter closes that gap.

9.1 IoT Security by Design

Security by design means treating encryption, authentication, and secure communication as first-class design requirements, not optional enhancements added after the hardware is built and running.

The cost-of-change principle from Chapter 3 applies directly: the three security properties below must be designed in, not added after.

Encryption ensures that data in transit cannot be read or modified by a third party. For an MQTT sorter node communicating over Wi-Fi, this means TLS 1.2 or TLS 1.3 between the device and the broker. Encryption without authentication is incomplete: an attacker who cannot read your messages can still inject their own if the broker accepts unauthenticated connections.

Authentication ensures that only legitimate devices and users can connect to your system. At minimum, each device must present a unique credential — a per-device username and password, or a client certificate — that the broker can verify and revoke independently of every other device. Shared credentials, such as a single "device" account used by all nodes, mean that compromising one device compromises every device on the network.

Secure update closes the lifecycle loop. Firmware that cannot be updated securely in the field is firmware that will run outdated cryptographic libraries indefinitely. Signing firmware images with a private key and verifying the signature on the device before flashing is the minimum acceptable posture for any deployed IoT prototype.

9.1.0.0.1 In Practice. An IoT prototype shipped with default MQTT broker credentials ("admin" / "password"). Within 48 hours of deployment, a network scanner found the open broker and began injecting spurious sort commands into the conveyor system. The fix — per-device credentials and TLS — took two days to implement after the fact. At design time, the same change would have taken twenty minutes.

9.2 Attack Surfaces

Every IoT system has three distinct **attack surfaces**: the layer at which an adversary can attempt to gain unauthorised access or cause harm. Understanding the surfaces helps you assign a control to each one at design time rather than discovering them during a security audit.

Figure 9.1 maps each layer to its principal threats.

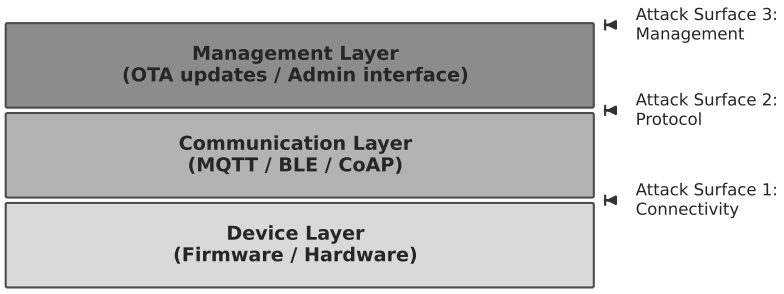

Figure 9.1: Layered IoT architecture for the MQTT sorter node. The connectivity layer (Wi-Fi/Ethernet), the communication layer (MQTT broker and topics), and the management layer (OTA update and admin interface) each carry a distinct attack surface requiring its own security control.

9.2.1 Connectivity Layer

The **connectivity layer** is the physical and link-layer network: Wi-Fi, Ethernet, or a cellular radio. Threats here include passive eavesdropping on unencrypted traffic, man-in-the-middle insertion, and denial-of-service through radio jamming or TCP flooding.

Apply network-layer encryption (WPA3 for Wi-Fi; VPN or private APN for cellular) and restrict the device to a dedicated IoT VLAN that has no route to general office or campus infrastructure. For the running-example sorter node, the MQTT broker should be reachable only from the IoT VLAN, not from a shared student network.

9.2.2 Communication Layer

The **communication layer** is the application protocol: MQTT topics, CoAP resources, or BLE GATT characteristics. Threats here include topic spoofing (an attacker publishes false sensor readings or spurious actuator commands), topic enumeration (an attacker subscribes to # and reads every message), and replay attacks (a captured command is re-sent later).

Authenticate every publisher and subscriber at the broker using per-device credentials; use topic-level access control lists (ACLs) so that a sensor node can publish to its own telemetry topic but cannot publish to the actuator command topic; include a message timestamp or sequence number to detect replays.

9.2.3 Management Layer

The **management layer** is the interface used to update firmware and configure the device: an OTA update server, a web dashboard, an SSH port, or a JTAG adapter left enabled in production firmware.

Threats here are the most severe because compromising the management layer gives an attacker the ability to re-flash every device. Disable all debug interfaces in release builds; sign firmware images; use mutual TLS for OTA update channels; change all default admin credentials before first deployment; enforce multi-factor authentication on the cloud dashboard.

9.2.3.0.1 Common Pitfalls. "Add security later" is the most expensive four-word sentence in embedded development. Retrofitting TLS onto an MQTT stack that was built without it requires restructuring the firmware state machine — the connection, subscription, publish, and reconnect logic all carry TLS context — not just swapping a library call. The second pitfall is shipping default credentials in production firmware. Every device with identical credentials is a single point of compromise: one leaked password grants access to the entire deployed fleet.

9.3 Privacy and Data Obligations

Sensor data is not automatically neutral. Certain categories of data collected by an IoT device trigger legal and ethical obligations that your design must accommodate from the start.

Personal data is any information that can identify a natural person, directly or indirectly. A CO_2 sensor reporting aggregate room occupancy is not personal data; a badge reader logging individual entry times is. Under Québec Law 25 (Loi 25) and the federal PIPEDA, collecting personal data requires a stated purpose, user consent, and documented retention limits. If your IoT device collects personal data, you must specify those elements before the device is deployed.

Location data is a subcategory that receives heightened scrutiny because it reveals movement patterns over time. A GPS tag on a delivery robot that reports position continuously creates a persistent location history. Design controls include anonymisation (report zone, not coordinates), aggregation (report 10-minute averages, not 1-second fixes), and retention limits (purge data older than 30 days).

Biometric data — fingerprint templates, facial recognition embeddings, voice prints — is treated as sensitive in virtually every jurisdiction. If a prototype uses a camera for person detection, you must assess whether the processing constitutes biometric identification and, if so, obtain explicit consent and provide a technical architecture that does not transmit raw biometric templates over a network.

The practical rule at Stage 3 is: for each sensor on your device, ask whether the data it produces could identify a person or reveal their behaviour. If yes, document the legal basis for collection, the purpose limitation, and the retention policy in your design records before Gate 3.

9.4 Protocol Selection

Choosing a communication protocol is simultaneously a security decision, a power decision, and an interoperability decision; each of the three protocols used at prototype scale — MQTT, CoAP, and BLE — carries a distinct security posture and power profile requiring its own design controls.

9.4.1 MQTT

MQTT (Message Queuing Telemetry Transport) is a publish-subscribe protocol designed for constrained devices on unreliable networks. It is TCP-based, which means it carries the overhead of a persistent connection but provides reliable, ordered delivery. For the running-example sorter node, MQTT is the protocol of record.

Mandatory security posture for MQTT in a prototype:

- Use TLS 1.2 or TLS 1.3 between every client and the broker. Plain-text MQTT on port 1883 is not acceptable on any network reachable from the internet.

- Use an **authenticated broker**: the broker must require a username and password (or client certificate) before accepting a connection.
- Issue **per-device credentials**: each node has its own username/password pair so that a compromised node can be revoked without affecting any other device.
- Define topic ACLs so that sensor nodes cannot publish to actuator command topics.

9.4.2 CoAP

CoAP (Constrained Application Protocol) is a request-response protocol modelled on HTTP but designed for UDP transport and very small code footprints. It is appropriate when the device cannot maintain a persistent TCP connection or when a RESTful resource model fits the application better than a topic hierarchy.

Security posture: use DTLS (Datagram TLS) to secure CoAP traffic. Plain-text CoAP has no confidentiality or integrity protection. DTLS adds handshake overhead to each session, but pre-shared key (PSK) mode keeps the footprint manageable on constrained microcontrollers.

9.4.3 BLE

BLE (Bluetooth Low Energy) is a short-range, low-power protocol suited to wearables, beacons, and devices that communicate with a nearby gateway or smartphone. Its power consumption advantage over Wi-Fi is significant: a BLE advertisement at 1 dBm typically costs less than 10 mA for a few milliseconds, while a Wi-Fi association sequence can draw 100–200 mA for hundreds of milliseconds.

Security posture: use **LE Secure Connections** (introduced in Bluetooth 4.2), which uses Elliptic Curve Diffie-Hellman for key exchange. Legacy "Just Works" pairing provides no meaningful security against passive eavesdropping; avoid it in any design that transmits data of value.

The trade-off summary: MQTT over TLS is the right choice for the sorter node because the broker provides centralised logging, topic ACLs, and a dashboard integration point. BLE would be appropriate if the node were battery-powered and within 10 m of a gateway. CoAP is appropriate for constrained nodes that cannot sustain a TCP connection.

9.5 Power and Battery Budgeting

Battery-powered IoT nodes operate in a **duty cycle**: they wake, perform a measurement and a transmission, then return to a deep-sleep mode that draws a fraction of the active current. The duty cycle D is the fraction of time the device spends in its active state.

9.5.1 Average Current

Because the device spends most of its time asleep, the battery sees a time-weighted average current that is much lower than the active current. The **average current** I_{avg} is:

$$I_{\text{avg}} = D \cdot I_{\text{active}} + (1 - D) \cdot I_{\text{sleep}} \tag{9.1}$$

where I_{active} is the current drawn during the active interval, I_{sleep} is the current drawn during the sleep interval, and D is the duty cycle (dimensionless, $0 < D < 1$).

At low duty cycles — say, $D = 0.02$ (2 %) — the sleep current often dominates the average even if it is two or three orders of magnitude smaller than the active current. This means that reducing I_{sleep} by choosing a microcontroller with a lower deep-sleep current may improve battery life more than reducing I_{active}.

9.5.2 Battery Life

Given I_{avg} and a battery capacity C_{batt} (in mAh), the estimated **battery life** t_{life} is:

$$t_{\text{life}} = \frac{C_{\text{batt}}}{I_{\text{avg}}} \tag{9.2}$$

This is an idealised estimate. Real battery capacity degrades with temperature, discharge rate, and age; a practical design should target t_{life} at least 20 % above the specification to account for these effects.

Figure 9.2 shows the current profile for a duty-cycled node, with I_{avg} marked as the equivalent constant-current drain.

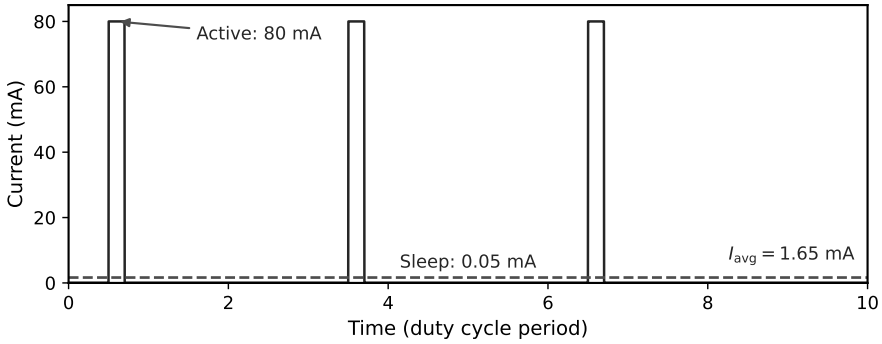

Figure 9.2: Duty-cycled current profile for an MQTT sensor node. Narrow active pulses (drawing I_{active}) are separated by long sleep intervals (drawing I_{sleep}). The dashed horizontal line marks I_{avg}, the constant current that would drain the battery in the same time. At $D = 2\,\%$, the sleep interval dominates the average.

9.5.3 Worked Example — Power Budget

The running-example MQTT sorter node has the following parameters: $I_{\text{active}} = 80$ mA, $I_{\text{sleep}} = 0.05$ mA, $D = 0.02$ (the node wakes every 10 s for a 200 ms measurement-

and-publish cycle), $C_{\text{batt}} = 2000$ mAh.

Step 1: Compute I_{avg}.

Using Equation 9.1:

$$I_{\text{avg}} = 0.02 \times 80 + 0.98 \times 0.05 = 1.600 + 0.049 = 1.649 \text{ mA}$$

Step 2: Compute t_{life}.

Using Equation 9.2:

$$t_{\text{life}} = \frac{2000}{1.649} = 1212.9 \text{ h} \approx 50.5 \text{ days}$$

Step 3: Sensitivity — halve the duty cycle.

If the team can reduce the reporting rate so that $D = 0.01$ (one report per 20 s instead of per 10 s):

$$I'_{\text{avg}} = 0.01 \times 80 + 0.99 \times 0.05 = 0.800 + 0.0495 = 0.8495 \text{ mA}$$

$$t'_{\text{life}} = \frac{2000}{0.8495} \approx 2354 \text{ h} \approx 98.1 \text{ days}$$

Halving the duty cycle nearly doubles the battery life (from 50.5 days to 98.1 days). The improvement is not exactly $2\times$ because the sleep current sets a floor: as $D \to 0$, $I_{\text{avg}} \to I_{\text{sleep}}$, not zero. This sensitivity analysis belongs in the power budget table of your design records.

9.6 Robotics Physical Safety

Any prototype that moves — a conveyor, a sort gate, a rotating arm, a wheeled platform — imposes physical-safety obligations that software alone cannot satisfy. Three features are non-negotiable for any moving robotic prototype in this course, and in professional practice.

Figure 9.3 identifies all three safety features in the running-example workspace.

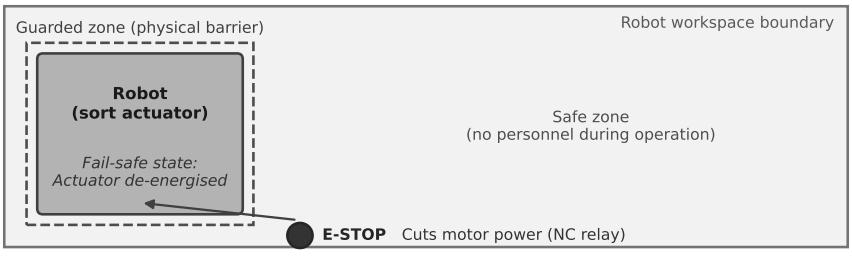

Figure 9.3: Sorter-node robot workspace. The exclusion zone (hatched) around the conveyor and sort gate is bounded by physical guarding (solid line). The e-stop button is positioned in the operator zone, outside the guarding, and its normally-closed contact sits in series with the motor power rail. The fail-safe state (conveyor stopped, gate centred) is the power-off default.

9.6.1 Emergency Stop

An **emergency stop (e-stop)** is a hardware mechanism — not a software function — that removes power from every actuator when activated. The key word is hardware: a normally-closed (NC) relay or contactor in series with the motor power supply that opens when the e-stop button is pressed or when the button circuit is broken.

A software e-stop (a flag checked in the main loop, an interrupt service routine that calls `motor_stop()`) does not satisfy this requirement. If the firmware crashes, the stack overflows, or a runaway interrupt handler locks the scheduler, the software e-stop cannot execute. The hardware relay opens regardless of firmware state.

Requirements for the running example:

- One NC e-stop button accessible from the operator zone, outside the guarding perimeter.
- The e-stop contact in series with the 12 V motor supply rail.
- Latching action: the system must not restart when the button is released; it requires a deliberate reset after the hazard has been cleared.

9.6.1.0.1 In Practice. A robot prototype during a lab test had only a software e-stop. During a firmware crash, the motor continued running and drove the conveyor belt into its mechanical end-stop. The belt was damaged and the test had to be postponed. A normally-closed hardware relay in the motor power path would have cut power the instant the crash was detected by the watchdog — or the moment a person reached for the e-stop button.

9.6.2 Guarding

Guarding is a physical barrier that prevents unintended contact between a person and a moving part. Guarding is not a "nice to have"; under Québec's Act Respecting Occupational Health and Safety (LSST) and the federal *Canada Labour Code*, any moving part that presents a hazard must be guarded or rendered inaccessible.

For the bench-scale sorter prototype, acceptable guarding includes:

- Acrylic or aluminium side panels along the conveyor belt that prevent fingers from reaching the belt, rollers, or sort gate during operation.
- An interlocked access panel: if the panel is opened, the interlock opens the motor power circuit (the same NC relay used by the e-stop).
- A minimum guarding height that prevents reaching over the barrier to the nearest pinch point.

Guarding must be specified in your design records as a dimensional requirement, not a verbal intention. State the material, minimum dimensions, and mounting method.

9.6.3 Fail-Safe State

A **fail-safe state** is the physical state the system enters when power is removed or a fault is detected. Designing the fail-safe state requires an explicit decision: what position should every actuator reach when power is cut?

For the sort gate actuator, the fail-safe state must be the centred (neutral) position. This means the actuator spring-returns to centre on power loss, not to the "sort left" or "sort right" extreme. For the conveyor motor, the fail-safe state is stopped. This is achieved by using a spring-return actuator and a motor driver that de-energises the motor on power loss, not one that holds its last commanded position.

Use this three-question checklist for each actuator:

1. What is the safest physical position on power loss?
2. Does the mechanism reach that position by gravity, spring return, or active braking?
3. Is the power-off default that position, or does it require a commanded move?

If the power-off default is not the safest position, redesign the mechanism or add a spring return.

9.6.4 Force, Speed Limits, and Human-Robot Interaction Zones

Beyond the three required features, professional practice for moving prototypes includes **force and speed limits**: the maximum force an actuator can exert and the maximum speed at which it can move when a person might be present.

For the bench sorter, define:

- A maximum conveyor belt speed (e.g., 0.3 m/s) that limits the kinetic energy of items being sorted and reduces the risk of ejection.
- A maximum sort-gate actuator force consistent with the SF calculation below.
- A **human-robot interaction (HRI) zone**: the area within which a person might reach the moving parts during normal operation. This zone determines the guarding perimeter.

9.7 Mechanical Safety Factor

Every load-bearing element in a moving prototype must be verified against a minimum **safety factor** SF:

$$\text{SF} = \frac{\text{capacity}}{\text{applied load}} \tag{9.3}$$

where capacity is the maximum load the element can sustain before failure (from material data or simulation) and applied load is the worst-case force the element will experience in service. The course minimum is $\text{SF} \geq 2$; this means the element can carry at least twice the worst-case load before failure. Any element that fails this criterion requires a redesign before it is installed in a moving prototype.

9.7.1 Worked Example — Safety Factor

The sort-gate actuator bracket is 3D-printed PLA. The bracket supports the actuator mass of 0.45 kg. During a misfire event (the sort gate attempts to move while an item is lodged in the mechanism), the impact load is estimated at $3\times$ the static weight.

Step 1: Compute the applied load.

$$F_{\text{applied}} = 3 \times 0.45\ \text{kg} \times 9.81\ \text{m/s}^2 = 3 \times 4.4145\ \text{N} = 13.24\ \text{N}$$

Step 2: Compute the capacity.

From the material specification for this PLA print orientation and infill density, the tensile strength is 35 MPa. The bracket cross-section area at the critical section is 2.0 mm^2.

$$F_{\text{capacity}} = 35\ \text{MPa} \times 2.0\ \text{mm}^2 = 35\ \text{N/mm}^2 \times 2.0\ \text{mm}^2 = 70\ \text{N}$$

Step 3: Compute SF.

$$\text{SF} = \frac{70\ \text{N}}{13.24\ \text{N}} = 5.29$$

$\text{SF} = 5.29 \geq 2$ — the bracket passes.

Sensitivity check — reduced cross-section.

If the design were modified to reduce the cross-section to 0.5 mm^2 (to save material or to route a cable channel through the bracket):

$$F'_{\text{capacity}} = 35 \times 0.5 = 17.5\ \text{N}$$

$$\text{SF}' = \frac{17.5}{13.24} = 1.32 < 2 \quad \Rightarrow \text{FAILS}$$

A cross-section of 0.5 mm^2 fails the safety criterion. The bracket must be redesigned — wider cross-section, different geometry, or a different material with higher tensile strength — before it is used. Record the failing calculation and the redesign rationale in your design records; Gate 4 reviewers will check that every load-bearing element has a documented SF.

9.8 EMC, ESD, PPE, and Regulatory Obligations

Three additional safety disciplines are absorbed into Stage 3 practice: EMC/ESD design, personal protective equipment (PPE) in the lab, and applicable safety law.

9.8.1 EMC and ESD

Electromagnetic compatibility (EMC) means that the device neither emits interference above regulated limits nor fails when exposed to external interference. For a prototype, the principal EMC obligation at Stage 3 is design hygiene: decoupling capacitors on every power rail close to each IC, ground planes on the PCB, and cable routing that minimises loop area. These are not certification activities; they are design habits that prevent self-interference and reduce the cost of formal EMC testing later.

Electrostatic discharge (ESD) protection means that the design can survive the voltage spikes generated by a person touching the device. Every I/O pin that is accessible in normal use should have TVS diode protection or an ESD-rated I/O expander. For the sorter node, this includes the MQTT status LED pins and the sort-gate actuator control lines.

9.8.2 Lab PPE

Personal protective equipment (PPE) requirements during prototype assembly and testing:

- Safety glasses during all power-on testing of circuits above 30 V DC or any circuit with capacitors above 100 μF.
- Anti-static wrist strap when handling bare PCBs or ICs outside of anti-static packaging.
- No loose clothing or jewellery within the guarded zone of the conveyor during any belt-speed test above 0.1 m/s.

These are documented in the Vanier College laboratory safety rules and in the course safety briefing. They are also design inputs: if the prototype requires testers to work near exposed moving parts, the guarding design is incomplete.

9.8.3 Applicable Safety Law

Three regulatory frameworks apply to the running example:

- **Québec LSST** (Act Respecting Occupational Health and Safety): requires guarding of all hazardous moving parts and a means of stopping the machine in an emergency.

- **Canada Labour Code Part II**: requires that every employer ensure that equipment is safe and that workers are informed of known hazards.
- **IEC 62368-1** (Audio/video, information and communication technology equipment — Safety requirements): the applicable IEC product safety standard for the electronic components in the sorter system. Compliance is not required for a prototype, but the standard's hazard-based safety engineering methodology is the reference model for Stage 3 safety decisions.

These frameworks do not require certification for a class prototype. They do require that you are aware of the obligations a product built from this prototype would carry, and that your design decisions are consistent with meeting those obligations.

9.8.3.0.1 Common Pitfalls. Omitting ESD protection on accessible I/O pins. A single unprotected pin touched during a live test can latch a microcontroller into an undefined state silently — no error, no crash, just corrupted peripheral registers. TVS diodes cost pennies and are impossible to retrofit onto a finished PCB without a re-spin.

9.9 Try It Yourself

A wireless CO_2 sensor node has the following parameters: $I_{\text{active}} = 45$ mA, $I_{\text{sleep}} = 0.08$ mA, $D = 0.05$ (the node wakes for a 3-s measurement cycle every 60 s), $C_{\text{batt}} = 1200$ mAh. A mounting bracket (aluminium, tensile capacity in this cross-section = 120 N) supports the sensor mass of 0.30 kg. The bracket experiences a peak load of $4\times$ the static weight during installation impact.

(a) Compute I_{avg} using Equation 9.1. Show every arithmetic step.

(b) Compute t_{life} using Equation 9.2. Express your answer in hours and in days.

(c) The team switches to a lower-power radio that reduces I_{active} to 20 mA while keeping $D = 0.05$. Recompute I_{avg} and t_{life}. What is the percentage improvement in battery life compared to part (b)?

(d) Compute the peak applied force in Newtons (use $g = 9.81$ m/s^2), then compute SF for the bracket using Equation 9.3. Does the bracket pass the SF ≥ 2 criterion?

9.10 Review Questions

Q1. *(Conceptual)* Identify the three IoT attack surfaces from Section 9.2. For each, give one security control that applies specifically to the running-example MQTT sorter node.

Q2. *(Conceptual)* Explain why designing security in at Stage 3 costs less than retrofitting it at Stage 4. Use the cost-of-change principle from Chapter 3 to support your

argument. Your answer should address at least one concrete technical reason (not just "it takes longer").

Q3. *(Applied, calculation)* An IoT sensor has $I_{\text{active}} = 120$ mA, $I_{\text{sleep}} = 0.02$ mA, $D = 0.03$, $C_{\text{batt}} = 3000$ mAh.

(a) Compute I_{avg} using Equation 9.1.

(b) Compute t_{life} in hours and days using Equation 9.2.

(c) The team wants $t_{\text{life}} \geq 90$ days (2160 h). What is the maximum duty cycle D that achieves this? Show your algebraic derivation starting from Equations 9.1 and 9.2.

Q4. *(Applied, multi-step)* A 3D-printed PLA conveyor guide bracket supports a hanging load of 0.80 kg. Peak impact load = $5\times$ static weight. PLA tensile strength = 42 MPa.

(a) Compute the peak applied force (use $g = 9.81$ m/s^2).

(b) The bracket cross-section is 3.5 mm^2. Compute the structural capacity in Newtons.

(c) Compute SF using Equation 9.3. Does it pass the SF ≥ 2 criterion?

(d) If the criterion is raised to SF ≥ 3, what minimum cross-section area is required? Show the algebra.

Q5. *(Conceptual)* Identify the three required physical-safety features from Section 9.6 for a moving robotic prototype. For each feature, explain what failure mode it prevents and why a software-only implementation of that feature is insufficient.

Portfolio Entry

Update your design records with three entries. Each entry is a section of your prototype documentation package and will be audited at Gate 3.

1. *Security design decisions* — protocol choice (MQTT over TLS, CoAP with DTLS, or BLE with LE Secure Connections), authentication method (per-device credentials or client certificates), and credential management policy (how credentials are generated, stored, distributed, and revoked).
2. *Power budget table* — I_{active}, I_{sleep}, D, I_{avg}, C_{batt}, t_{life}, and at least one sensitivity row (e.g., halved duty cycle or reduced sleep current).
3. *Safety design decisions* — e-stop type, location, and wiring diagram; guarding specification (material, dimensions, mounting method); fail-safe state definition for every actuator; and SF calculation for all load-bearing elements, with cross-section area and material data cited.

Chapter Summary

Chapter 9 treated security and physical safety as quantitative Stage 3 obligations, not qualitative afterthoughts. The quantitative tools this chapter introduced are:

- $I_{avg} = D \cdot I_{active} + (1-D) \cdot I_{sleep}$ and $t_{life} = C_{batt}/I_{avg}$ — the power budget that determines whether your duty-cycle design meets the battery-life requirement.
- $SF = \text{capacity}/\text{applied load}$ — the structural safety floor, with $SF \geq 2$ required for every load-bearing element in a moving prototype.

The three non-negotiable physical-safety features for any moving prototype are: a hardware e-stop in the motor power path, physical guarding around every hazardous moving part, and a documented fail-safe state for every actuator.

Chapter 9 produced three design-record entries — a security design decision log, a power budget table, and a safety design decision log — that together bound every Stage 3 hardware commitment. Chapter 10 takes those entries as inputs and organises the build into a schedule: a work breakdown structure, a critical path, and a Gantt chart that makes Gate 3 achievable rather than aspirational.

CHAPTER 10

PLANNING & SCHEDULE MANAGEMENT

10.0.0.0.1 Where this fits. This chapter is a **Stage 3 (Development)** deliverable. A realistic project schedule and an updated risk register are **Gate 3** artefacts; a schedule built on optimistic guesses or missing lead times is a gate failure *before* the prototype is assembled.

Chapter 9 established the electrical and safety architecture of the IoT robotic sorting subsystem: the power budget confirmed a 50.5-day battery life at $I_{\text{avg}} = 1.649$ mA, and the safety design log recorded a load-path safety factor of $SF = 5.29$. That work left one critical question unanswered: *in what order does the team build everything, and when will it be done?* This chapter answers that question. You will decompose the build into a Work Breakdown Structure, link activities with dependencies, identify the critical path, handle procurement lead times as explicit schedule risks, and finish with a Gantt chart and an updated risk register.

10.0.0.0.2 Learning Objectives. By the end of this chapter you will be able to:

1. Identify all project activities, organise them into a **Work Breakdown Structure** (WBS), and determine the critical path using the Critical Path Method (CPM).

2. **Build** a Gantt chart from CPM results, annotating critical-path bars, slack extensions on non-critical activities, and milestone markers.

3. Compute early/late start and finish times, slack, and the critical path for a multi-activity network.

4. Estimate activity durations under uncertainty by applying the PERT three-point formula and interpreting the resulting standard deviation.

5. Maintain a living risk register that treats procurement lead times and supplier delays as first-class schedule risks.

10.1 Work Breakdown Structure

A **Work Breakdown Structure** (WBS) is a hierarchical decomposition of all the work required to complete the project. Every leaf node of the WBS is a **work package**: a discrete, assignable, and estimable unit of effort.

10.1.1 Why decompose first?

Dependencies, durations, and risks can only be managed once work is visible. A single task labelled "build the prototype" cannot be scheduled, assigned, or monitored. Breaking it into atomic work packages — PCB design, component procurement, firmware integration, system test — makes each element plannable and observable.

10.1.2 WBS for the sorter prototype

The sorter build decomposes into two top-level branches: *Hardware* and *Software*. Under Hardware the packages are PCB design (A), component procurement (B), PCB fabrication (C), and PCB assembly (D). Under Software the packages are firmware integration (E). System test (F) spans both branches and sits at the integration level.

Table 10.1: WBS work packages for the sorter prototype build.

ID	Work package	Duration (d)	Predecessors
A	PCB design	3	—
B	Component procurement	7	A
C	PCB fabrication	5	A
D	PCB assembly	2	B, C
E	Firmware integration	4	A
F	System test	3	D, E

10.1.2.0.1 In Practice. WBS depth should match the team's ability to estimate and track. For a four-week prototype sprint, work packages of one to five days are practical; packages longer than a week are usually hiding sub-tasks that carry independent risks.

10.2 Dependencies and Critical Path

10.2.1 Activity-on-node notation

In the **Critical Path Method** (CPM) each work package is drawn as a node and each dependency as a directed arc from predecessor to successor. Figure 10.1 shows the six-activity network for the sorter build; the critical path is highlighted with bold arcs.

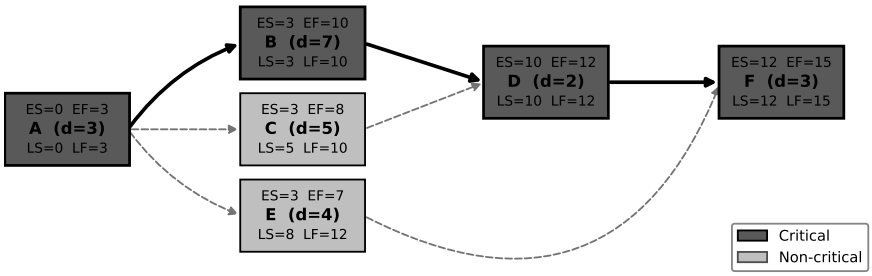

Figure 10.1: Activity-on-node CPM network for the sorter prototype build. Bold arcs mark the critical path A → B → D → F (15 days). Numbers inside each node show ES / EF on the top row and LS / LF on the bottom row.

10.2.2 Forward pass — earliest start and finish

The **early start** (ES) of an activity is the earliest day on which it can begin, given all predecessors are complete. The **early finish** (EF) is: $EF = ES +$ duration.

Processing the network from left to right:

(A) $ES_A = 0; \quad EF_A = 0 + 3 = 3$

(B) $ES_B = EF_A = 3; \quad EF_B = 3 + 7 = 10$

(C) $ES_C = EF_A = 3; \quad EF_C = 3 + 5 = 8$

(D) $ES_D = \max(EF_B,\, EF_C) = \max(10,\, 8) = 10; \quad EF_D = 10 + 2 = 12$

(E) $ES_E = EF_A = 3; \quad EF_E = 3 + 4 = 7$

(F) $ES_F = \max(EF_D,\, EF_E) = \max(12,\, 7) = 12; \quad EF_F = 12 + 3 = 15$

The **project duration** is $T = EF_F = \mathbf{15}$ days.

10.2.3 Backward pass — latest start and finish

The **late finish** (LF) of an activity is the latest day by which it must finish without delaying the project. The **late start** (LS) is: $LS = LF -$ duration.

Processing right to left (setting $LF_F = 15$):

(F) $LF_F = 15; \quad LS_F = 15 - 3 = 12$

(D) $LF_D = LS_F = 12; \quad LS_D = 12 - 2 = 10$

(E) $LF_E = LS_F = 12; \quad LS_E = 12 - 4 = 8$

(C) $LF_C = LS_D = 10; \quad LS_C = 10 - 5 = 5$

(B) $LF_B = LS_D = 10; \quad LS_B = 10 - 7 = 3$

(A) $LF_A = \min(LS_B,\, LS_C,\, LS_E) = \min(3,\, 5,\, 8) = 3; \quad LS_A = 3 - 3 = 0$

10.2.4 Slack and the critical path

Slack (also called float) is the amount by which an activity can be delayed without extending the project:

$$\text{slack} = LS - ES = LF - EF \tag{10.1}$$

Applying Equation 10.1 to each activity:

Table 10.2: CPM results for the sorter prototype build.

ID	Package	Dur	ES	EF	LS	LF	Slack	Critical?
A	PCB design	3	0	3	0	3	0	**Yes**
B	Component procurement	7	3	10	3	10	0	**Yes**
C	PCB fabrication	5	3	8	5	10	2	No
D	PCB assembly	2	10	12	10	12	0	**Yes**
E	Firmware integration	4	3	7	8	12	5	No
F	System test	3	12	15	12	15	0	**Yes**

The **critical path** consists of every activity with zero slack. Here that is **A** $\rightarrow$ **B** $\rightarrow$ **D** $\rightarrow$ **F**, with a total duration of $3 + 7 + 2 + 3 = \mathbf{15}$ days.

The competing path A $\rightarrow$ C $\rightarrow$ D $\rightarrow$ F spans only $3 + 5 + 2 + 3 = 13$ days, giving C a slack of 2 days. Path A $\rightarrow$ E $\rightarrow$ F spans $3 + 4 + 3 = 10$ days, giving E a slack of 5 days.

10.2.4.0.1 Common Pitfalls. A common error is to identify the critical path as the sequence containing the *longest single task*. Activity B (7 days) is the longest task, but the critical path is defined by *total path length*, not individual activity duration. A short activity with zero slack (e.g., PCB assembly at 2 days) is just as critical as the longest one.

10.3 Gantt Scheduling

A **Gantt chart** maps each work package to a horizontal bar on a calendar timeline. It communicates planned start and finish dates, parallel tracks, and milestones to every stakeholder — and it must be updated as actuals deviate from plan.

Figure 10.2 shows the Gantt chart for the sorter build derived directly from the CPM results in Table 10.2. Activities on the critical path (A, B, D, F) are shown with solid dark bars; non-critical activities (C, E) are shown with lighter bars and their slack is visible as an extension to the right of each bar. Milestones (PCB design complete at day 3; integration ready at day 12; system test complete at day 15) are marked with diamond symbols.

10.3.1 Keeping the Gantt alive

A Gantt chart is not a snapshot taken once at project kickoff; it is a **living document** updated at every weekly check-in. When Activity B (component procurement)

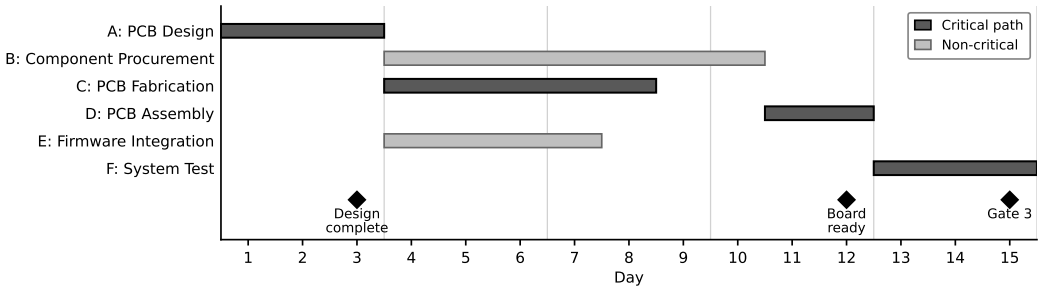

Figure 10.2: Gantt chart for the sorter prototype build (15 days). Solid dark bars are critical-path activities; lighter bars are non-critical activities. The dashed extensions to the right of C and E indicate their available slack (2 days and 5 days, respectively). Diamond markers denote milestones.

slips by even one day, the entire critical path extends by the same amount. Teams that file the Gantt after sprint planning and never revisit it are the ones who discover a two-week slip the day before their gate review.

10.3.1.0.1 In Practice. Use colour or line-weight to distinguish planned bars from actual progress bars. Add a *today* vertical line so any reader can immediately see what is late. In a four-person prototype team, a shared spreadsheet Gantt updated every Monday morning is sufficient; dedicated project management software adds value only when the WBS exceeds roughly 20 work packages.

10.3.1.0.2 Common Pitfalls. When the Gantt is updated but the critical path is not re-evaluated, the team gains false reassurance: the bars move but the slip remains invisible. Any change to a critical-path activity duration must trigger an immediate recalculation of the project end date.

10.4 PERT Three-Point Estimation

CPM uses single-point duration estimates. For activities with high uncertainty — particularly procurement — that single number conceals the real range of outcomes. The **Program Evaluation and Review Technique** (PERT) replaces the single estimate with three:

- a — **optimistic**: the duration if everything goes well.
- m — **most likely**: the mode of the distribution; the single best estimate.
- b — **pessimistic**: the duration if most things go wrong (short of catastrophe).

10.4.1 PERT Formulas

The **PERT expected duration** is a weighted average placing four times the weight on the most-likely estimate:

$$t_e = \frac{a + 4m + b}{6} \tag{10.2}$$

The **PERT standard deviation** quantifies the spread of the estimate:

$$\sigma = \frac{b - a}{6} \tag{10.3}$$

A large σ warns the team that this activity is a schedule risk even if the expected duration looks acceptable.

10.4.2 Worked example: Activity B (component procurement)

Component procurement is the highest-uncertainty activity on the critical path. The team estimates:

- $a = 4$ days (supplier has parts in stock, ships next day)
- $m = 7$ days (typical online order with standard shipping)
- $b = 14$ days (back-order delay or customs hold)

Applying Equation 10.2:

$$t_e = \frac{4 + 4(7) + 14}{6} = \frac{4 + 28 + 14}{6} = \frac{46}{6} \approx 7.67 \text{ days}$$

The CPM single-point estimate of 7 days is slightly optimistic; the PERT-adjusted estimate of 7.67 days should be used in any schedule risk analysis.

Applying Equation 10.3:

$$\sigma = \frac{14 - 4}{6} = \frac{10}{6} \approx 1.67 \text{ days}$$

A standard deviation of 1.67 days on a critical-path activity means there is a realistic scenario in which the project finishes two or more days late. This uncertainty must appear in the risk register (Section 10.6).

Figure 10.3 illustrates the three-point distribution for Activity B with t_e and σ annotated.

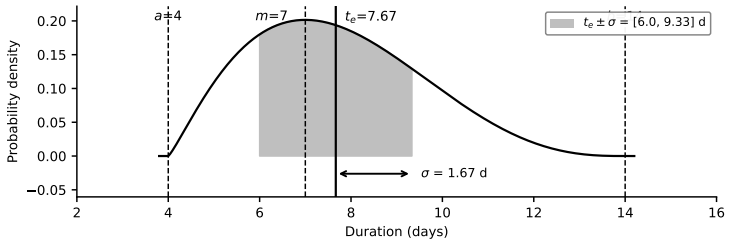

Figure 10.3: PERT distribution for Activity B (component procurement) with $a = 4$ d, $m = 7$ d, $b = 14$ d. The expected duration $t_e \approx 7.67$ d is shifted right of the mode because the pessimistic tail is longer. The $\pm 1\sigma$ band (≈ 1.67 d) spans from roughly 6.0 d to 9.34 d.

10.4.2.0.1 In Practice. The asymmetry in the PERT formula ($t_e > m$ when the pessimistic tail is longer) is intentional and well-founded: in practice, activities almost always run long rather than short. When gathering three-point estimates from team members, ask for the pessimistic estimate last — people systematically underestimate bad scenarios when they anchor on the most-likely value first.

10.5 Lead Time as a Planning Risk

Lead time is the elapsed calendar time between placing a component order and having parts in hand. It is not "somebody else's problem" — it is a first-class **schedule risk** that must be modelled explicitly in the network.

10.5.1 Why lead time matters on the critical path

In the sorter schedule, Activity B (component procurement, $t_e = 7.67$ d) sits on the critical path. If the team waits until PCB design is complete before placing an order, every day of procurement delay directly delays system test. There is no buffer.

Several mitigation strategies are available:

1. **Preliminary BOM early.** Draft a provisional Bill of Materials during detailed design (Chapter 11) and place orders before Activity A is finished. This converts B from a strict successor of A into a partial overlap, shortening the critical path.
2. **Pre-qualify multiple suppliers.** If the primary supplier quotes a 14-day lead time, a secondary supplier at 7 days keeps the schedule intact.
3. **Order buffer stock.** For inexpensive passive components (resistors, capacitors) the cost of ordering 10% extra is negligible compared to a one-week slip caused by a mis-soldered component that cannot be replaced.
4. **Expedite only as a last resort.** Expediting fees are expensive; build the schedule so that expediting is never needed.

10.5.2 Long-lead items deserve their own register entries

Any component with a quoted lead time greater than the slack of its downstream path belongs in the risk register as its own entry, with a named owner and a specific mitigation plan. For the sorter, the microcontroller (if on back-order) and the motor driver IC are the most likely culprits.

10.5.2.0.1 Common Pitfalls. Teams frequently omit lead time from the schedule entirely, treating procurement as an instantaneous event. The result is a day-one crisis: design is "done" but parts will not arrive for ten days and the project duration instantly extends. Schedule the lead time explicitly, with the same three-point PERT estimate as any other uncertain activity.

10.6 Maintaining the Risk Register

A **risk register** is a living table that catalogues every identified threat to project objectives, its likelihood, its impact, and the mitigation strategy the team has chosen. It was introduced in Stage 2 with technical risks (Chapter 7). At Stage 3 it must grow to include **schedule risks** and **procurement risks** .

10.6.1 Standard register fields

Each row in the register should carry:

1. **ID** — a stable reference code (e.g., R-SCH-01).
2. **Description** — one sentence naming the risk event.
3. **Likelihood** — Low / Medium / High (or 1–5 scale).
4. **Impact** — Low / Medium / High (or 1–5 scale).
5. **Risk score** — Likelihood $\times$ Impact.
6. **Mitigation** — the specific action the team will take.
7. **Owner** — the team member responsible.
8. **Status** — Open / In progress / Closed.

10.6.2 Schedule-specific risks for the sorter build

Table 10.3: Gate 3 risk register entries for the sorter prototype schedule.

ID	Description	L	I	Mitigation
R-SCH-01	Component procurement (Act. B) exceeds 7 days due to supplier back-order, delaying system test.	M	H	Pre-qualify secondary supplier; place order before PCB design is finalised.
R-SCH-02	PCB fabrication house queue delay extends Act. C beyond 5 days.	L	M	Use local fab service with 3-day turnaround as fall-back; ≥ 2 days slack available.
R-SCH-03	Firmware integration (Act. E) blocked by undiscovered hardware bug discovered at Act. D, requiring design iteration.	M	H	Daily smoke-test of assembled PCB before firmware handoff; maintain 5 days slack on Act. E.
R-SCH-04	Gate 3 review scheduled on day 14; project duration is 15 days — one-day slip causes missed gate.	M	H	Move gate review to day 16; flag to supervisor immediately if Act. B slips by > 1 day.

10.6.3 How often to update

The risk register should be reviewed at every weekly team meeting, in the same session as the Gantt update. Risks change status: a component that was "in stock" last week may be back-ordered today. Close a risk only when its triggering condition can no longer occur; do not close it because the team has stopped worrying about it.

10.6.3.0.1 In Practice. At Gate 3 reviewers will scan the register for *completeness* (have schedule and procurement risks been added since Gate 2?) and *actionability* (do mitigations name a specific person and a specific action?). Vague entries such as "monitor the situation" are not mitigations; they are placeholders that signal the team has not thought through the risk.

Try It Yourself

The following five-activity network describes a sensor-node firmware release.

Table 10.4: Try It Yourself: firmware release work packages.

ID	Work package	Duration (d)	Predecessors
P	Requirements review	2	—
Q	Driver development	6	P
R	Unit testing	3	P
S	Integration testing	4	Q, R
T	Documentation & release	2	S

1. Perform the forward and backward pass to find ES, EF, LS, LF, and slack for every activity.
2. Identify the critical path and state the project duration.
3. Activity Q (driver development) has three-point estimates $a = 3$ days, $m = 6$ days, $b = 12$ days. Use Equations 10.2 and 10.3 to compute t_e and σ. Substitute t_e for the activity's duration, recompute the total length of each path through the network, and state whether the critical path changes. Show both path lengths explicitly.
4. Add one schedule risk for this network to a risk register, following the format of Table 10.3.

(No answer key provided.)

Review Questions

1. **(Conceptual)** Explain in your own words why the critical path is defined by activities with zero slack rather than by the activity with the longest duration.

Give an example where the longest individual activity is *not* on the critical path.

2. **(Conceptual)** A team builds a Gantt chart on the first day of the sprint and does not update it for three weeks. Identify two specific ways in which this practice can cause a gate failure.

3. **(Applied)** A project has four activities: W (4 d, no predecessors), X (3 d, predecessor W), Y (6 d, predecessor W), Z (2 d, predecessors X and Y). Compute ES, EF, LS, LF, and slack for each activity. State the critical path and project duration.

4. **(Applied)** Activity Y above has three-point estimates $a = 5$ days, $m = 6$ days, $b = 10$ days. Compute t_e and σ using Equations 10.2 and 10.3. If the PERT-adjusted duration is used in place of 6 days, does the critical path change? Justify your answer numerically.

5. **(Applied — multi-step)** A project has five activities: J (3 days, no predecessors); K (6 days, predecessor J); L (4 days, predecessor J); M (2 days, predecessors K and L); N (3 days, predecessor M).

   (a) Perform the forward and backward pass. Record ES, EF, LS, LF, and slack for every activity.

   (b) State the critical path and project duration.

   (c) Activity K has three-point estimates $a = 4$ days, $m = 6$ days, $b = 13$ days. Compute t_e and σ using Equations 10.2 and 10.3.

   (d) Write a complete eight-field risk-register entry for the risk that Activity K exceeds its pessimistic estimate.

Portfolio Entry

Gate 3 deliverables produced in this chapter:

(a) **Gantt chart** — A calendar-based bar chart showing all six work packages, their planned start and finish days, the critical path highlighted, slack extensions on non-critical activities, and milestone markers at days 3, 12, and 15. The chart is a living document; it must be updated at every team meeting with actual progress.

(b) **Updated risk register** — The Stage 2 technical risk register (from Chapter 7) extended with the four schedule and procurement risks in Table 10.3. Each new entry carries a unique R-SCH-XX identifier, a named owner, and a specific — not vague — mitigation action. Submit the register as part of the Gate 3 package alongside the Gantt chart.

Chapter Summary

This chapter transformed the sorter team's verified design into a buildable, trackable plan:

1. A **Work Breakdown Structure** decomposed the build into six atomic work packages (A through F), each assignable and estimable.
2. The **CPM forward and backward pass** produced early/late start and finish times for every activity. Applying Equation 10.1 revealed that A, B, D, and F carry zero slack, making $A \rightarrow B \rightarrow D \rightarrow F$ the 15-day critical path.
3. A **Gantt chart** (Figure 10.2) mapped the schedule to a calendar timeline. It must be updated weekly; a static Gantt is a liability, not an asset.
4. **PERT** three-point estimation (Equations 10.2 and 10.3) quantified the uncertainty in Activity B: $t_e \approx 7.67$ days, $\sigma \approx 1.67$ days — a warning that the critical path carries real schedule risk.
5. **Lead time** is a first-class schedule risk: placing orders before PCB design is finalised and pre-qualifying secondary suppliers are the primary mitigations.
6. The **risk register** was extended with four schedule-specific entries (Table 10.3), meeting the Gate 3 completeness requirement.

You now have a Gantt chart and an updated risk register — the two deliverables required for Gate 3 sign-off. Chapter 11 takes that risk register and the project schedule as its starting point and builds the full Bill of Materials and procurement chain, turning the component list implicit in Activity B into a concrete sourcing plan with part numbers, quantities, costs, and supplier alternatives.

CHAPTER 11

BILL OF MATERIALS & PROCUREMENT

11.0.0.0.1 Where this fits. This chapter operates at Stage 3 (Development), targeting Gate 3. The Stage-Gate model requires that every Gate 3 review package include a *costed* BOM: a document that ties every physical part to a quantity, a unit price, a supplier, and a lead time. An uncosted or incomplete BOM cannot support the business case that Gate 3 demands — no reviewer can approve a prototype budget without knowing what it costs to build.

11.0.0.0.2 Learning Objectives. After completing this chapter you will be able to:

1. Build a multi-level costed BOM with correct reference designators, quantities, unit costs, extended costs, and supplier information.
2. Derive a purchase list from the BOM and compare supplier quotes on both unit price and lead time.
3. Manage the full procurement document chain: purchase order, receipt, invoice verification, and as-built reconciliation.

Knowing *when* work happens (the Gantt chart from Chapter 10) and what could go wrong (the risk register) is only actionable once you know what to order, from whom, at what price, and within what lead time. This chapter provides that foundation.

11.1 BOM as a Design Document

A **bill of materials** (BOM) is not a shopping list. It is a design document that captures the complete set of physical parts required to build one unit of the system, together with the quantities, reference designators, unit costs, and supply-chain metadata necessary to procure and assemble those parts. The distinction matters: a shopping list can be incomplete without consequences until you arrive at the checkout; an incomplete BOM discovered during assembly halts production, delays the schedule, and invalidates cost estimates already submitted to the Gate 3 review.

11.1.1 BOM Levels

A **multi-level BOM** structures the system hierarchically: the top level names the assembled product; the second level names subsystems or subassemblies; the lowest level names individual purchased or manufactured components. This structure mirrors the system architecture developed in Chapter 5 and makes it possible to roll costs up by subsystem, identify which assembly contributes most to total cost, and manage procurement independently for each assembly team.

Figure 11.1 shows the multi-level BOM tree for the IoT robotic sorting subsystem. The top level is the complete sorter. The second level contains three subsystems: Control, Actuation, and Structure. The third level contains the individual line items.

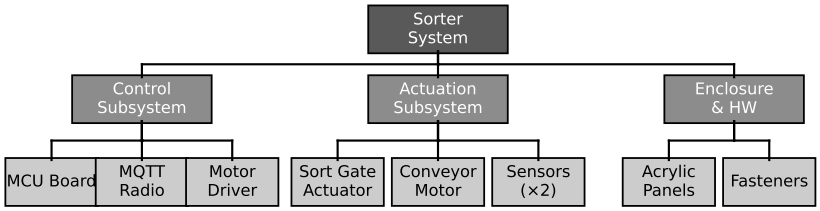

Figure 11.1: Multi-level BOM tree for the IoT robotic sorting subsystem. The top-level assembly (Sorter) decomposes into three subsystems (Control, Actuation, Structure), each of which decomposes into individual purchased components at Level 3. Reference designators (U1, U2, Q1, M1, B1, B2, H1) appear at the leaf nodes.

11.1.2 BOM Fields

Every BOM line requires at minimum:

(a) **Reference designator** — the unique identifier linking the BOM line to the schematic or assembly drawing (e.g., U1, Q1, M1). A line without a designator cannot be placed on a board or found in an assembly drawing.

(b) **Description and part number** — the manufacturer part number (MPN) is the only unambiguous identifier. A description without an MPN allows the assembler to substitute any part that "sounds like" the intended one.

(c) **Quantity per assembly** (q_i) — the number of that part needed to build one unit.

(d) **Unit cost** (u_i, in CAD) — the price paid per single piece at the procurement quantity.

(e) **Extended cost** — $q_i \times u_i$, the total spend on that line for one assembly.

(f) **Preferred supplier and part number** — the distributor from whom the part will be ordered, with the distributor's stock number.

(g) **Lead time** — the calendar days from order placement to receipt, used to back-schedule the procurement milestone in the Gantt chart.

(h) **Alternate part** — a second-source MPN that satisfies parametric equivalence and footprint compatibility, mitigating single-source risk identified in the risk register.

11.1.2.0.1 In Practice. A team building the sorter prototype omitted reference designators from the BOM because the schematic had not been finalised when the BOM was started. At assembly, the PCB had two footprint-compatible motor drivers with different pinouts. Without the designator, the assembler soldered the wrong driver onto U2, applying power in the wrong sequence and destroying the part. The loss was $18 and two days of schedule. Reference designators must be locked before the BOM is submitted, not after.

11.1.3 The Alternate-Part Column

Single-sourced components are a procurement liability. If the sole supplier for a critical part is out of stock, the project stalls. Every BOM line for a component that is not a commodity (i.e., not a standard resistor, capacitor, or fastener available from every distributor) should carry a vetted alternate: a second MPN confirmed to be parametrically equivalent, footprint-compatible, and in Active lifecycle status. Chapter 6 described the process for qualifying a second source; the BOM is where that work is recorded and made visible to reviewers.

11.2 Purchase List from BOM

A **purchase list** is derived from the BOM by collapsing identical MPNs that appear in different subsystems into a single line and then computing the order quantity for each line. The purchase list is the document you send to a supplier or purchasing department; it is not the BOM itself.

11.2.1 Order Quantity with Overage

For passive components and consumables — resistors, capacitors, solder, heat-shrink — the risk of loss during assembly justifies ordering more than the exact quantity the BOM requires. The **order quantity with overage** is:

$$q_i^{\text{order}} = \lceil (1 + \alpha)\, q_i \rceil \tag{11.1}$$

where q_i is the BOM quantity, α is the fractional overage (a dimensionless value between 0 and 1, e.g., $\alpha = 0.10$ for 10% overage), and $\lceil \cdot \rceil$ denotes the ceiling function (round up to the nearest whole unit). The ceiling is necessary because you cannot order a fractional part.

Overage is *not* applied blindly to every line. Expensive unique components — the MCU board, the motor driver IC, the enclosure — are ordered to exact BOM quantity, possibly with one spare unit explicitly approved by the project manager.

Applying 10% overage to a $65 MCU board would order an unneeded extra unit and misrepresent the budget.

11.2.1.0.1 Common Pitfalls. Teams sometimes apply overage to the BOM quantity used for cost estimation, inflating C_{BOM}. Overage belongs only on the purchase list, applied to q_i^{order}; the BOM cost calculation always uses the exact per-assembly quantity q_i.

11.3 Quotes and Supplier Comparison

A **request for quotation** (RFQ) specifies the MPN, the order quantity, the required delivery date, and the delivery address; send one to at least two suppliers for every non-commodity line item before committing to a purchase order.

11.3.1 What to Compare

Quote comparison is not a simple lowest-price sort. The relevant criteria are:

(a) **Unit price at your order quantity** — distributor pricing is typically tiered; the unit price at quantity 1 may be three times the price at quantity 10. Always quote at the quantity you intend to order.

(b) **Lead time** — a part that is $2.00 cheaper but arrives four weeks later may miss the assembly milestone and delay Gate 3. Lead time must be evaluated against the Gantt chart, not in isolation.

(c) **Stock availability** — a listed unit price is irrelevant if the distributor has zero units in stock. Confirm "in-stock" quantity at time of RFQ.

(d) **Minimum order quantity** (MOQ) — some distributors impose a minimum order of 5 or 10 units for certain components. The effective cost per unit changes when the MOQ exceeds your order quantity.

(e) **Shipping and handling** — a $1.50 unit price advantage is erased by a $15 shipping surcharge on a small order.

The procurement document chain that connects the BOM to payment is shown in Figure 11.2.

11.4 Purchase Orders

A **purchase order** (PO) is a formal document that authorises a supplier to ship specific goods at a specific price. It is a legal instrument: once a supplier accepts a PO, both parties are bound by its terms. For student projects, purchase orders are often replaced by a supervisor-approved purchase requisition or a direct online order; the discipline of treating that order as a PO — with a unique PO number, a

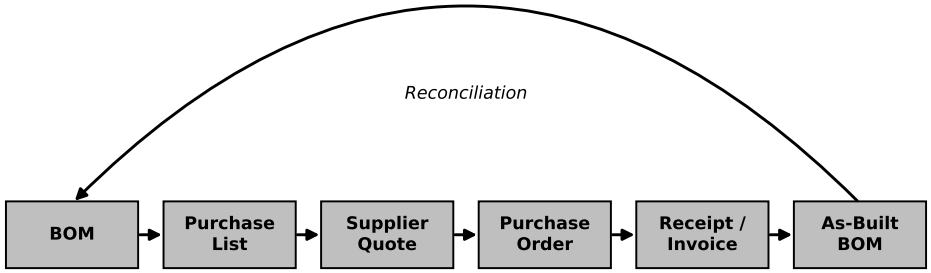

Figure 11.2: Procurement document chain for the sorter prototype. Flow proceeds left to right: the costed BOM feeds the purchase list; the purchase list generates RFQs to suppliers; selected quotes become purchase orders (POs); POs trigger shipments that produce a packing slip and invoice; receipt verification produces the as-built BOM through reconciliation. Each arrow represents a document hand-off with a defined format and responsible party.

line-item list, and a price commitment — is nonetheless valuable and expected in industry.

11.4.1 Required PO Fields

A well-formed PO contains:

(a) **PO number** — a unique identifier used to cross-reference the invoice and packing slip.

(b) **Issue date and required-by date**.

(c) **Line items**: supplier part number, MPN, description, ordered quantity, agreed unit price, and line total.

(d) **Ship-to address** and delivery instructions.

(e) **Payment terms** (e.g., Net 30, credit card on shipment).

(f) **Authorising signature or approval** — in a student lab context, this is the instructor or technician approval on the requisition form.

11.4.1.0.1 In Practice. PO numbers serve a critical traceability function. When two packages arrive on the same day from the same distributor, the packing slip PO reference is the only reliable way to match each package to its originating order without opening and counting every item. Assign PO numbers sequentially from the project start and record them in the procurement log.

11.5 Receipt, Invoice, and Verification

Receiving a shipment is not the end of the procurement process; it is the beginning of the verification step. Every received shipment must be checked against three documents: the PO, the packing slip, and the invoice.

11.5.1 Three-Way Match

The **three-way match** is the standard industry procedure for approving payment on a received order:

(a) **PO vs. packing slip**: confirm that the items shipped (part numbers, quantities) match what was ordered.

(b) **Packing slip vs. physical contents**: confirm that what arrived physically matches what the packing slip claims.

(c) **PO vs. invoice**: confirm that the prices billed match the prices committed in the PO.

Any discrepancy — a short shipment, a substituted part, a price that does not match — must be resolved with the supplier before payment is authorised and before the part enters the build. A substituted part may have a different footprint or electrical characteristic than the one ordered; using it without verification can damage the board.

11.5.2 Physical Inspection

For mechanical and electronic components, physical inspection after receipt includes:

(a) Checking packaging integrity (moisture-barrier bags for ICs, ESD-safe packaging for sensitive components).

(b) Reading the manufacturer label on the tape reel or bag to confirm MPN and date code.

(c) Measuring one sample from each lot with a multimeter or LCR meter where practical (resistor value, capacitor value, continuity).

Counterfeit components are a real risk when ordering from grey-market sources. Sticking to authorised distributors (Digi-Key, Mouser, Arrow) eliminates this risk for prototype quantities.

11.6 Reconciliation and the As-Built BOM

The **as-built BOM** records the design as it was actually built: every substitution, lot code, and quantity deviation from the original design BOM is captured here. It is created by reconciling the design BOM against the procurement records (POs, invoices, packing slips) and the physical build records (assembly log, rework log).

11.6.1 Why Reconciliation Matters

If a part was substituted during procurement — because the preferred part was out of stock — the substitution must be captured in the as-built BOM so that a future builder replicating the prototype knows which part was actually used and tested. An as-built BOM that simply says "same as design BOM" is only credible if the three-way match confirmed no substitutions for every line.

Reconciliation also closes out the budget. Compare the total actually spent (sum of invoice line totals) against the C_{BOM} estimate. Any variance larger than 5 % warrants an explanation: price change, MOQ forcing excess purchase, or a substitution at a different price point. This variance record feeds the project's financial close-out report and informs cost estimates for the next project.

11.6.1.0.1 In Practice. A sorter prototype team ordered the preferred motor driver at $7.40/unit. The part went out of stock after the PO was placed; the supplier substituted an alternate at $9.15/unit without notifying the team. The three-way match caught the price discrepancy. The team confirmed the alternate was parametrically compatible (same current rating, same footprint), approved the substitution, updated the as-built BOM, and logged the $1.75/unit variance. Without the three-way match, the variance would have been discovered only at budget close-out — too late to question or contest.

11.7 Worked Example: Sorter BOM

This section builds the complete costed BOM for one unit of the IoT robotic sorting subsystem, derives the cost equations, and works through a supplier quote comparison for one line item.

11.7.1 The Eight-Line Sorter BOM

The sorter contains the following components at the third level of the BOM hierarchy (see Figure 11.1):

(a) **U1** — MCU board (ESP32-DevKitC-32E, Espressif)

(b) **U2** — MQTT radio module (ESP-WROOM-32U, Espressif variant with external antenna connector)

(c) **Q1** — Motor driver IC (DRV8833PWPR, Texas Instruments)

(d) **M1** — Conveyor motor (Pololu 37D gear motor, 12 V, 150 rpm, Pololu 4759)

(e) **K1** — Sort-gate actuator (HS-82MG micro servo, HiTec)

(f) **B1** — Colour sensor (TCS34725FN, ams-OSRAM)

(g) **B2** — Proximity sensor (VL53L0X ToF module, STMicroelectronics)

(h) **H1** — Enclosure hardware kit (M3 standoffs, screws, nuts — assorted 50-piece kit)

With the line items defined, the cost equations follow directly.

The **extended cost** for line i is:

$$\text{ext}_i = q_i \times u_i \tag{11.2}$$

where q_i is the per-assembly quantity and u_i is the unit cost in CAD.

The **BOM total** C_{BOM} sums all extended costs:

$$C_{\text{BOM}} = \sum_{i=1}^{N} q_i \times u_i \tag{11.3}$$

Table 11.1 presents the eight-line BOM with all fields populated. Costs are in Canadian dollars at single-unit distributor pricing as of the course edition date.

Table 11.1: Costed BOM for the IoT robotic sorting subsystem (one assembly unit). Unit and extended costs in CAD. Lead times from preferred supplier. Ref: reference designator; Qty: per-assembly quantity; UC: unit cost; EC: extended cost.

Ref	Description / MPN	Qty	UC (CAD)	EC (CAD)	Supplier	Lead time
U1	ESP32-DevKitC-32E	1	12.50	12.50	Digi-Key	3 days
U2	ESP-WROOM-32U (ext. ant.)	1	8.75	8.75	Mouser	5 days
Q1	DRV8833PWPR motor driver	1	3.20	3.20	Digi-Key	3 days
M1	Pololu 4759 gear motor	1	54.95	54.95	Pololu	7 days
K1	HS-82MG micro servo	1	22.40	22.40	RobotShop	4 days
B1	TCS34725FN colour sensor	1	6.80	6.80	Mouser	5 days
B2	VL53L0X ToF proximity module	1	11.30	11.30	Digi-Key	3 days
H1	M3 hardware kit (50-pc)	1	4.95	4.95	Amazon CA	2 days
			C_{BOM}	**124.85**		

Equation 11.2 applied to each line yields the extended costs shown in Table 11.1. Summing by Equation 11.3:

$$C_{\text{BOM}} = 12.50 + 8.75 + 3.20 + 54.95 + 22.40 + 6.80 + 11.30 + 4.95 = \$124.85 \text{ CAD}$$

The subsystem cost breakdown is: Control (U1 + U2 + Q1 + B1 + B2) = \$42.55; Actuation (M1 + K1) = \$77.35; Structure (H1) = \$4.95. The cost rollup by subsystem is shown in Figure 11.3.

11.7.2 Overage Application

Applying $\alpha = 0.10$ (10% overage) using Equation 11.1 to the consumable and passive line:

(a) H1 (enclosure hardware kit, fasteners) — $q_{\text{H1}} = 1$, $q_{\text{H1}}^{\text{order}} = \lceil 1.10 \times 1 \rceil = \lceil 1.10 \rceil = 2$

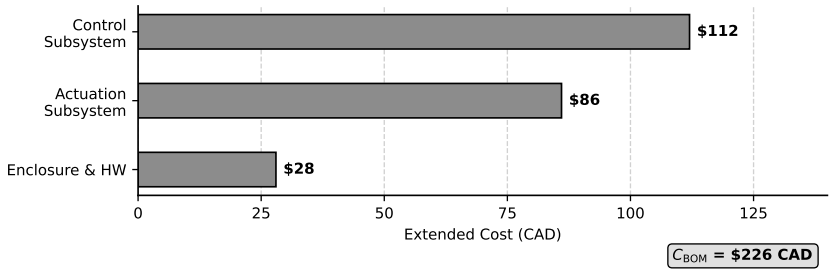

Figure 11.3: BOM cost rollup by subsystem for the IoT robotic sorting subsystem. Horizontal bars show the contribution of each subsystem to the $C_{\text{BOM}} = \$124.85$ CAD total. Actuation dominates at \$77.35 (62 %), driven primarily by the gear motor (M1 at \$54.95). Control contributes \$42.55 (34 %) and Structure \$4.95 (4 %).

The hardware kit is the only line where overage applies in this BOM: the electronic modules are ordered to exact quantity (1 each), and one spare unit is noted separately in the risk register rather than inflating the BOM cost. The purchase list therefore shows H1 at quantity 2 ($2 \times \$4.95 = \$9.90$) while the BOM cost calculation retains $q_{\text{H1}} = 1$, $\text{ext}_{\text{H1}} = \$4.95$.

11.7.3 Supplier Quote Comparison: MQTT Radio Module (U2)

Two distributors were asked to quote the ESP-WROOM-32U at quantity 1.

Table 11.2: Supplier quotes for U2 (ESP-WROOM-32U, Qty 1). All prices in CAD including applicable taxes and standard shipping to the lab address.

Supplier	Unit price (CAD)	Lead time	Notes
Mouser Electronics	\$8.75	5 business days	In stock (427 units); standard shipping included
DigiParts CA	\$7.90	14 business days	Ships from overseas warehouse; no local stock

DigiParts CA offers a unit price \$0.85 lower than Mouser. However, the lead time of 14 business days (approximately three calendar weeks) places delivery after the assembly milestone on the Gantt chart from Chapter 10. Missing that milestone delays Gate 3 by at least two weeks.

Preferred choice: Mouser Electronics. The \$0.85 price advantage of DigiParts CA is outweighed by the lead-time risk. At a project cost of \$124.85 per unit, the \$0.85 saving represents 0.7 % of BOM cost — an amount that is not worth delaying the project for.

The general procurement principle this illustrates: *lead time is a schedule input, not a footnote*. Any supplier whose lead time threatens a milestone date must be disqualified regardless of unit price, unless the project manager explicitly approves the schedule extension.

11.7.3.0.1 Common Pitfalls. The four most common BOM and procurement errors on prototype projects are:

(a) **BOM as an afterthought.** Starting the BOM after assembly begins means ordering parts under time pressure, paying expedite fees, and finding missing parts only when a build step stalls. The BOM should be started during architecture (Chapter 5) and completed before procurement begins.

(b) **No alternates listed.** A BOM with no alternate part column is a single-sourced BOM. When the preferred part is out of stock — which happens routinely with popular ICs — the project stalls while the team qualifies an alternate under pressure.

(c) **Missing reference designators.** Ordering a part without confirming the footprint against the reference designator on the PCB has no recovery path except rework or board respins. The reference designator column in the BOM is the link between the procurement document and the assembly drawing.

(d) **Single-sourced parts with no risk register entry.** A component that can only be purchased from one supplier, with no qualified alternate and no risk register entry flagging the exposure, is an unmanaged risk. Every single-source line in the BOM should have a corresponding risk register entry with a mitigation (order extra stock, qualify an alternate before production).

Try It Yourself

Your team is prototyping a battery-powered environmental sensor node. The design BOM has six line items (one assembly unit):

Table 11.3: Design BOM for the environmental sensor node (Try It Yourself). Unit costs in CAD at single-unit pricing.

Ref	Description / MPN	Qty	UC (CAD)	Supplier
U1	ATSAMD21G18A-MU MCU	1	6.30	Digi-Key
U2	RFM95W LoRa radio module	1	9.45	Mouser
U3	BME680 environmental sensor	1	7.20	Digi-Key
BT1	18650 Li-ion cell (2500 mAh)	2	4.80	BatterySpace
C1	100 µF electrolytic cap (assorted)	5	0.30	Digi-Key
H1	Enclosure + mounting hardware	1	8.75	Amazon CA

1. Calculate the extended cost for each of the six lines using Equation 11.2.
2. Calculate C_{BOM} using Equation 11.3.
3. Apply an overage of $\alpha = 0.15$ (15%) to the capacitor line (C1) only, using Equation 11.1. What is $q_{\text{C1}}^{\text{order}}$?
4. A second supplier quotes the RFM95W at $8.10 CAD but with a lead time of 18 business days (your assembly milestone is in 10 business days). Identify the preferred supplier and justify your choice using both unit price and lead time criteria.
5. List two fields this BOM is missing that would be required for a Gate 3 submission.

Review Questions

1. **(Conceptual)** Explain the difference between a BOM and a purchase list. Under what circumstances does a purchase list differ from the BOM in both line count and quantity per line?
2. **(Conceptual)** Why is an as-built BOM required even when the assembled unit matches the design BOM exactly? What does the as-built BOM provide that the design BOM does not?
3. **(Applied)** A BOM has a line item for 0.1 µF MLCC capacitors with $q = 12$ and $u = \$0.08$ CAD.
   (a) Calculate the extended cost using Equation 11.2.
   (b) Calculate the order quantity with $\alpha = 0.10$ using Equation 11.1.
4. **(Applied)** A BOM has five line items with extended costs of \$24.50, \$67.00, \$8.40, \$13.75, and \$5.90 (all CAD). Calculate C_{BOM} using Equation 11.3 and identify which single line contributes the greatest fraction of total cost.
5. **(Multi-step)** Supplier A quotes a microcontroller at \$11.20 CAD with a 4-business-day lead time and 50 units in stock. Supplier B quotes the same MPN at \$9.85 CAD with a 12-business-day lead time and 200 units in stock. Your assembly milestone is 7 business days from today and you need 3 units.
   (a) Calculate the total cost from each supplier for 3 units.
   (b) Identify the preferred supplier with explicit reasoning that addresses both price and schedule.
   (c) Describe one scenario in which the higher-priced supplier would *not* be preferred despite meeting the schedule.

Portfolio Entry

The Gate 3 deliverable from this chapter is a procurement package consisting of three documents submitted together:

(a) **Costed BOM**: all fields populated (reference designator, MPN, description, quantity, unit cost, extended cost, preferred supplier, distributor part number, lead time, alternate part).

(b) **Purchase list**: one row per unique MPN with order quantities reflecting overage where applicable, derived from the BOM as described in Section 11.2.

(c) **Procurement records**: for completed purchases, the set of POs, packing slips, and invoices with the three-way match completed; for pending purchases at Gate 3 submission, the set of accepted quotes and issued POs.

A Gate 3 reviewer who cannot find a unit cost, a lead time, or an alternate part for any line item in the costed BOM will flag the submission as incomplete. The BOM is a design document; an incomplete design document is not ready for review.

Chapter Summary

The bill of materials functions as a design document: its completeness is a precondition for Gate 3 approval, not a post-assembly record. It records the full set of physical parts needed for one assembled unit — reference designators, MPNs, quantities, unit costs, extended costs, supplier information, lead times, and alternate parts — and makes the cost and supply-chain risk of the design visible to every reviewer. The **extended cost** for each line is $\text{ext}_i = q_i \times u_i$ (Equation 11.2). The **BOM total** is $C_{\text{BOM}} = \sum q_i \times u_i$ (Equation 11.3). Order quantities for consumables are padded by an overage factor α using $q_i^{\text{order}} = \lceil (1 + \alpha)\, q_i \rceil$ (Equation 11.1), but overage belongs only on the purchase list — never in the BOM cost calculation.

Supplier selection weighs unit price, lead time, stock availability, MOQ, and shipping; the cheapest quote is not always the correct choice when lead time threatens a Gantt milestone. The procurement document chain — BOM, purchase list, RFQ, PO, packing slip, invoice, three-way match, as-built BOM — provides traceability from design intent to physical assembly and closes out the procurement budget with a recorded variance.

You now have a costed BOM, a purchase list, and a set of procurement records. Chapter 12 takes C_{BOM} as its starting point and asks what happens to unit cost when the design scales from one prototype to low-volume production — identifying the cost-reduction leverage points a Gate 4 reviewer will require.

CHAPTER 12

LOW-VOLUME PRODUCTION ECONOMICS

12.0.0.0.1 Where this fits. Chapter 11 delivered a validated prototype and a costed Bill of Materials for the IoT robotic sorting subsystem; this chapter sits at Gate 3 of the Stage-Gate model and fills the gap between that one-off prototype cost and the production economics that underpin the go/kill decision. A team that passes Gate 3 has demonstrated a working prototype and a credible BOM; the gate committee now asks a harder question: *can this product be made and sold profitably?* The cost projection produced in this chapter is the economic half of that answer. Gate 3 reviewers will scrutinise your unit-cost curve and break-even analysis as carefully as they scrutinise your test results.

12.0.0.0.2 Learning Objectives.

1. Compute and plot the unit-cost curve $C_{\text{unit}}(N)$ from a prototype BOM and an estimated fixed cost.
2. Find the break-even volume N^* between two competing production processes.
3. Connect design decisions made at Stage 3 to downstream manufacturing cost, using the DFM/DFA framework introduced in Chapter 7.

12.1 Why Low-Volume Bridge Production Exists

Between a validated prototype and a full production run lies a zone that engineers call **bridge production** or **low-volume production**. Volumes in this zone are typically 10 to 500 units — too many to hand-build one at a time, too few to justify the tooling cost of high-volume manufacturing.

Bridge production serves three purposes. First, it generates the first real product units for pilot customers, whose feedback feeds directly back into the design before a large capital commitment is made. Second, it exposes assembly and yield problems that never appear in a single-unit prototype, where defects are caught and reworked by the same engineer who built the board. Third, it provides the actual cost data that replaces the estimates in the Gate 3 projection.

12.1.0.0.1 In Practice. Many hardware startups and college capstone projects skip bridge production entirely: they build one prototype, quote a retail price, and claim the product is viable. The prototype BOM is then mistaken for the unit cost. The result is a Gate 3 slide showing a $226 unit cost for a product priced at $300 — a margin of 25% that evaporates as soon as a contract manufacturer quotes assembly labour, packaging, and regulatory compliance.

12.2 Fixed vs. Variable Cost

Every cost in a production run belongs to one of two categories.

Variable cost (c_{var}) is the cost that changes in proportion to the number of units produced. Components, raw materials, and direct assembly labour are variable: build one more unit and you spend c_{var} more. For the sorting subsystem, Chapter 11 established $c_{\text{var}} = \$480$ CAD, which reflects component cost plus estimated assembly labour at production volume.

Fixed cost (C_{fixed}) is the cost paid once, regardless of how many units are produced. Tooling (injection-mould tools, PCB test fixtures, jigs), non-recurring engineering (NRE) charges from a contract manufacturer, certification fees, and software licences are fixed: doubling the run does not double these costs. For the sorting subsystem, $C_{\text{fixed}} = \$8{,}000$ CAD, which covers the PCB test fixture ($2,400), the injection-mould tool for the housing ($4,800), and NRE setup charges ($800).

Common Pitfalls. Three errors appear consistently in student Gate 3 submissions.

(a) **Treating prototype unit cost as production unit cost.** The prototype BOM ($226) includes retail component prices and one-off shipping. At volume, component prices drop and shipping is amortised; variable cost and prototype cost are different quantities.

(b) **Forgetting that c_{var} excludes fixed costs.** The formula for C_{unit} (see Section 12.3) adds fixed costs explicitly; if you fold tooling into your variable cost estimate you will double-count it.

(c) **Omitting hidden fixed costs.** Certification (CE, FCC, CSA) and first-article inspection are fixed costs that do not scale. A product sold in Canada and shipped to the US carries two regulatory budgets.

12.3 The Unit-Cost Curve

When C_{fixed} is spread over N units and c_{var} is paid per unit, the total cost of a run of N units is

$$C_{\text{total}}(N) = C_{\text{fixed}} + N \cdot c_{\text{var}}.$$

Dividing by N gives the **unit cost**:

$$C_{\text{unit}}(N) = \frac{C_{\text{fixed}}}{N} + c_{\text{var}}. \tag{12.1}$$

Equation 12.1 is a rectangular hyperbola in N. As $N \to \infty$, C_{unit} asymptotes to c_{var} from above: no matter how large the run, you cannot drive unit cost below the variable cost per unit. At $N = 1$, unit cost equals $C_{\text{fixed}} + c_{\text{var}}$: the entire tooling and setup burden falls on a single unit.

Figure 12.1 illustrates this relationship for the sorting subsystem.

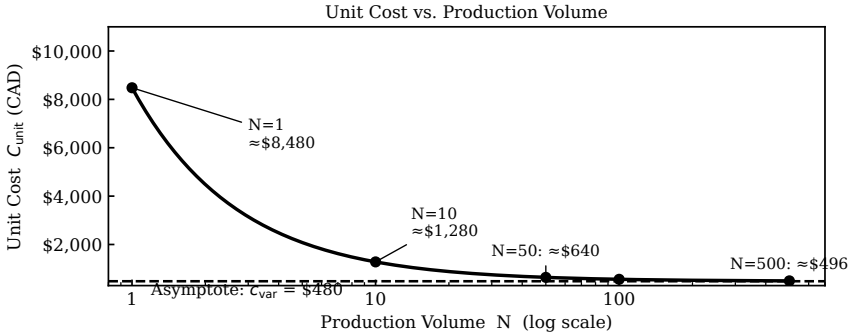

Figure 12.1: $C_{\text{unit}}(N)$ for the IoT sorting subsystem ($C_{\text{fixed}} = \$8{,}000$ CAD, $c_{\text{var}} = \$480$ CAD). The curve is a rectangular hyperbola that asymptotes to \$480 as volume grows. The dashed horizontal line marks the \$480 asymptote; the dotted vertical line marks $N = 80$, where the fixed-cost contribution falls to \$100 per unit.

12.3.1 Worked Example A: Computing the Unit-Cost Table

Given $C_{\text{fixed}} = \$8{,}000$ CAD and $c_{\text{var}} = \$480$ CAD, apply Equation 12.1 at five production volumes.

$N = 1$:

$$C_{\text{unit}}(1) = \frac{8{,}000}{1} + 480 = 8{,}000 + 480 = \$8{,}480 \text{ CAD}.$$

$N = 10$:

$$C_{\text{unit}}(10) = \frac{8{,}000}{10} + 480 = 800 + 480 = \$1{,}280 \text{ CAD}.$$

$N = 50$:

$$C_{\text{unit}}(50) = \frac{8{,}000}{50} + 480 = 160 + 480 = \$640 \text{ CAD}.$$

$N = 100$:

$$C_{\text{unit}}(100) = \frac{8{,}000}{100} + 480 = 80 + 480 = \$560 \text{ CAD}.$$

$N = 500$:

$$C_{\text{unit}}(500) = \frac{8{,}000}{500} + 480 = 16 + 480 = \$496 \text{ CAD}.$$

These five values are collected in Table 12.1.

Table 12.1: Unit cost at selected production volumes ($C_{\text{fixed}} = \$8{,}000$ CAD, $c_{\text{var}} = \$480$ CAD).

N	C_{fixed}/N	c_{var}	C_{unit}	Gross margin at $p = \$1{,}200$
1	\$8,000	\$480	\$8,480	−606 %
10	\$800	\$480	\$1,280	−6.7 %
50	\$160	\$480	\$640	46.7 %
100	\$80	\$480	\$560	53.3 %
500	\$16	\$480	\$496	58.7 %

Notice the rapid improvement between $N = 1$ and $N = 50$: the unit cost falls by \$7,840 in that interval, compared with only \$144 in the interval from $N = 50$ to $N = 500$. The curve is steep at low volume and flat at high volume — this is the economic signature of the hyperbola.

At what N does the fixed-cost contribution fall below \$100?

Set the fixed-cost contribution equal to \$100 and solve for N:

$$\frac{C_{\text{fixed}}}{N} = 100 \implies N = \frac{C_{\text{fixed}}}{100} = \frac{8{,}000}{100} = 80 \text{ units}.$$

At $N = 80$ the fixed cost adds exactly \$100 per unit; below 80 units each unit carries more than \$100 of tooling burden. This threshold is marked in Figure 12.1.

12.4 Process Break-Even

Different manufacturing processes carry different cost structures. A process with low fixed cost but high variable cost (such as 3D printing) is cheapest at low volume. A process with high fixed cost but low variable cost (such as injection moulding) is cheapest at high volume. The **process break-even volume** N^* is the production quantity at which the total cost of the two processes is equal.

For two processes with fixed costs C_{f1}, C_{f2} and variable costs c_{v1}, c_{v2} (where process 1 has the lower fixed cost and higher variable cost, $c_{v1} > c_{v2}$), set total costs equal:

$$C_{f1} + N^* c_{v1} = C_{f2} + N^* c_{v2}.$$

Solving for N^*:

$$N^* = \frac{C_{f2} - C_{f1}}{c_{v1} - c_{v2}}. \tag{12.2}$$

Figure 12.2 shows that below N^* the gap between the two cost lines widens rapidly, making early process selection consequential.

12.4.1 Worked Example B: 3D-Printing vs. Injection Moulding

The sorting subsystem housing can be produced by either process:

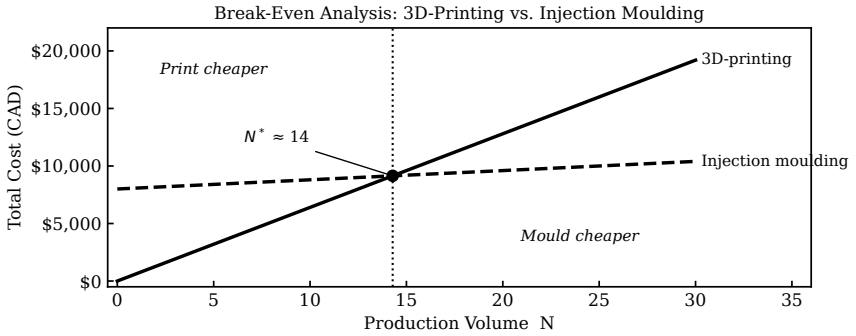

Figure 12.2: Total cost for 3D printing and injection moulding as a function of production volume. The lines cross at $N^* = 15$ units. Below 15 units the printed process is cheaper; above 15 units the moulded process is cheaper. The shaded zone around N^* represents the indifference region where process selection should be driven by lead-time, quality, and risk rather than cost alone.

- **Process 1 — FDM 3D printing:** no tooling required, $C_{f1} = \$0$ CAD; per-unit material plus machine time, $c_{v1} = \$640$ CAD.
- **Process 2 — injection moulding:** single-cavity aluminium tool, $C_{f2} = \$8{,}000$ CAD; cycle time and material per shot, $c_{v2} = \$80$ CAD.

Apply Equation 12.2:

$$N^* = \frac{C_{f2} - C_{f1}}{c_{v1} - c_{v2}} = \frac{8{,}000 - 0}{640 - 80} = \frac{8{,}000}{560} \approx 14.3 \text{ units.}$$

Because a fractional unit is not physically meaningful, round up: $N^* = 15$ units.

Decision rule:

- Below $N = 15$: use FDM printing. Total cost at $N = 10$: printing costs $0 + 10 \times 640 = \$6{,}400$; moulding costs $8{,}000 + 10 \times 80 = \$8{,}800$. Printing saves $2,400.
- Above $N = 15$: use injection moulding. Total cost at $N = 50$: printing costs $0 + 50 \times 640 = \$32{,}000$; moulding costs $8{,}000 + 50 \times 80 = \$12{,}000$. Moulding saves $20,000.

12.4.1.0.1 Common Pitfalls. N^* is not a sharp switch. In the neighbourhood of 15 units the cost advantage of either process is small (a few hundred dollars over the run), while process differences in lead time, surface finish, and dimensional tolerance can be decisive. Treat N^* as the centre of a zone of indifference — roughly $\pm 20\%$ around the calculated value — and let non-cost factors govern the decision within that zone.

12.5 DFM, DFA, and the Design-Locks-Cost Principle

Chapter 7 introduced Design for Manufacturability and Design for Assembly as Stage 3 disciplines. This chapter puts an exact dollar value on each.

The design-locks-cost principle states that approximately 70–80 % of the lifetime manufacturing cost of a product is determined by decisions made before the first prototype is built. Once a geometry is fixed, a component is specified, or an assembly sequence is locked in, the variable cost c_{var} is largely set. Subsequent process optimisation can reduce c_{var} by a few percent; only a redesign can change it significantly.

12.5.1 DFM Impact on Variable Cost

Each DFM decision made in Chapter 7 has a direct entry in c_{var}. Consider the FDM housing for the sorting subsystem.

- **Wall thickness at 1.5 mm** (above the 1.2 mm minimum): two complete perimeters plus a partial infill perimeter. Reducing wall thickness to the minimum saves approximately 8 % of material mass and roughly 12 minutes of print time per unit. At a machine rate of \$2.50/hr, this is a saving of \$0.50 per unit — small at 10 units (\$5 total), meaningful at 500 units (\$250 total).
- **Support-free cable channels**: eliminating support material saves post-processing labour. At \$20/hr labour rate and 8 minutes per unit, the saving is \$2.67 per unit, or \$1,335 over 500 units.

DFM decisions compound: a design with three or four such improvements can reduce c_{var} by 10–15 %, shifting the entire unit-cost curve downward.

12.5.2 DFA Impact on Assembly Cost

The **DFA assembly efficiency** η_A introduced in Chapter 7 quantifies the ratio of theoretically necessary assembly operations to actual operations. A low η_A means redundant parts, redundant fasteners, or awkward handling — all of which inflate the assembly-labour component of c_{var}.

For the sorting subsystem, the housing originally used four M3 screws to secure the PCB. Replacing these with two snap-fit latches (a DFA decision from Chapter 7) reduces the fastener count from four to zero, saves approximately 3 minutes of assembly time per unit, and removes four BOM line items. At 500 units this represents 25 hours of assembly labour — roughly \$500 in direct savings, plus the reduction in torque-tool calibration cost.

12.5.2.0.1 In Practice. DFM and DFA improvements are most valuable when they are identified at Stage 3, before the design is frozen. A snap-fit geometry that replaces screws is trivial to implement in CAD before the mould tool is cut. It is expensive to retrofit after the tool exists: a mould modification ("steel-safe" change) costs \$500–\$2,000 and adds 2–4 weeks to the schedule. This is the practical meaning of the design-locks-cost principle.

12.6 Failure Economics

A unit-cost analysis that excludes failure rates is optimistic. **Yield** is the fraction of produced units that pass acceptance testing and ship to customers. A yield of 95% means 5 out of every 100 units are scrapped or reworked, adding their cost to the run without adding to revenue.

The **effective variable cost** under a yield Y is

$$c_{\text{eff}} = \frac{c_{\text{var}}}{Y}. \tag{12.3}$$

At $Y = 0.95$ and $c_{\text{var}} = \$480$:

$$c_{\text{eff}} = \frac{480}{0.95} \approx \$505 \text{ CAD per shipped unit.}$$

The \$25 difference appears small; at $N = 500$ shipped units it represents an additional \$12,500 in unrecovered production cost.

12.6.1 Cost of a Design Defect Found Late

Figure 12.3 illustrates the **cost-committed curve**: the cumulative fraction of lifetime cost that is locked in at each project phase, plotted against the cost actually incurred.

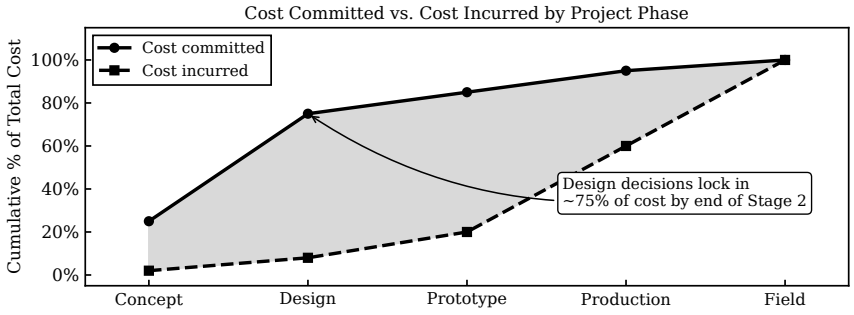

Figure 12.3: Cost committed vs. cost incurred over the five Stage-Gate phases. By the end of Stage 3 (Development), approximately 75% of lifetime manufacturing cost is locked in, yet only 15–20% of total project spend has occurred. The gap between the two curves is the economic justification for intensive front-end design work.

A defect found in Stage 3 (a DFA violation, an over-toleranced component, a housing that requires support material) costs a CAD change and a reprint. The same defect found in Stage 4, after the mould tool is cut, costs a tool modification, a delay, and likely a BOM change. The **rule of ten** states that the cost of fixing a defect multiplies by approximately ten at each stage boundary.

Try It Yourself

A student team is developing a wearable environmental sensor. The BOM and process estimates give the following parameters:

- $C_{\text{fixed}} = \$6{,}000$ CAD (PCB test fixture \$1,800; custom enclosure tool \$3,600; NRE \$600).
- $c_{\text{var}} = \$35$ CAD per unit (components \$22, assembly labour \$13).
- Selling price $p = \$199$ CAD.

(a) Using Equation 12.1, compute C_{unit} at $N = 1$, $N = 25$, $N = 100$, and $N = 500$. Show every arithmetic step.

(b) At what production volume does the fixed-cost contribution per unit fall below \$20? (Set up the inequality $C_{\text{fixed}}/N < 20$ and solve for N.)

(c) The team is comparing hand-assembly (soldering iron, jig) against contract-manufacturer reflow:

- Hand-assembly: $C_{f1} = \$200$ (jig), $c_{v1} = \$58$ CAD.
- CM reflow: $C_{f2} = \$1{,}800$ (setup and stencil), $c_{v2} = \$22$ CAD.

Use Equation 12.2 to find N^*. Round to the nearest whole unit. State the decision rule.

(d) Yield testing shows 92 % of assembled units pass first-pass inspection ($Y = 0.92$). Compute the effective variable cost c_{eff} using Equation 12.3. How does this change the gross margin at $N = 100$?

Review Questions

1. **(Conceptual)** Explain in one paragraph why a prototype unit cost cannot be used directly as the production unit cost. Identify at least two cost categories that change between the prototype build and a production run of 100 units.
2. **(Conceptual)** The design-locks-cost principle states that 75 % of manufacturing cost is committed by the end of Stage 3. What practical implication does this have for when DFM and DFA reviews should be conducted? Why is it economically preferable to discover an assembly violation at Stage 3 rather than Stage 4?
3. **(Applied — calculation)** A product has $C_{\text{fixed}} = \$12{,}000$ CAD and $c_{\text{var}} = \$75$ CAD. Compute C_{unit} at $N = 20$, $N = 80$, and $N = 400$. At what volume does the fixed-cost contribution per unit drop below \$50?
4. **(Applied — calculation)** Two enclosure processes are available:

   - Laser-cut acrylic: $C_{f1} = \$0$, $c_{v1} = \$95$ CAD.
   - Vacuum-formed HDPE: $C_{f2} = \$3{,}500$ CAD, $c_{v2} = \$18$ CAD.

   Use Equation 12.2 to calculate N^*. Show full algebra. For a planned run of 60 units, which process minimises total cost, and by how much?
5. **(Multi-step)** A hardware team projects $C_{\text{fixed}} = \$5{,}000$ CAD, $c_{\text{var}} = \$120$ CAD, selling price $p = \$350$ CAD, and yield $Y = 0.88$.

   (a) Compute the effective variable cost c_{eff}.

(b) Compute C_{unit} at $N = 50$ using c_{eff} rather than c_{var}.

(c) Compute the gross margin $((p - C_{\text{unit}})/p)$ at $N = 50$.

(d) How many units must be produced for the gross margin to reach 40 %? *Hint:* Set $(p - C_{\text{unit}})/p = 0.40$ and solve for N, using c_{eff} in the expression for C_{unit}.

Portfolio Entry

The deliverable for this chapter is a **Preliminary Cost Analysis** document that will feed directly into the Chapter 17 financial model. The document must contain the following elements:

(a) **Parameter table.** State C_{fixed}, c_{var}, selling price p, and yield Y with sources for each value.

(b) **Unit-cost table.** C_{unit} at $N = 50$, $N = 100$, and $N = 500$, computed from Equation 12.1. For the sorting subsystem:

N	C_{unit}	Gross margin
50	\$640 CAD	46.7 %
100	\$560 CAD	53.3 %
500	\$496 CAD	58.7 %

(c) **Process break-even.** Identify the competing processes, state C_{f1}, C_{f2}, c_{v1}, c_{v2}, compute N^* using Equation 12.2, and state the decision rule.

(d) **DFM/DFA flags.** List at least two design decisions from your Stage 3 review (Chapter 7) that affect c_{var}, and quantify the per-unit and per-run impact at $N = 100$.

(e) **Gate 3 recommendation.** One sentence: given the unit-cost curve and the selling price, does the economics support proceeding to Stage 4?

Chapter Summary

Low-volume production economics turns on a single equation: fixed costs spread over the run, variable costs paid per unit, yielding a hyperbolic unit-cost curve that falls steeply below 50 units and flattens toward c_{var} above 100. Fixed costs (C_{fixed}) are paid once and spread over the run; variable costs (c_{var}) scale with every unit. Their combination produces the unit-cost curve (Equation 12.1), a hyperbola that falls steeply at low volume and flattens toward c_{var} at high volume. For the sorting subsystem ($C_{\text{fixed}} = \$8{,}000$ CAD, $c_{\text{var}} = \$480$ CAD), unit cost drops from \$640 at 50 units to \$496 at 500 units — a 22 % improvement that is already 97 % of the maximum possible gain from volume scaling alone.

Process break-even (Equation 12.2) showed that injection moulding becomes cheaper than FDM printing at $N^* = 15$ units for the housing, but that N^* should be treated as a zone rather than a sharp switch.

The design-locks-cost principle connects this chapter directly to Chapter 7: DFM and DFA decisions made at Stage 3 set c_{var} for the lifetime of the product. The cost-committed curve confirms that front-end design investment is the highest-return activity available to the team.

You now have a Preliminary Cost Analysis with C_{unit} at $N = 50$, $N = 100$, and $N = 500$, a process break-even decision, and a Gate 3 recommendation. Chapter 13 takes that full design package and shows how to establish a version-controlled design history file — the Configuration Management Record — so that every artefact can be recovered, compared, and audited as the design evolves toward the Stage 4 release.

CHAPTER 13

VERSION CONTROL & DESIGN HISTORY

13.0.0.0.1 Where this fits. Chapter 12 delivered the cost analysis for the IoT robotic sorting subsystem — unit cost at $N = 50$, 100, and 500, break-even volume, and the economic case to proceed. This chapter sits at the same Stage 3 (Development) gate, where those cost projections must be backed by an auditable design record. Gate 3 requires a complete design package — schematics, firmware, CAD models, BOM, and test records — that a reviewer (or a future team member) can evaluate, reproduce, and certify. A design package without a traceable history is not a package; it is a collection of files. This chapter provides the tools and discipline to turn that collection into an auditable record.

13.0.0.0.2 Learning Objectives. By the end of this chapter you will be able to:

1. apply git branching, release tagging, and git-lfs configuration to a hardware design repository containing firmware, schematic, and CAD artifacts;

2. write meaningful commit messages and manage branches for a hardware design project, including tagging milestone releases; and

3. identify the purpose and required contents of a Design History File (DHF) and construct a DHF index that maps each required record category to its repository location.

13.1 Why Version Control Matters for Hardware

Software developers absorb version control as a survival skill early in their careers. Hardware engineers have historically been slower to adopt it, partly because the dominant artifacts — schematics, PCB layouts, mechanical drawings — are binary files that do not diff cleanly, and partly because "it worked yesterday" is a powerful disincentive to formalising a process that feels like overhead.

Both objections dissolve under Gate pressure.

A change without a record is indistinguishable from an error. When a reviewer at Gate 3 finds that the BOM lists a 4.7 kΩ resistor and the schematic shows a 10 kΩ resistor, there are exactly two possibilities: someone made an unauthorised change, or a change was made and not recorded. Without version history, the team cannot tell which. Both outcomes are treated the same way by a certification auditor: as evidence of an uncontrolled process.

13.1.1 Design History as Design Evidence

In regulated industries (medical devices under ISO 13485, aerospace under AS9100, automotive under IATF 16949), the **design history** is not a nice-to-have supplement to the design; it is the design's proof of existence. A product cannot be certified based on what it currently is; it is certified based on how it got there — what decisions were made, when, by whom, in response to what evidence.

Even outside regulated industries, the same logic applies at Gate 3 of a student project:

- A reviewer asks, "Why did you change the motor driver in Sprint 4?" Without a record, the answer is "I think we had noise issues" — not defensible. With a version history, the answer is commit `b3c7a1f`: *"fix(motor): replace DRV8833 with TB6612 to eliminate PWM-induced MQTT packet loss (see test log 2026-04-28)"* — citable, traceable, and credible.
- A teammate who joined mid-project needs to understand the current state without interrupting the rest of the team. The commit log is the onboarding document.
- Future you — reproducing this design six months after submission — has no memory of which firmware version ran during the Gate 3 demo. A release tag does.

13.1.2 Scope: What Needs Version Control

For the IoT robotic sorting subsystem, every artifact that describes the design or its performance must be under version control:

- **Firmware** — all source files, build scripts, configuration headers, and dependency manifests (`CMakeLists.txt`, `platformio.ini`, `requirements.txt`).
- **Schematics and PCB layouts** — KiCad project files (`.kicad_sch`, `.kicad_pcb`), plus exported PDFs and Gerber archives as release artifacts.
- **CAD models** — Fusion 360 version history internally; exported STEP and STL files committed as release artifacts.
- **Documentation** — specifications (Chapter 4 artifact), BOM (Chapter 11 artifact), cost analysis (Chapter 12 artifact), test logs, and this chapter's DHF index.

13.2 Git for Firmware

Students in this program already use git for software coursework. The discipline transfers directly to firmware; the differences are in commit granularity, branching conventions, and the meaning of a release tag.

13.2.1 Commit Discipline

A commit is an atomic, self-contained unit of change accompanied by a message that explains *why* the change was made. "WIP" is not a commit message. "Fixed stuff" is not a commit message. These messages carry zero information for future readers — worse, they give the illusion that a record exists while providing none.

A well-formed commit message follows three rules:

1. **Imperative mood subject line, $\leq$72 characters.** Write the subject as if completing the sentence "If applied, this commit will..." For example: "add TLS client certificate authentication" not "added TLS" or "adding TLS."

2. **Type prefix.** A short prefix categorises the change: `feat` (new capability), `fix` (corrects a defect), `refactor` (restructures without changing behaviour), `docs` (documentation only), `test` (adds or updates tests), `chore` (tooling, CI, dependency bumps). This prefix lets a reviewer scan the log and immediately understand what category of change each commit represents.

3. **Reference the evidence.** When a commit fixes a problem discovered during testing, name the test log or issue in the message body. "... (see test log 2026-04-28)" turns a commit into a traceable decision record.

13.2.1.0.1 Common Pitfalls.

- **"WIP" as a commit message.** Work-in-progress commits are acceptable on a feature branch during development, but every commit that merges to `develop` or `main` must have a meaningful message. If your current practice is to commit with "WIP" and clean up later, use `git rebase -i` before merging to squash and rename those commits.

- **Committing for backup, not history.** Git is not Dropbox. A commit should represent a coherent state of the design, not merely the last save before leaving the lab. Separate "save" behaviour from "history" behaviour by using `git stash` for in-progress work and committing only when a logical unit is complete.

- **Giant commits.** A single commit that changes the schematic, three firmware modules, the BOM, and the README simultaneously cannot be reverted cleanly. Commit each logical change separately.

13.2.2 Branching Strategy

A branching strategy is a shared agreement about which branches exist, what they contain, and how changes flow between them. For a design project at Stage 3, a lightweight adaptation of the Gitflow model works well. Figure 13.1 illustrates this branch structure with prototype milestones labelled.

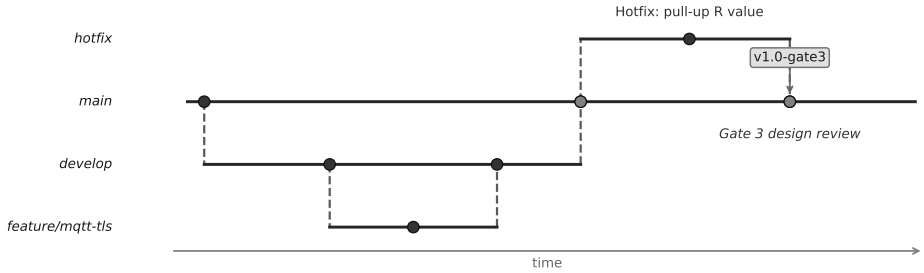

Figure 13.1: Branch and merge diagram for the IoT robotic sorting subsystem design repository. The `main` branch carries only tagged, reviewed releases. Day-to-day integration happens on `develop`. Each new subsystem experiment opens a `feature/` branch that merges back to `develop` after team review. Emergency corrections after a milestone commit flow through a `hotfix/` branch directly to `main` and back to `develop`.

The four branches have distinct roles:

- `main` — the golden branch. Every commit on `main` is a reviewed, tagged release. No one pushes directly to `main`; changes arrive only via a reviewed pull request or hotfix merge.
- `develop` — the integration branch. This is where feature branches land after team review. It represents the current, buildable state of the design. Daily work happens here via merges from feature branches.
- `feature/<name>` — subsystem experiment branches. When a team member is exploring an alternative sensor, a new MQTT library, or a mechanical bracket redesign, they work on a feature branch. This keeps half-finished experiments off `develop` while still providing version control for the experimental work. Examples: `feature/tls-auth`, `feature/tb6612-motor-driver`, `feature/belt-bracket-v2`.
- `hotfix/<name>` — post-release corrections. If a flaw is discovered after a Gate milestone tag, the fix branches from `main`, is applied minimally, and merges back to both `main` (with a new patch tag) and `develop` so the correction propagates forward.

13.2.3 Tagging Releases

A release tag marks a specific commit as a named, reproducible state of the design. Tags are not branches; they do not move as new commits are added. For this course, the tagging convention is:

- `v0.1-proto` — first functional prototype, breadboard or perfboard stage.
- `v0.2-proto` — iterated prototype, subsystems integrated.

- `v1.0-gate3` — the design submitted for Gate 3 review; this tag freezes the exact state of every file the reviewer evaluates.
- `v1.1-post` — post-Gate 3 hotfix, if a defect is discovered after submission.

Tagging the Gate 3 submission is not bureaucratic formality. It means that six months from now, `git checkout v1.0-gate3` recovers the exact design state the reviewer approved. Without the tag, "the version we submitted" is ambiguous.

13.3 Worked Example: Five Commits from the Sorting Subsystem

The following git log covers three weeks of the IoT robotic sorting subsystem project, from an early prototype build through the Gate 3 submission. Read from bottom (oldest) to top (newest).

```
commit a1b2c3d (tag: v1.0-gate3, HEAD -> main)
Author: Team Sorter <team@example.com>
Date:   2026-05-15

    feat(mqtt): add TLS client certificate authentication

    Broker rejects unauthenticated connections per security review
    2026-05-12. Provisioned client cert from Let's Encrypt staging
    CA. See docs/security-review-2026-05-12.md.

commit f4e5d6c (origin/develop, develop)
Merge: 9d8e7f6 b1a2c3d
Author: Team Sorter <team@example.com>
Date:   2026-05-13

    Merge branch 'feature/tls-auth' into develop

    Reviewed by: J. Santos. All MQTT latency tests pass at <= 112 ms
    p99 after TLS handshake amortisation (see test log 2026-05-13).

commit b1a2c3d (feature/tls-auth)
Author: A. Kowalski <a.kowalski@example.com>
Date:   2026-05-10

    feat(mqtt): implement TLS 1.3 handshake on ESP32-S3

    Replaces plaintext MQTT with TLS 1.3 using mbedTLS bundled in
    ESP-IDF v5.2. Latency overhead measured at +18 ms p99 on first
    connect; amortised to +2 ms on subsequent publishes.

commit 9d8e7f6
Author: M. Tremblay <m.tremblay@example.com>
Date:   2026-04-28
```

```
    fix(motor): replace DRV8833 with TB6612 to eliminate
    PWM-induced MQTT packet loss

    DRV8833 switching noise at 20 kHz PWM caused Wi-Fi RF
    desensitisation (+6 dB noise floor on 2.4 GHz). TB6612 tested
    2026-04-27; packet loss dropped from 14% to <1%. BOM updated.
    See test log 2026-04-28.

commit 3c4d5e6 (tag: v0.1-proto)
Author: Team Sorter <team@example.com>
Date:   2026-04-20

    feat(sort): complete first-pass sensor + actuator integration

    Time-of-flight sensor (VL53L1X) and servo divert arm integrated
    on breadboard. Sort accuracy 91% over 50-item run. Known issue:
    PWM noise degrades Wi-Fi (tracked as issue #7).
```

Before commit discipline was applied, these four changes were recorded as: `WIP`, `update mqtt`, `fixed motor`, `done`. None names the subsystem, none states the outcome, none cites evidence. These messages are not archived history; they are noise.

What this log demonstrates:

- Each commit message names a subsystem in parentheses, states an action in the imperative, and (in commits 2 and 4) cites a test log.
- The feature branch `feature/tls-auth` appears as three distinct commits — the branch creation (implicit), the implementation commit, and the merge commit — making the experiment's lifecycle visible in the log.
- Two release tags appear: `v0.1-proto` at first integration, and `v1.0-gate3` at the Gate 3 milestone.
- Commit `9d8e7f6` cross-references issue #7 opened in the `v0.1-proto` commit message, showing that a known defect was logged, tracked, and resolved — the essence of a controlled design process.

13.3.0.1 Reconstructing Design State at Any Commit

The most powerful property of a commit log is reproducibility. To recover the exact state of the sorter firmware, schematic, and documentation as it stood at the `v0.1-proto` tag:

```
git checkout v0.1-proto
```

Git places the working tree in a "detached HEAD" state at that commit. Every file is exactly as it was when that commit was recorded — including the BOM, the firmware, and any exported PDFs committed alongside the source. To inspect the state at an arbitrary commit hash:

```
git checkout 9d8e7f6
```

To return to the current `main`:

```
git checkout main
```

This capability is what converts a repository from a backup system into a design history: the ability to answer "exactly what was the design on 2026-04-28?" with a single command.

13.4 Version Control for Design Files

Firmware is plain text; git handles it natively. Schematic, PCB, and CAD files range from structured text (KiCad's native format) to proprietary binary (older ECAD tools, Fusion 360 native files). Each category requires a slightly different approach.

13.4.1 EDA Files: KiCad and Git

KiCad 7 and later store schematics (`.kicad_sch`) and PCB layouts (`.kicad_pcb`) as structured text files (S-expression format). These files diff legibly — a net renamed, a footprint repositioned, or a trace re-routed produces a human-readable diff. KiCad projects belong in the git repository alongside firmware.

A recommended KiCad project layout within the repository:

```
hardware/
  sorter-pcb/
    sorter-pcb.kicad_pro
    sorter-pcb.kicad_sch
    sorter-pcb.kicad_pcb
    fp-lib-table
    sym-lib-table
    exports/
      sorter-pcb-schematic-v1.0.pdf
      gerbers-v1.0/
```

The `exports/` subdirectory holds generated artifacts committed at each release milestone. Reviewers receive the PDF; the source files allow regeneration.

A `.gitignore` for a KiCad project should exclude autogenerated files that change on every open:

```
# KiCad autogenerated
*.kicad_prl
*-backups/
fp-info-cache
```

13.4.2 CAD Files: Fusion 360 and STEP Exports

Fusion 360 maintains its own cloud version history per design — every save creates a numbered version accessible from the timeline panel. This internal history is not a git repository, but it provides equivalent capability for pure mechanical work: named versions, rollback, and branch-like "branch design" features.

For the purposes of the DHF and Gate 3 package, the key practice is exporting milestone versions as **STEP files** (`.step`) and committing them to the repository:

```
hardware/
  sorter-chassis/
    exports/
      sorter-chassis-v0.1-proto.step
      sorter-chassis-v1.0-gate3.step
```

STEP is a neutral, open exchange format readable by any CAD tool. Committing STEP exports ensures that the Gate 3 package contains manufacturable geometry even if the original Fusion 360 licence expires or the cloud account is unavailable.

13.4.3 Binary Files and Git LFS

Git stores file history by retaining every version of every file. For large binary files — Gerber archives, STEP models, compiled firmware binaries, rendered images — this causes repository bloat: cloning the repository downloads every historical version of every binary, which can quickly reach hundreds of megabytes.

Git Large File Storage (git-lfs) solves this by replacing large binaries in the repository with small pointer files, while storing the actual binary content on a compatible server (GitHub LFS, GitLab LFS, or a self-hosted server). Only the versions actually checked out are downloaded.

Setup requires two steps, run once per repository:

```
git lfs install
git lfs track "*.step" "*.pdf" "*.zip" "*.bin"
git add .gitattributes
git commit -m "chore: configure git-lfs for binary design artifacts"
```

The `git lfs track` command adds patterns to `.gitattributes`. All team members must have git-lfs installed; cloning without it produces pointer-file stubs instead of actual content.

13.4.3.0.1 Common Pitfalls.

- **Committing binary blobs without LFS.** Adding a 15 MB Gerber ZIP to git without LFS embeds that 15 MB permanently in the repository history. `git rm` does

not help; the blob persists in every future clone. Removing it requires rewriting history with `git filter-branch` or the BFG Repo Cleaner, which disrupts all collaborators. Configure LFS *before* committing any binary artifact.

- **Tracking every file type with LFS.** KiCad `.kicad_sch` and `.kicad_pcb` files are structured text and should *not* go through LFS — their diffs are the most valuable part of the schematic history. Only true binaries and large generated outputs belong in LFS.

13.5 The Design History File

The **Design History File (DHF)** is the auditable record of all design decisions, changes, approvals, and test results associated with a product. In regulated product development, the DHF is a formal document set required for certification; a product without a DHF cannot be cleared, approved, or certified.

In this course, the **portfolio is the DHF**. Every artifact produced across Chapters 1 through 14 — the central question, the specification, the risk register, the architecture diagram, the BOM, the cost analysis, and the test records — collectively constitutes the design history. The DHF index page produced in this chapter is the document that proves those artifacts exist, locates them in the repository, and shows that they are current.

13.5.1 Required Contents of a DHF

A complete DHF contains four categories of records. Figure 13.2, shown after the list, illustrates how these records feed the Gate 3 package.

1. **Design records** — the specification (Chapter 4), architecture diagram (Chapter 5), schematic and PCB files, CAD models, firmware source, and BOM (Chapter 11). These describe *what the design is*.
2. **Test records** — bench test logs, latency characterisation results, accuracy trial data, environmental tests. These describe *what was observed*.
3. **Change records** — Engineering Change Orders (Section 13.6) or, in this course, annotated git commit messages and merge records. These describe *what changed and why*.
4. **Approval records** — peer review sign-offs, instructor feedback records, and Gate decision documents. These describe *who reviewed and accepted each change*.

13.5.2 The DHF Index

The DHF index is a single document — typically a Markdown or LaTeX file at the root of the repository — that maps each required record to its location. Its purpose is

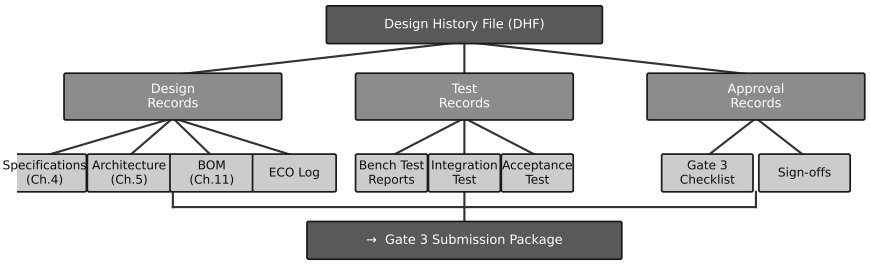

Figure 13.2: The Design History File as a tree of records feeding into the Gate 3 document. Design records capture what was decided; test records capture what was observed; change records capture what was modified and why; approval records capture who reviewed and accepted each change. Together they form the traceable chain of evidence required for Gate 3.

navigation: a reviewer should be able to open the index and locate any specific record within two clicks or two commands.

A minimal DHF index for the sorting subsystem project:

```
# Design History File Index - IoT Robotic Sorting Subsystem
# Gate 3 Submission: tag v1.0-gate3 (2026-05-15)

## Design Records
- Specification (REQ-01 to REQ-05):
    docs/specification.md  (commit 3c4d5e6)
- Architecture diagram:
    docs/architecture.pdf  (commit 7a8b9c0)
- Schematic (KiCad):
    hardware/sorter-pcb/sorter-pcb.kicad_sch
- BOM v1.0:
    hardware/bom-v1.0.csv  (commit e2f3a4b)
- Firmware source:
    firmware/src/          (tag v1.0-gate3)

## Test Records
- MQTT latency characterisation (2026-05-13):
    docs/test-logs/mqtt-latency-2026-05-13.md
- Sort accuracy trial, 100-item run (2026-05-08):
    docs/test-logs/sort-accuracy-2026-05-08.md

## Change Records
- Motor driver substitution (DRV8833 -> TB6612):
    git log 9d8e7f6
- TLS authentication addition:
    git log b1a2c3d

## Approval Records
- Gate 3 review:
    docs/gate3-review-2026-05-16.md
```

Every entry names the file and the commit or tag at which that file was current.

A future reviewer can `git checkout` the named commit and retrieve the exact version of each record that was present at Gate 3.

13.5.2.0.1 In Practice. Teams who maintain the DHF index throughout the project — updating it whenever a new artifact is produced — find Gate 3 preparation trivial: the index already exists, the records are already filed, and the submission is a matter of tagging the repository and attaching the index. Teams who leave the DHF index for the last week before Gate 3 discover that they cannot locate test logs, cannot match BOM versions to schematic revisions, and cannot reconstruct the decision trail that led to the current design. The DHF is not preparation for Gate 3; it is the continuous record that makes Gate 3 possible.

13.6 Change Management

Every design changes during development. The question is not whether changes will happen but whether they will be recorded. Change management is the discipline of deciding, for each change, whether it requires a formal record and, if so, ensuring that record is created.

13.6.1 Informal Fix vs. Formal ECO

Not every change requires an Engineering Change Order. A typo correction in a comment, a cosmetic alignment fix in a schematic, or a variable rename in firmware can be committed directly with an appropriate message. The decision threshold is whether the change affects form, fit, or function:

- **Form** — physical dimensions, connector locations, mounting hole patterns.
- **Fit** — how the component or assembly interfaces with adjacent parts.
- **Function** — what the system does, including electrical characteristics, software behaviour, or communication protocol parameters.

Any change that affects form, fit, or function in a way that could alter test results or require retesting triggers a formal change record. Figure 13.3 maps this decision.

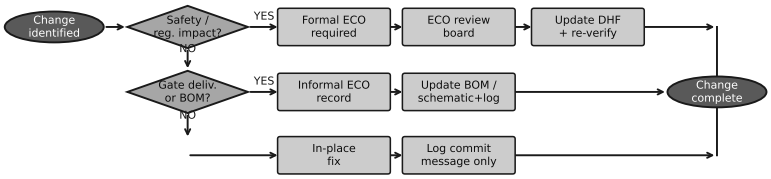

Figure 13.3: Engineering change order decision flow. Changes that do not affect form, fit, or function proceed as a standard commit. Changes that affect form, fit, or function require a documented rationale and, if the design is post-release (past a Gate milestone tag), a formal ECO with reviewer sign-off before implementation.

13.6.2 The ECO in Practice

A formal Engineering Change Order contains:

1. **Change ID and date** — a unique identifier (e.g., ECO-003) and the date the change was initiated.
2. **Description of the change** — what is being changed, from what to what, in each affected artifact (schematic, firmware, BOM, documentation).
3. **Reason for the change** — the evidence or observation that motivated it (test log reference, defect report, customer feedback, new component availability).
4. **Impact assessment** — which other subsystems, tests, or documents are affected; whether retesting is required.
5. **Approval** — who authorised the change; date authorised.
6. **Implementation reference** — the git commit hash or tag at which the change was applied.

In this course, the ECO is implemented as a short Markdown file committed to `docs/eco/ECO-003.md` with the above fields. The git commit that implements the change references the ECO number in its message: `fix(motor): replace DRV8833 per ECO-003`.

13.6.2.0.1 In Practice. A team made a last-minute resistor value change without recording it. The prototype passed testing, went to Gate 3, and the reviewer asked why the BOM listed 4.7 kΩ and the schematic showed 10 kΩ. No one could answer. The change had been made on the bench during a late-night session, the schematic was not updated, and no commit was made. From the reviewer's perspective, the design was incoherent: the BOM and schematic described two different circuits. An ECO process — or even a committed git message reading *"fix(adc-bias): change R12 from 10k to 4.7k to correct voltage divider for 3.3V ADC reference (see test-log-2026-05-09)"* — would have caught this. A 60-second commit prevents a Gate-day crisis.

13.7 Practical Tooling

This section summarises the tool stack for the sorting subsystem project and provides configuration guidance.

13.7.1 Git and GitHub/GitLab

The repository should be hosted on a platform that supports:

- **Pull requests / merge requests** — the mechanism for team code review before merging feature branches to `develop`.

- **Issues** — the lightweight defect and task tracker; commit messages reference issue numbers (`closes #7`).
- **Git LFS** — large binary storage; both GitHub and GitLab offer LFS storage with free-tier quotas sufficient for a student project.
- **Releases** — GitHub and GitLab both surface git tags as named releases with attached assets; tagging `v1.0-gate3` automatically populates the Releases page, providing a clean download link for the Gate 3 submission package.

A recommended repository root structure for the sorter project:

```
sorter-project/
  firmware/
    src/
    include/
    platformio.ini
  hardware/
    sorter-pcb/          (KiCad project)
    sorter-chassis/
      exports/           (STEP, STL)
  docs/
    specification.md
    architecture.pdf
    bom-v1.0.csv
    test-logs/
    eco/
    dhf-index.md
  .gitattributes         (git-lfs patterns)
  .gitignore
  README.md
```

13.7.2 KiCad Git Integration

KiCad 7's native file format is git-friendly by design. The recommended workflow:

1. Open the KiCad project folder as the git working directory (not as a subdirectory of a larger project with unrelated files).
2. Add the `.gitignore` exclusions for `*.kicad_prl` and `*-backups/` (KiCad auto-generates these on every open).
3. Commit schematic and PCB changes separately from firmware changes. A single commit that mixes a schematic net change with a firmware fix is harder to review and harder to revert.
4. Export schematic PDF and BOM CSV at each release milestone and commit them to `hardware/sorter-pcb/exports/` before tagging.

13.7.3 Fusion 360 Version History

Fusion 360 saves version history to Autodesk cloud storage automatically on each save. Each version can be named (`v0.1-proto-belt-bracket`, `v1.0-gate3-chassis`) from the Version History panel. This naming should mirror the git tag convention so that a reviewer can correlate the mechanical version with the corresponding firmware and schematic commit.

The synchronisation point is the STEP export committed to the git repository at each release tag. The process:

1. In Fusion 360, name the current version to match the planned git tag (e.g., `v1.0-gate3`).
2. Export as STEP: `File > Export > STEP (.step)`.
3. Place the STEP file in `hardware/sorter-chassis/exports/` and commit it to git with the message `chore(cad): export v1.0-gate3 STEP for DHF`.
4. Tag the repository: `git tag -a v1.0-gate3 -m "Gate 3 submission"`.

13.7.4 Keeping the DHF in the Repository

The DHF index (`docs/dhf-index.md`) lives in the same repository as the design. This means:

- The DHF is versioned alongside the design it documents.
- `git checkout v1.0-gate3` retrieves the DHF index as it stood at Gate 3 submission, listing the exact file versions that were current at that moment.
- There is no separate document management system to synchronise; the repository *is* the document management system.

Try It Yourself

> **Scenario:** A team is developing a **smart aquarium controller** that monitors water temperature, pH, and turbidity, controls a heating element and a peristaltic dosing pump, and publishes sensor readings to a local MQTT broker over Wi-Fi. Their git log for the most recent four commits reads:

```
commit d4e5f6a (HEAD -> main)
Author: Aqua Team <aqua@example.com>
Date:   2026-05-10

    stuff

commit 9b8c7d6
Author: Aqua Team <aqua@example.com>
```

```
Date:   2026-05-09

    WIP sensor code

commit 1a2b3c4
Author: Aqua Team <aqua@example.com>
Date:   2026-05-07

    update

commit 5e6f7a8
Author: Aqua Team <aqua@example.com>
Date:   2026-05-03

    feat(temp): add DS18B20 one-wire temperature sampling at 1 Hz

    Sampling loop validated against NIST-traceable reference
    thermometer; error < 0.2 degC across 15-35 degC range.
    See test-log-2026-05-03.md.
```

Your tasks:

1. Identify which commits have poorly written messages and explain specifically what information is missing from each.
2. Rewrite the three poorly written commit messages following the type-prefix, imperative mood, and evidence-citation conventions introduced in Section 13.2.1. Use plausible content consistent with the aquarium controller context. Do not invent test results; instead reference a test log by filename.
3. The team has a STEP export of their sensor bracket committed without git-lfs (`sensor-bracket.step`, 22 MB). Address the following:
   (a) How to prevent future large files from bypassing LFS.
   (b) Why the existing 22 MB blob cannot be removed with `git rm` alone and what tool is required.
4. Sketch (in plain text or a simple diagram) a DHF index for the aquarium controller at a Gate 3 milestone, listing at least one entry in each of the four DHF categories from Section 13.5.1.

Review Questions

1. **(Conceptual)** Explain the statement "a change without a record is indistinguishable from an error." A team's prototype passes all Gate 3 bench tests, but the reviewer flags a discrepancy between the BOM and the schematic. What two possibilities does the reviewer face, and why can the team not resolve the ambiguity without version history?
2. **(Conceptual)** Compare the roles of the `develop` branch and the `main` branch in the Gitflow branching strategy described in this chapter. Why should direct pushes to `main` be prohibited, and what mechanism should be used instead?

3. **(Applied)** The following four commit messages appear in a project repository. For each, state whether it is acceptable or poor, and if poor, rewrite it following the conventions in Section 13.2.1.

   (a) `WIP`

   (b) `feat(lidar): add VL53L1X ranging loop with 50 ms sampling interval, validated against tape-measure reference over 0.1-2.0 m (see test-log-2026-04-15.md)`

   (c) `fixed the thing`

   (d) `updated files`

4. **(Applied)** A teammate commits a 40 MB Gerber ZIP file to the repository without git-lfs.

   (a) Explain why `git rm gerbers.zip && git commit` does not reduce the repository size for new clones.

   (b) Describe the correct remediation steps, including which external tool is commonly used to rewrite repository history and what the operational consequence is for all other team members after the history rewrite.

   (c) State the `git lfs track` command that would have prevented this problem for all future ZIP files.

5. **(Multi-step — conceptual + applied)** A team completes Gate 3. One week later, a reviewer discovers that the pulse-width modulation frequency used in the actuator firmware was changed from 20 kHz to 10 kHz during the final sprint but does not appear in any commit message, the BOM, the schematic, or the DHF index.

   (a) Identify which of the four DHF record categories (Section 13.5.1) is missing for this change.

   (b) Describe how an ECO record, had it been created, would have captured this change, naming the six fields from Section 13.6.2.

   (c) Explain whether the existing `v1.0-gate3` tag can be used to reconstruct the state of the firmware at Gate 3 submission, and what additional step would now be required to document the discrepancy without destroying the Gate 3 record.

Portfolio Entry

This chapter produces two linked deliverables that together constitute the version-control component of your Gate 3 package:

1. **Git repository with a meaningful commit log.** Your project repository on GitHub or GitLab must contain:

   (a) A `main/develop/feature` branch structure consistent with the strategy in Section 13.2.2.

   (b) At least five commits with meaningful messages following the type-prefix and imperative-mood conventions, covering the design period from your first prototype integration to Gate 3.

(c) At least one feature branch that was created for a subsystem experiment and merged back to `develop` via a pull request.

(d) A release tag `v1.0-gate3` applied to the commit that represents your Gate 3 submission state.

2. **DHF index page.** A file `docs/dhf-index.md` committed to the repository at the `v1.0-gate3` tag, containing:

   (a) At least one entry in each of the four DHF record categories (design records, test records, change records, approval records).

   (b) For each entry: the filename (relative to the repository root), the git commit hash or tag at which that file was current, and a one-line description of the record's content.

   (c) The full command required to reproduce the Gate 3 submission state: `git checkout v1.0-gate3`.

Chapter Summary

A design without version control is not under control; it is under the assumption that nothing will go wrong. This chapter established three interlocking disciplines.

First, every design artifact — firmware, schematic, PCB layout, CAD model, BOM, and documentation — belongs in a version-controlled repository. Design history is design evidence: reviewers certify not what a product currently is, but how it got there.

Second, commit discipline and branching strategy are not software-only concerns. Meaningful commit messages in imperative mood with type prefixes and evidence citations turn a git log into a design decision trail. The `main`/`develop`/`feature` branch structure keeps experimental work isolated until it is reviewed and ready to integrate. Release tags — `v0.1-proto`, `v1.0-gate3` — freeze exact design states that can be recovered with a single `git checkout` command.

Third, the Design History File is not a document produced at the end of a project; it is a continuous record maintained throughout the project. The DHF index maps design records, test records, change records, and approval records to their repository locations. The ECO process — deciding whether a change affects form, fit, or function, and recording it if it does — is the mechanism that keeps the DHF current. A 60-second commit referencing a test log prevents the Gate-day crisis of a BOM that disagrees with a schematic.

The chapter produces a git repository with a meaningful commit log covering the design period, and a DHF index page linking design records, test records, change records, and approval records to their commits. Chapter 14 takes that documented design and validates it against the acceptance criteria established in Chapter 4 — turning the record of what was built into proof that it works.

INDEX